

TWN4

API Reference

DocRev31, April 15, 2026



ELATEC GmbH

Contents

1	Definitions and Function Prototypes	3
2	System Functions	4
2.1	SysCall	4
2.2	Reset	4
2.3	StartBootloader	4
2.4	GetSysTicks	4
2.5	GetVersionString	5
2.6	GetUSBType	5
2.7	GetDeviceType	6
2.8	Sleep	6
2.9	GetDeviceUID	7
2.10	SetParameters	7
2.11	GetLastError	8
3	Interrupt System	9
3.1	SetInterruptHandler	9
3.2	GPIOConfigureInterrupt	11
4	I/O Functions	12
4.1	Configuration	12
4.1.1	Set COM-Port Parameters	12
4.1.2	Get USB Device State	13
4.1.3	Get Host Channel	13
4.1.4	Set Host Channel	13
4.2	Miscellaneous Functions	14
4.2.1	Wake Up Host	14
4.3	Data I/O	14
4.3.1	Query I/O Buffer Size	14
4.3.2	Get I/O Buffer Byte Count	14
4.3.3	Test Empty	15
4.3.4	Test Full	15
4.3.5	Send Byte	16
4.3.6	Send Multiple Bytes	16
4.3.7	Read Byte	17
4.3.8	Read Multiple Bytes	17
5	Memory Functions	18
5.1	Byte Operations	18
5.1.1	Compare Bytes	18
5.1.2	Copy Bytes	18
5.1.3	Fill Bytes	19
5.1.4	Swap Bytes	19
5.2	Bit Operations	19
5.2.1	Read Bit	19
5.2.2	Write Bit	20
5.2.3	Copy Bit	20
5.2.4	Compare Bits	21

5.2.5	Copy Bits	21
5.2.6	Fill Bits	22
5.2.7	Swap Bits	22
5.2.8	Count Bits	23
6	Peripheral Functions	24
6.1	General Purpose Inputs/Outputs (GPIOs)	24
6.1.1	Configuration	24
6.1.1.1	Outputs	24
6.1.1.2	Inputs	24
6.1.2	Basic Port Functions	25
6.1.2.1	Set GPIOs to Logical Level	25
6.1.2.2	Toggle GPIOs	25
6.1.2.3	Waveform Generation	25
6.1.2.4	Read GPIOs	26
6.1.3	Higher Level Port Functions	26
6.1.3.1	Send Data in Wiegand Format	26
6.1.3.2	Send Data in Omron Format	28
6.2	Beeper	29
6.2.1	Duration controlled by function	29
6.2.2	Duration controlled by App	29
6.3	LEDs	30
6.3.1	General Purpose LED Functions	30
6.3.1.1	Initialization	30
6.3.1.2	Set LEDs On/Off	30
6.3.1.3	Toggle LEDs	30
6.3.1.4	Blink LEDs	31
6.3.2	Diagnostic LED	31
6.3.2.1	Set Diagnostic LED On/Off	31
6.3.2.2	Toggle Diagnostic LED	31
6.3.2.3	Get LED State	32
7	Conversion Functions	33
7.1	Hexadecimal ASCII to Binary	33
7.1.1	Scan Hexadecimal Character	33
7.1.2	Scan Hexadecimal String	33
7.2	Binary to Hexadecimal ASCII	34
8	I2C Functions	35
8.1	Initialization/Deinitialization	35
8.1.1	I2CInit	35
8.1.2	I2CDeInit	35
8.1.3	Examples	35
8.2	Communication (Master)	35
8.2.1	I2CMasterStart	35
8.2.2	I2CMasterStop	36
8.2.3	I2CMasterTransmitByte	36
8.2.4	I2CMasterReceiveByte	36
8.2.5	I2CMasterBeginWrite	36
8.2.6	I2CMasterBeginRead	36
8.2.7	I2CMasterSetAck	37
8.2.8	Examples	37
8.3	Communication (Slave)	37
8.3.1	Slave to Master	38

8.3.2	Master to Slave	38
8.3.3	Examples	38
9	SPI Functions	40
9.1	Slave Mode	40
9.1.1	Timing	40
9.1.2	Communication Protocol	41
9.1.2.1	Master to Slave	41
9.1.2.2	Slave to Master	41
9.1.2.3	Example of a SPI Host Implementation	41
9.2	Initialization/Deinitialization	42
9.2.1	SPIInit	42
9.2.2	SPIDeInit	43
9.3	Communication	43
9.3.1	SPIMasterBeginTransfer	43
9.3.2	SPIMasterEndTransfer	44
9.3.3	SPITransceive	44
9.3.4	SPITransmit	44
9.3.5	SPIReceive	44
9.4	Examples	45
10	RF Functions	46
10.1	SearchTag	46
10.2	SetRFOff	46
10.3	SetTagTypes	46
10.3.1	Supported Types of LF Tags (125 kHz - 134.2 kHz)	47
10.3.2	Supported Types of HF Tags (13.56 MHz, Bluetooth)	47
10.4	GetTagTypes	48
10.5	GetSupportedTagTypes	48
11	EM4x02-Specific Transponder Operations	49
11.1	Function	49
11.1.1	EM4102_GetTagInfo	49
12	HITAG 1- and HITAG S-Specific Transponder Operations	50
12.1	Read/Write Data	50
12.1.1	Hitag1S_ReadPage	50
12.1.2	Hitag1S_WritePage	50
12.1.3	Hitag1S_ReadBlock	51
12.1.4	Hitag1S_WriteBlock	51
12.2	Hitag1S_Halt	51
13	HITAG 2-Specific Transponder Operations	53
13.1	Read/Write Data	53
13.1.1	Hitag2_ReadPage	53
13.1.2	Hitag2_WritePage	53
13.1.3	Hitag2_SetPassword	54
13.2	Hitag2_Halt	54
14	EM4x50-Specific Transponder Operations	55
14.1	Functions	55
14.1.1	EM4150_Login	55
14.1.2	EM4150_ReadWord	55
14.1.3	EM4150_WriteWord	55
14.1.4	EM4150_WritePassword	56
14.1.5	EM4150_GetTagInfo	56

15	EM4305-Specific Transponder Operations	57
15.1	Functions	57
15.1.1	EM4305_Begin	57
15.1.2	EM4305_Read	58
15.1.3	EM4305_Write	58
15.1.4	EM4305_Login	58
15.1.5	EM4305_Protect	58
15.1.6	EM4305_Disable	59
16	AT55xx-Specific Transponder Operations	60
16.1	Control Functions	60
16.1.1	AT55_Begin	60
16.2	Read Data	60
16.2.1	AT55_ReadBlock	61
16.2.2	AT55_ReadBlockProtected	61
16.3	Write Data	61
16.3.1	AT55_WriteBlock	61
16.3.2	AT55_WriteBlockProtected	62
16.3.3	AT55_WriteBlockAndLock	62
16.3.4	AT55_WriteBlockProtectedAndLock	62
17	TILF (TIRIS) Functions	63
17.1	Search Function	63
17.1.1	TILF_SearchTag	63
17.2	Single-Page Read/Write Function	63
17.2.1	TILF_ChargeOnlyRead	63
17.2.2	TILF_ChargeOnlyReadLo	64
17.2.3	TILF_SPPProgramPage	64
17.2.4	TILF_SPPProgramPageLo	64
17.3	Multi-Page Read/Write Function	65
17.3.1	TILF_MPGeneralReadPage	65
17.3.2	TILF_MPSelectiveReadPage	65
17.3.3	TILF_MPPProgramPage	65
17.3.4	TILF_MPSelectiveProgramPage	65
17.3.5	TILF_MPLockPage	66
17.3.6	TILF_MPSelectiveLockPage	66
17.3.7	TILF_MPGeneralReadPageLo	66
17.3.8	TILF_MPSelectiveReadPageLo	67
17.3.9	TILF_MPPProgramPageLo	67
17.3.10	TILF_MPSelectiveProgramPageLo	67
17.3.11	TILF_MPLockPageLo	68
17.3.12	TILF_MPSelectiveLockPageLo	68
17.4	Multi-Usage Read/Write Function	68
17.4.1	TILF_MUGeneralReadPage	68
17.4.2	TILF_MUSelectiveReadPage	69
17.4.3	TILF_MUSpecialReadPage	69
17.4.4	TILF_MUProgramPage	69
17.4.5	TILF_MUSelectiveProgramPage	70
17.4.6	TILF_MUSpecialProgramPage	70
17.4.7	TILF_MULockPage	71
17.4.8	TILF_MUSelectiveLockPage	71
17.4.9	TILF_MUSpecialLockPage	71

18 ISO14443 Transponder Operations	72
18.1 ISO14443A	72
18.1.1 Get ATQA	72
18.1.2 Get SAK	72
18.1.3 Get ATS	72
18.2 ISO14443B	73
18.2.1 Get ATQB	73
18.2.2 Get Answer to ATTRIB	73
18.3 Check Presence	74
18.4 ISO14443-3 Transparent Data Exchange	74
18.5 ISO14443-4 Transparent Data Exchange	75
18.6 Multiple Tag Handling	75
18.6.1 Search for Transponders	76
18.6.2 Select Transponder	76
19 MIFARE Classic Specific Transponder Operations	77
19.1 Login	78
19.2 Read/Write Data	79
19.2.1 Read Data Block	79
19.2.2 Write Data Block	79
19.3 Handling of Value Blocks	80
19.3.1 Read Value Block	80
19.3.2 Write Value Block	80
19.3.3 Increment Value Block	80
19.3.4 Decrement Value Block	81
19.3.5 Copy Value Block	81
20 MIFARE Plus Specific Transponder Operations	82
20.1 Personalisation	82
20.1.1 Write Personalisation	82
20.1.2 Commit Personalisation	83
20.2 Authenticate AES	83
20.3 Security Level 3	84
20.3.1 Read/Write Data	84
20.3.1.1 Read Data Block	84
20.3.1.2 Write Data Block	84
20.3.2 Handling of Value Blocks	85
20.3.2.1 Read Value Block	85
20.3.2.2 Write Value Block	85
20.3.2.3 Increment Value Block	85
20.3.2.4 Decrement Value Block	86
20.3.2.5 Copy Value Block	86
21 MIFARE Ultralight/Ultralight C/Ultralight EV1 Specific Transponder Operations	87
21.1 Authentication (Ultralight C only)	87
21.1.1 Authentication with given Key	87
21.1.2 Authentication using SAM Card	88
21.2 Write Key from SAM to Transponder Key Storage Area	88
21.3 Read/Write Data	89
21.3.1 Read Page	89
21.3.2 Write Page	89
21.4 Mifare Ultralight EV1	90
21.4.1 Fast Read	90
21.4.2 Increment Counter	90

21.4.3 Read Counter	90
21.4.4 Read ECC Signature	91
21.4.5 Get Transponder Information	91
21.4.6 Password Authentication	91
21.4.7 Check Tearing Event	92
22 NTAG Specific Transponder Operations	93
22.1 Read/Write Data	93
22.1.1 Read Page	93
22.1.2 Write Page	93
22.1.3 Fast Read	94
22.2 Miscellaneous functions	94
22.2.1 Read Counter	94
22.2.2 Read ECC Signature	94
22.2.3 Get Transponder Information	95
22.2.4 Password Authentication	95
22.2.5 Select Sector	95
23 DESFire Specific Transponder Operations	96
23.1 Security Related Operations	97
23.1.1 Authenticate	97
23.1.2 Get Key Version	100
23.1.3 Get Key Settings	100
23.1.4 Change Key Settings	102
23.1.5 Change Key	103
23.2 Transponder Related Operations	105
23.2.1 Create Application	105
23.2.2 Delete Application	106
23.2.3 Get Application IDs	106
23.2.4 Select Application	107
23.2.5 Format Transponder	108
23.2.6 Get Transponder Information	108
23.2.7 Get Available Memory Space	109
23.2.8 Get Card UID	110
23.2.9 Set Transponder Configuration	110
23.2.9.1 Disable Format Tag	110
23.2.9.2 Enable Random ID	111
23.2.9.3 Set Default Key	111
23.2.9.4 Set User-defined Answer To Select (ATS)	112
23.3 Application Related Operations	112
23.3.1 Create File	112
23.3.2 Delete File	115
23.3.3 Get File IDs	116
23.3.4 Get File Settings	116
23.3.5 Change File Settings	118
23.4 File Related Operations	118
23.4.1 Data Files	118
23.4.1.1 Read Data	118
23.4.1.2 Write Data	120
23.4.2 Value Files	122
23.4.2.1 Get Value	122
23.4.2.2 Debit	123
23.4.2.3 Credit	124

23.4.2.4 Limited Credit	124
23.4.3 Record Files	125
23.4.3.1 Read Records	125
23.4.3.2 Write Record	126
23.4.3.3 Clear Record File	127
23.4.4 Commit Transaction	127
23.4.5 Abort Transaction	128
24 SAM AV1/AV2	129
24.1 Host Authentication	129
24.2 Query Key Entry	129
25 ISO15693 Specific Transponder Operations	131
25.1 Generic ISO15693 Command	131
25.1.1 Example of creating a Standard ISO15693 Command	131
25.2 Gather Tag Specific Information	132
25.2.1 Get System Information	132
25.2.2 Get Tag Type	133
25.2.2.1 Get Tag Type From UID	133
25.2.2.2 Get Tag Type From System Information	134
25.3 Read/Write Data	136
25.3.1 Read Single Block	136
25.3.2 Write Single Block	136
26 LEGIC-Specific Functions	138
26.1 Direct Access of LEGIC Chip	138
26.1.1 SM4X00_GenericRaw	138
26.1.2 SM4X00_Generic	139
26.1.3 SM4X00_StartBootloader	139
26.1.4 SM4X00_EraseFlash	139
26.1.5 SM4X00_ProgramBlock	140
27 iCLASS Specific Transponder Operations	141
27.1 Read PAC Bits	141
27.1.1 Example	141
27.2 Select Page	142
27.3 Authenticate	142
27.4 Read Data Block	143
27.5 Write Data Block	143
28 FeliCa	145
28.1 Polling	145
28.2 Request System Code	145
28.3 Request Service	146
28.4 Read Without Encryption	146
28.5 Write Without Encryption	147
28.6 Transparent Data Exchange	148
29 Topaz Specific Transponder Operations	149
29.1 Read UID	149
29.2 Read Data	149
29.2.1 Read Single Byte	149
29.2.2 Read All Transponder Data	149
29.3 Write Data	150
29.3.1 Write Single Byte With Erase	150
29.3.2 Write Single Byte Without Erase	150

30	CTS Specific Transponder Operations	152
30.1	Read Block Data	152
30.2	Write Block Data	152
30.3	Update Block Data	153
31	SRX Specific Transponder Operations	154
31.1	Read Block Data	154
31.2	Write Block Data	154
32	BLE Functions	155
32.1	GATT Server Definition	156
32.2	BLEPresetConfig	157
32.3	BLEPresetUserData	158
32.4	BLEInit	160
32.5	BLETxRxModuleCommand	161
32.6	BLEGetEvent	162
32.7	BLECheckEvent	162
32.8	BLEStartAdvertisement	164
32.9	BLEGetAddress	165
32.10	BLEGetVersion	165
32.11	BLEGetEnvironment	166
32.12	BLEGetGattServerAttributeValue	167
32.13	BLESetGattServerAttributeValue	167
32.14	BLERequestRssi	168
32.15	BLERequestEndpointClose	168
32.16	BLEGetGattServerCharacteristicStatus	168
32.17	BLEFindGattServerAttribute	169
32.18	BLEDiscover	169
32.19	BLECheckDiscoveredString	171
32.20	BLEConnectToDevice	172
32.21	BLEDisconnectFromDevice	173
32.22	BLEGattGetAttribute	173
32.23	BLEGattGetValue	174
32.24	BLEGattSetValue	176
32.25	BLECommand	177
32.26	BLESecurity	179
32.27	BLESecuritySetOob	182
32.28	BLESecurityUseScOob	183
32.29	BLESetStreamingMode	184
32.30	BLESetStreamingUUID	185
32.31	BLEGetDiscoveredData	186
33	Contact Card Operations	187
33.1	Microprocessor Cards	187
33.1.1	Query Card Slot Status	187
33.1.2	Card Activation	188
33.1.3	Card Deactivation	189
33.1.4	Set Communication Settings	189
33.1.5	Transparent Data Transmission	192
33.1.6	Exchange Of APDUs	192
33.1.7	Examples	194
33.1.7.1	PPS Example	194
33.1.7.2	Communication Example	194

33.2 SLE Memory Cards	196
33.2.1 Get ATR	196
33.2.2 Read Main Memory	196
33.2.3 Write Main Memory	197
33.2.4 Read Security Memory	197
33.2.5 Write Security Memory	197
33.2.6 Read Protection Memory	198
33.2.7 Write Protection Memory	198
33.2.8 Compare Verification Data	198
33.3 I2C Memory Cards	199
33.3.1 Read Memory	199
33.3.2 Write Memory	199
34 Cryptographic Operations	201
34.1 Initialization	203
34.2 Encrypt	203
34.3 Decrypt	204
34.4 Reset Init Vector	204
35 Storage Functions	205
35.1 Management Functions	205
35.1.1 FSMount	205
35.1.2 FSFormat	206
35.2 File Functions	206
35.2.1 FSOpen	206
35.2.2 FSClose	207
35.2.3 FSCloseAll	208
35.2.4 FSSeek	208
35.2.5 FSTell	208
35.2.6 FSReadBytes	209
35.2.7 FSWriteBytes	209
35.3 Directory Functions	210
35.3.1 FSFindFirst	210
35.3.2 FSFindNext	210
35.3.3 FSDelete	210
35.3.4 FSRename	211
35.4 Miscellaneous Functions	211
35.4.1 FSGetStorageInfo	211
35.5 Examples	212
36 PKOC Specific Transponder Operations	214
36.1 Read NFC Credential	214
36.2 Set NFC Parameters	214
36.3 Set BLE Parameters	215
36.4 Set Site Key	215
37 System Parameters	216
37.1 TLV Format	216
37.2 Manifest	216
37.3 Available Parameters	218
38 System Errors	221
39 Runtime Library	223
39.1 Timer Functions	223
39.1.1 StartTimer	223
39.1.2 StopTimer	223

39.1.3 TestTimer	224
39.2 Host Communication	224
39.2.1 HostTestByte	224
39.2.2 HostReadByte	224
39.2.3 HostTestChar	224
39.2.4 HostReadChar	225
39.2.5 HostWriteByte	225
39.2.6 HostWriteChar	225
39.2.7 HostWriteString	225
39.2.8 HostWriteRadix	226
39.2.9 HostWriteBin	226
39.2.10 HostWriteDec	226
39.2.11 HostWriteHex	227
39.2.12 HostWriteVersion	227
39.3 Beep Functions	227
39.3.1 SetVolume	227
39.3.2 GetVolume	228
39.3.3 BeepLow	228
39.3.4 BeepHigh	228
39.4 Compatibility to TWN3	228
39.4.1 ConvertTagTypeToTWN3	228
39.5 Simple Protocol	229
39.5.1 SimpleProtoInit	229
39.5.2 SimpleProtoTestCommand	230
39.5.3 SimpleProtoExecuteCommand	230
39.5.4 SimpleProtoSendResponse	230
40 Compatibility of TWN4 MultiTech Mini Reader	231
41 Disclaimer	232

1 Definitions and Function Prototypes

All definitions and function prototypes can be found in the header file `twn4.sys.h`, which is located in the folder `/Tools/sys` of the TWN4 Developer Pack.

2 System Functions

2.1 SysCall

This function is useful for writing interfaces, which do a remote call of a system function,

```
bool SysCall(TEnvSysCall *Env);
```

Parameters:

TEnvSysCall *Env	Pointer to a structure which specifies parameters of the functions to be called.
------------------	--

Return:

If the function has been called the return value is `true`, otherwise it is `false`. In this case the specified function does not exist.

2.2 Reset

This functions is performing a reset of the firmware, which also includes a restart of the currently running App.

```
void Reset(void);
```

<u>Parameters:</u>	None.
--------------------	-------

<u>Return:</u>	None.
----------------	-------

2.3 StartBootloader

This function is performing a manual call of the boot loader. As a consequence the execution of the App is stopped.

```
void StartBootloader(void);
```

<u>Parameters:</u>	None.
--------------------	-------

<u>Return:</u>	None.
----------------	-------

2.4 GetSysTicks

Retrieve number of system ticks, specified in multiple of 1 milliseconds, since startup of the firmware.

```
unsigned long GetSysTicks(void);
```

Parameters: None.

Return: Number of system ticks since startup of the firmware. The returned value will restart at 0 after 2^{32} system ticks (around 1193 hours).

2.5 GetVersionString

Retrieve version information. The function generates a ASCII string, terminated by 0.

```
int GetVersionString(char *VersionString, int MaxLen);
```

Parameters:

`char *VersionString` Pointer to an array of characters, which will receive the version information.
`int MaxLen` Maximum number of characters, the specified byte array can receive excluding the 0-termination.

Return: Length of the returned string excluding the 0-termination.

Example:

```
// This sample demonstrates, how to send the version string
// to the host
void WriteChar(char Char)
{
    HostWriteByte(Char);
}
void WriteString(const char *String)
{
    while (*String)
        WriteChar(*String++);
}
void WriteVersion(void)
{
    char Version[30+1];
    GetVersionString(Version, sizeof(Version)-1);
    WriteString(Version);
}
```

2.6 GetUSBType

Retrieve type of USB communication. This could be keyboard emulation or CDC emulation or some other value for future or custom implementations.

```
int GetUSBType(void);
```

Parameters: None.

Return: USBTYPE_NONE: No USB stack,
 USBTYPE_CDC: CDC device (virtual COM port),
 USBTYPE_KEYBOARD: HID keyboard,
 USBTYPE_CCID_HID: CCID + HID (compound device),
 USBTYPE_REPORTS: CCID + HID reports,
 USBTYPE_CCID_CDC: CCID + CDC (compound device),
 USBTYPE_CCID: CCID

2.7 GetDeviceType

Retrieve type of underlying TWN4 hardware.

```
int GetDeviceType(void);
```

Parameters: None.

Return: DEVTYPE_LEGICNFC: TWN4 LEGIC, DEVTYPE_MIFARENFC: TWN4 MIFARE

2.8 Sleep

The device enters the sleep state for a specified time. During sleep state, the device reduces the current consumption to a value, which depends on the mode of sleep.

```
int Sleep(unsigned long Ticks, unsigned long Flags)
```

Parameters:

`unsigned long Ticks` Time, specified in milliseconds, the device should enter the sleep state.

`unsigned long Flags` Events, which cause the function immediately to return. The parameter is a bitwise OR of all events to be handled.

Return: See table return values below

Definition	Value	Description
SLEEPMODE_SLEEP	0x0000	During sleep, device still can be waked up via communication port or LPCD event. In this mode, the device has higher current consumption.
SLEEPMODE_STOP	0x0100	During stop, device still can be waked up via communication port (not USB). In this mode, the device has lowest current consumption.

Notes:

- For details of current consumption, see respective hardware documentation
- TWN4 optionally allows to operate a watchdog timer. The watchdog timer is reset in SLEEPMODE_SLEEP but *not* in SLEEPMODE_STOP. For this reason and if the watchdog timer is activated, the maximum time in SLEEPMODE_STOP is 2 seconds (2000 ticks).

The sleep mode can optionally be cancelled by events. The events are bitwise or-combined and are specified as parameters in the call of the function `Sleep`. Following events are defined:

Definition	Value	Description
WAKEUP_BY_USB_MSK	0x01	Cancel sleep mode, if USB received at least one byte.
WAKEUP_BY_COM1_MSK	0x02	Cancel sleep mode, if COM1 received at least one byte.
WAKEUP_BY_COM2_MSK	0x04	Cancel sleep mode, if COM2 received at least one byte.
WAKEUP_BY_TIMEOUT_MSK	0x10	Cancel sleep mode after specified period of time.
WAKEUP_BY_LPCD_MSK	0x20	Cancel sleep mode, if a transponder card was detected. (Supported by TWN4 MultiTech Nano with HF frontend NXP PN5180 only)

Depending on the source, which woke up the device, `Sleep` returns following values:

Definition	Value	Description
WAKEUP_SOURCE_USB	0	USB input channel received at least one byte.
WAKEUP_SOURCE_COM1	1	The input channel of COM1 received at least one byte.
WAKEUP_SOURCE_COM2	2	The input channel of COM2 received at least one byte.
WAKEUP_SOURCE_TIMEOUT	4	Sleep time ran out.
WAKEUP_SOURCE_LPCD	5	The presence of a transponder card was detected. (Supported by TWN4 MultiTech Nano with HF frontend NXP PN5180 only)

2.9 GetDeviceUID

This function returns a UID, which is unique to the specific TWN4 device.

```
void GetDeviceUID(byte *UID)
```

Parameters:

byte *UID Pointer to an array of bytes, which receives 12 bytes. These 12 bytes represent the UID of the device.

Return: None.

2.10 SetParameters

This function allows to set parameters, which influence the behaviour of the TWN4 firmware. See also chapter System Parameters for a description of the TLV list and all available parameters.

```
bool SetParameters(const byte *TLV, int ByteCount)
```


Parameters:

`const byte *TLV` Pointer to an array of bytes, which contains the TLV list.
`int ByteCount` Length counted in bytes, the TLV list contains.

Return: The function returns `true`, if the parameters was set to the new value. Otherwise the function returns `false`.

Example:

```
// This sample demonstrates a call of function SetParameters.
const byte TLVBytes[] =
{
    ICLASS_READMODE, 1, ICLASS_READMODE_PAC, // Read PAC from iClass.
    INDITAG_READMODE, 1, INDITAG_READMODE_2, // Set Inditag readmode 2
    TLV_END // End of TLV
};

int main(void)
{
    // ...
    SetParameters(TLVBytes, sizeof(TLVBytes));
    // ...
}
```

2.11 GetLastError

This function allows to read the last error code, which was generated by any system function. For a list of available error code see chapter System Errors.

`unsigned int GetLastError(void)`

Parameters: None.

Return: The error code.

3 Interrupt System

TWN4 allows to install interrupt handlers on certain events such as character reception or an active edge detected on GPIOs.

Notes:

- Take care of the execution time of the interrupt handler in order to keep the system viable.
- The execution time of the App is roughly one 1µs per machine command. This means, the execution time of a installed sys tick handler (1000 events per second) is theoretically limited to 1000 machine commands (1.000.000 commands/s / 1.000 ticks/s = 1.000 commands/tick). Practically spoken, this number must be kept much lower to keep the system running.
- Blocking behavior in a interrupt handler (such as waiting for reception of a character) can lead to unpredictable system behavior!
- A single C-statement is - depending on the complexity of the statement - compiled into several machine commands, which reduces the number of of C-statements within a interrupt handler even more. An exact value cannot be specified.

For a actual implementations see following source codes:

- App_INTD101_Interrupt_Demo.c
- App_GIOD100_Interrupt_Demo.c

in the folder Apps\Samples\Interrupt\ of the development pack.

3.1 SetInterruptHandler

Installs or uninstalls an interrupt handler for a specific event. The interrupt handler requires following prototype:

```
void InterruptHandler(void);
```

The function SetInterruptHandler is defined as follows:

```
bool SetInterruptHandler(TInterruptHandler InterruptHandler, int IntNo)
```

Parameters:

InterruptHandler Address of interrupt handler or NULL to uninstall the handler.

int IntNo Number of interrupt in question, see table below.

Return: The function returns true, if parameter IntNo is valid. Otherwise the function returns false.

Following possible values for `int` IntNo:

Definition	Value	Description
INTNO_SYSTICK	0	Systick interrupt, which occurs every 1 millisecond
INTNO_USB_BYTES_TRANSMITTED	1	One or more bytes where sent via USB
INTNO_USB_BYTES_RECEIVED	2	One or more bytes where received via USB
INTNO_COM1_BYTE_TRANSMITTED	3	One byte was sent via COM1
INTNO_COM1_BYTE_RECEIVED	4	One byte was received via COM1
INTNO_COM2_BYTE_TRANSMITTED	5	One byte was sent via COM2
INTNO_COM2_BYTE_RECEIVED	6	One byte was received via COM2
INTNO_GPIO0_TRIGGERED	7	An edge was detected on GPIO0
INTNO_GPIO1_TRIGGERED	8	An edge was detected on GPIO1
INTNO_GPIO2_TRIGGERED	9	An edge was detected on GPIO2
INTNO_GPIO3_TRIGGERED	10	An edge was detected on GPIO3
INTNO_GPIO4_TRIGGERED	11	An edge was detected on GPIO4
INTNO_GPIO5_TRIGGERED	12	An edge was detected on GPIO5
INTNO_GPIO6_TRIGGERED	13	An edge was detected on GPIO6
INTNO_GPIO7_TRIGGERED	14	An edge was detected on GPIO7

3.2 GPIOConfigureInterrupt

```
bool GPIOConfigureInterrupt(int GPIOBits, bool Enable, int Edge)
```

Parameters:

int GPIOBits Specify the GPIOs to be configured. Several GPIOs can be configured simultaneously by using the bitwise or-operator (|). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.

bool Enable Enable (true) or disable (IntNo) interrupts.

int Edge Specify edge on which a interrupt should be generated (see table below).

Return: The function returns true, if parameter IntNo is valid. Otherwise the function returns false.

Definition	Value	Description
TRIGGER_RISING	0	Generate an interrupt on the rising edge.
TRIGGER_FALLING	1	Generate an interrupt on the falling edge.
TRIGGER_RISING_FALLING	2	Generate an interrupt on the rising or falling edge.

4 I/O Functions

4.1 Configuration

4.1.1 Set COM-Port Parameters

This function can be used to configure the asynchronous serial communication ports COM1 and COM2.

```
bool SetCOMParameters
(
    int Channel,
    TCOMParameters* COMParameters
);
```

Parameters:

`int` Channel

Specify the communication port which shall be configured. Use one of the predefined constants CHANNEL_COM1 or CHANNEL_COM2.

TCOMParameters*
COMParameters

Reference to the structure that holds the communication parameters. See the description of TCOMParameters for details.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

Members	Length (Bits)	Description
<code>unsigned long</code> BaudRate	32	This member holds the baud rate.
<code>byte</code> WordLength	8	This member holds the word-length in bits. Use the predefined constant COM_WORDLENGTH_8.
<code>byte</code> Parity	8	This member holds the type of parity to be used. Use one of the predefined constants COM_PARITY_NONE, COM_PARITY_ODD or COM_PARITY_EVEN.
<code>byte</code> StopBits	8	This member holds the number of stop bits. Use one of the predefined constants COM_STOPBITS_0_5, COM_STOPBITS_1, COM_STOPBITS_1_5 or COM_STOPBITS_2.
<code>byte</code> FlowControl	8	This member holds the type of flow control to be used. Use the predefined constant COM_FLOWCONTROL_NONE.

Table 4.1: Definition of TCOMParameters

4.1.2 Get USB Device State

This function returns the functional state of the USB-controller in case the reader is running as USB-device.

```
int GetUSBDeviceState(void);
```

Parameters: None.

Return: Depending on the functional state, the return value is one of the predefined constants USB_DEVICE_STATE_NOTINIT, USB_DEVICE_STATE_DEFAULT, USB_DEVICE_STATE_ADDRESSED, USB_DEVICE_STATE_CONFIGURED or USB_DEVICE_STATE_SUSPENDED.

4.1.3 Get Host Channel

This function returns the channel, which is actually configured for host communication.

```
int GetHostChannel(void);
```

Parameters: None.

Return: The return value is one of the predefined constants CHANNEL_NONE, CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_I2C, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

Note: TWN4 Mini Reader has connected the internal communication lines of USB, COM1 and SPI to one physical port. This means, only one of these communication channels can be active at the same time. The resulting communication channel is CHANNEL_HOST.

4.1.4 Set Host Channel

Specifies the channel to be used for the host communication. By default, the channel is determined by the connected communication cable, which therefore is either USB or COM1.

```
bool SetHostChannel(int Channel);
```

Parameters:

int Channel Specifies the communication channel to be used. This might be CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2 or CHANNEL_SPI or CHANNEL_NONE. If CHANNEL_NONE is specified, channel will be chosen depending on connected communication cable.

Return: If a valid channel was specified, the function returns true. Otherwise, it returns false.

4.2 Miscellaneous Functions

4.2.1 Wake Up Host

This function allows to remotely wake up a host, which is connected via USB. This function is supported by USB keyboard only.

```
void USBRemoteWakeup(void);
```

Parameters: None.

Return: None.

4.3 Data I/O

4.3.1 Query I/O Buffer Size

Use this function to retrieve the input/output buffer size of a specific communication channel.

```
int GetBufferSize  
(  
    int Channel,  
    int Dir  
);
```

Parameters:

`int Channel` Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

`int Dir` Specify the direction. Use one of the predefined constants DIR_OUT or DIR_IN.

Return: The buffer size in bytes.

4.3.2 Get I/O Buffer Byte Count

Use this function to retrieve the number of bytes that are actually stored in the respective I/O buffer. In case of querying the output direction, the functions returns the number of bytes that have not been sent yet, in case of the input direction the number of available bytes that can be read is returned.

```
int GetByteCount  
(  
    int Channel,  
    int Dir  
);
```

Parameters:

`int Channel` Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

`int Dir` Specify the direction. Use one of the predefined constants DIR_OUT or DIR_IN.

Return: The number of bytes that are stored in the buffer.

4.3.3 Test Empty

Check if there are any bytes in the specified I/O buffer.

```
bool TestEmpty
(
    int Channel,
    int Dir
);
```

Parameters:

`int Channel` Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

`int Dir` Specify the direction. Use one of the predefined constants DIR_OUT or DIR_IN.

Return: If the buffer is empty, the return value is true, otherwise it is false.

4.3.4 Test Full

Check if the specified I/O buffer can receive any further data.

```
bool TestFull
(
    int Channel,
    int Dir
);
```


Parameters:

`int` Channel Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

`int` Dir Specify the direction. Use one of the predefined constants DIR_OUT or DIR_IN.

Return: If the buffer is full, the return value is `true`, otherwise it is `false`.

4.3.5 Send Byte

Use this function to send one byte through a specific communication channel. If the respective output buffer is completely occupied, the function blocks until there is enough space.

```
void WriteByte
(
    int Channel,
    byte Byte
);
```

Parameters:

`int` Channel Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.

`byte` Byte The byte to be sent.

Return: None.

4.3.6 Send Multiple Bytes

Use this function to send multiple bytes through a specific communication channel. If there is not enough space in the respective output buffer, the function sends the number of bytes that fit into the buffer and returns this value.

```
int WriteBytes
(
    int Channel,
    const byte* Bytes,
    int ByteCount
);
```

Parameters:

<code>int</code> Channel	Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.
<code>const byte*</code> Bytes	The bytes to be sent.
<u>Return:</u>	Number of bytes sent.

4.3.7 Read Byte

Use this function to read a byte from the input buffer of a specific communication channel. If there is no byte available, the function blocks until there is one.

```
byte ReadByte
(
    int Channel
);
```

Parameters:

<code>int</code> Channel	Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.
<u>Return:</u>	The byte which was read from the input buffer.

4.3.8 Read Multiple Bytes

Use this function to read a desired number of bytes from the input buffer of a specific communication channel. If there is less data available than desired, the function reads the available number of bytes.

```
int ReadBytes
(
    int Channel,
    byte* Bytes,
    int ByteCount
);
```

Parameters:

<code>int</code> Channel	Specify the communication channel. Use one of the predefined constants CHANNEL_USB, CHANNEL_COM1, CHANNEL_COM2, CHANNEL_CCID_DATA, CHANNEL_CCID_CTRL, CHANNEL_I2C, CHANNEL_RNG, CHANNEL_SPI or CHANNEL_HOST. Note that CHANNEL_HOST is only available on TWN4 Mini Reader.
<code>byte*</code> Bytes	The received data is stored in this buffer.
<code>int</code> ByteCount	Specify the number of bytes to be read.
<u>Return:</u>	The byte which was read from the input buffer.

5 Memory Functions

5.1 Byte Operations

5.1.1 Compare Bytes

Compare two byte arrays.

```
bool CompBytes
(
    const byte* Data1,
    const byte* Data2,
    int ByteCount
);
```

Parameters:

`const byte* Data1` Reference to an array of bytes.

`const byte* Data1` Reference to an array of bytes.

`int ByteCount` Number of bytes (beginning from index 0) to be compared.

Return: If the two arrays are identical, the return value is `true`, otherwise it is `false`.

5.1.2 Copy Bytes

Copy bytes from a source to a destination. Source and destination may be identical and the source section may overlap the destination. Depending on that, the correct method for copying will be chosen.

```
void CopyBytes
(
    byte* DestBytes,
    const byte* SourceBytes,
    int ByteCount
);
```

Parameters:

`byte* DestBytes` Reference to an array of bytes which is the destination of the copy operation.

`const byte* SourceBytes` Reference to an array of bytes which is the source of the copy operation.

`int ByteCount` Number of bytes to be copied.

Return: None.

5.1.3 Fill Bytes

Fill bytes within a given array with a value.

```
void FillBytes
(
    byte* Dest,
    byte Value,
    int ByteCount
);
```

Parameters:

<code>byte*</code> Dest	Reference to an array of bytes which is the destination for the operation.
<code>byte</code> Value	The byte value with which the array will be filled.
<code>int</code> ByteCount	Number of bytes to be filled.

Return: None.

5.1.4 Swap Bytes

Swap the order of bytes within an array.

```
void SwapBytes
(
    byte* Data,
    int ByteCount
);
```

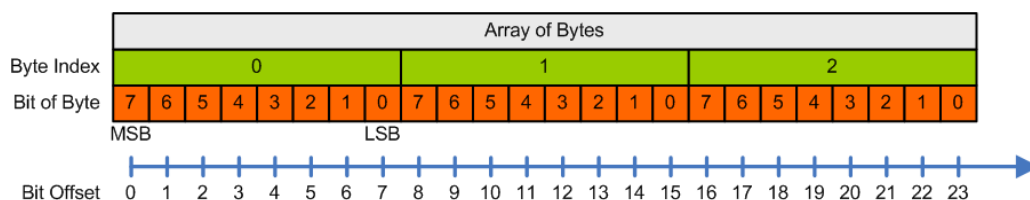
Parameters:

<code>byte*</code> Data	Reference to an array of bytes which is the destination for the operation.
<code>int</code> ByteCount	Number of bytes to be swapped.

Return: None.

5.2 Bit Operations

Bit operations are working on bit fields. A bit field is represented by an array of bytes. The diagram below shows how bit operations are interpreting a given bit offset within an array of bytes:



5.2.1 Read Bit

Read the value of one single bit within a bit field.

```
bool ReadBit
(
    const byte* Byte,
    int BitNr
);
```

Parameters:

<code>const byte*</code> Byte	Reference to an array of bytes which represents the bit field where one bit shall be read.
<code>int</code> BitNr	Position of the bit within the bit field.
<u>Return:</u>	The bit value: <code>true</code> means 1, <code>false</code> means 0.

5.2.2 Write Bit

Set one single bit within a bit field to a given value.

```
void WriteBit
(
    byte* Byte,
    int BitNr,
    bool Value
);
```

Parameters:

<code>byte*</code> Byte	Reference to an array of bytes which represents the bit field where one bit shall be written.
<code>int</code> BitNr	Position within the bit field, where the bit is to be written.
<code>bool</code> Value	The bit value: <code>true</code> means 1, <code>false</code> means 0.
<u>Return:</u>	None.

5.2.3 Copy Bit

Copy one single bit from a source to a destination. Source and destination may be identical.

```
void CopyBit
(
    byte* Dest,
    int DestBitNr,
    const byte* Source,
    int SourceBitNr
);
```

Parameters:

<code>byte*</code> Dest	Reference to an array of bytes which is the destination for the operation.
<code>int</code> DestBitNr	Position within the destination bit field, where the bit is copied to.
<code>const byte*</code> Source	Reference to an array of bytes which is the source for the operation.
<code>int</code> SourceBitNr	Position within the source bit field, where the bit is copied from.
<u>Return:</u>	None.

5.2.4 Compare Bits

Compare two bit sets.

```
bool CompBits
(
    const byte* Data1,
    int Data1StartBit,
    const byte* Data2,
    int Data2StartBit,
    int BitCount
);
```

Parameters:

<code>const byte*</code> Data1	Reference to an array of bytes which represents a bit field.
<code>int</code> Data1StartBit	Start-index (beginning from 0) of the first bit field.
<code>const byte*</code> Data2	Reference to an array of bytes which represents a bit field.
<code>int</code> Data2StartBit	Start-index (beginning from 0) of the second bit field.
<code>int</code> BitCount	Number of bits to be compared.
<u>Return:</u>	If the two bit-sets are identical, the return value is <code>true</code> , otherwise it is <code>false</code> .

5.2.5 Copy Bits

Copy bits from a source to a destination. Source and destination may be identical and the source section may overlap the destination. Depending on that, the correct method for copying will be chosen.

```
void CopyBits
(
    byte* DestBits,
    int StartDestBit,
    const byte* SourceBits,
    int StartSourceBit,
    int BitCount
);
```

Parameters:

<code>byte*</code> DestBits	Reference to an array of bytes which represents a bit field which is the destination of the copy operation.
<code>int</code> StartDestBit	First bit within the destination bit field where the bits are copied to.
<code>const byte*</code> SourceBits	Reference to an array of bytes which represents a bit field which is the source of the copy operation.
<code>int</code> StartSourceBit	First bit within the source bit field where the bits are copied from.
<code>int</code> BitCount	Number of bits to be copied.
<u>Return:</u>	None.

5.2.6 Fill Bits

Fill bits within a given bit field with either 0 or 1.

```
void FillBits
(
    byte* Dest,
    int StartBit,
    bool Value,
    int BitCount
);
```

Parameters:

<code>byte*</code> Dest	Reference to an array of bytes which represents a bit field which is the destination for the operation.
<code>int</code> StartBit	First bit within the bit field where the bits are filled.
<code>bool</code> Value	The bit value: <code>true</code> means 1, <code>false</code> means 0.
<code>int</code> BitCount	Number of bits to be filled.
<u>Return:</u>	None.

5.2.7 Swap Bits

Swap the order of bits within a bit field.

```
void SwapBits
(
    byte* Data,
    int StartBit,
    int BitCount
);
```

Parameters:

<code>byte*</code> Data	Reference to an array of bytes which represents a bit field which is the destination for the operation.
<code>int</code> StartBit	First bit within the bit field where bits are swapped.
<code>int</code> BitCount	Number of bits to be swapped.
<u>Return:</u>	None.

5.2.8 Count Bits

Count the number of ones or zeros within a bit field.

```
int CountBits
(
    const byte* Data,
    int StartBit,
    bool Value,
    int BitCount
);
```

Parameters:

<code>const byte*</code> Data	Reference to an array of bytes which represents a bit field.
<code>int</code> StartBit	First bit within the bit field where counting shall start.
<code>bool</code> Value	The bit value: <code>true</code> means count ones, <code>false</code> means count zeros.
<code>int</code> BitCount	Size of the bit field.
<u>Return:</u>	Number of counted bits.

6 Peripheral Functions

6.1 General Purpose Inputs/Outputs (GPIOs)

6.1.1 Configuration

6.1.1.1 Outputs

Use this function to configure one or several GPIOs as output. Each output can be configured to have an integrated pull-up or pull-down resistor. The output driver characteristic is either Push-Pull or Open Drain.

```
void GPIOConfigureOutputs
(
    int Bits,
    int PullUpDown,
    int OutputType
);
```

Parameters:

<code>int</code> Bits	Specify the GPIOs that shall be configured for output. Several GPIOs can be configured simultaneously by using the bitwise or-operator (). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.
<code>int</code> PullUpDown	Specify the behaviour of the internal weak pull-up/down resistor. Use one of the predefined constants GPIO_PUPD_NOPULL, GPIO_PUPD_PULLUP or GPIO_PUPD_PULLDOWN.
<code>int</code> OutputType	Specify the output driver characteristic. Use one the predefined constants GPIO_OTYPE_PUSHPULL or GPIO_OTYPE_OPENDRAIN.

Return: None.

6.1.1.2 Inputs

Use this function to configure one or several GPIOs as input. Each output can be configured to have an integrated pull-up or pull-down resistor, alternatively it can be left floating.

```
void GPIOConfigureInputs
(
    int Bits,
    int PullUpDown
);
```

Parameters:

`int` Bits Specify the GPIOs that shall be configured for input. Several GPIOs can be configured simultaneously by using the bitwise or-operator (|). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.

`int` PullUpDown Specify the behaviour of the internal weak pull-up/down resistor. Use one of the predefined constants GPIO_PUPD_NOPULL, GPIO_PUPD_PULLUP or GPIO_PUPD_PULLDOWN.

Return: None.

6.1.2 Basic Port Functions

6.1.2.1 Set GPIOs to Logical Level

Use this function to set one or several GPIOs to logical high or low level. The respective ports must have been configured to output in advance.

```
void GPIOSetBits(int Bits);  
void GPIOClearBits(int Bits);
```

Parameters:

`int` Bits Specify the GPIOs that shall be set to a logical level. Several GPIOs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.

Return: None.

6.1.2.2 Toggle GPIOs

Use this function to toggle the logical level of one or several GPIOs. The respective ports must have been configured to output in advance.

```
void GPIOToggleBits  
(  
    int Bits  
);
```

Parameters:

`int` Bits Specify the GPIOs that shall be toggled. Several GPIOs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.

Return: None.

6.1.2.3 Waveform Generation

Use this function to generate a pulse-width modulated square waveform with constant frequency on one or several GPIOs. The respective ports must have been configured to output in advance.

```
void GPIOBlinkBits
(
    int Bits,
    int TimeHi,
    int TimeLo
);
```

Parameters:

int Bits Specify the GPIOs that shall generate the waveform. Several GPIOs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants GPIO0 through GPIO7 for specifying the GPIOs.

int TimeHi Specify the duration for logical high level in milliseconds.

int TimeLo Specify the duration for logical low level in milliseconds.

Return: None.

6.1.2.4 Read GPIOs

Use this function to read the logical level of one GPIO that has been configured as input.

```
int GPIOTestBit
(
    int Bit
);
```

Parameters:

int Bits Specify the GPIO that shall be read. Use one of the predefined constants GPIO0 through GPIO7 for specifying the GPIO.

Return: If the GPIO has logical high level, the return value is 1, otherwise it is 0.

6.1.3 Higher Level Port Functions

6.1.3.1 Send Data in Wiegand Format

Use this function to send a bitstream via a software emulated Wiegand interface. A Wiegand interface uses two data lines, one line is used to transmit ones, the other one is used to transmit zeros. Each GPIO can be individually configured to act as data line. Note that the integrated API LED-functions are working with GPIO0 to GPIO2 by default, so the Wiegand data lines should be selected carefully.

```
void SendWiegand(int GPIOData0,int GPIOData1,int PulseTime,
                int IntervalTime,byte* Bits,int BitCount);
```

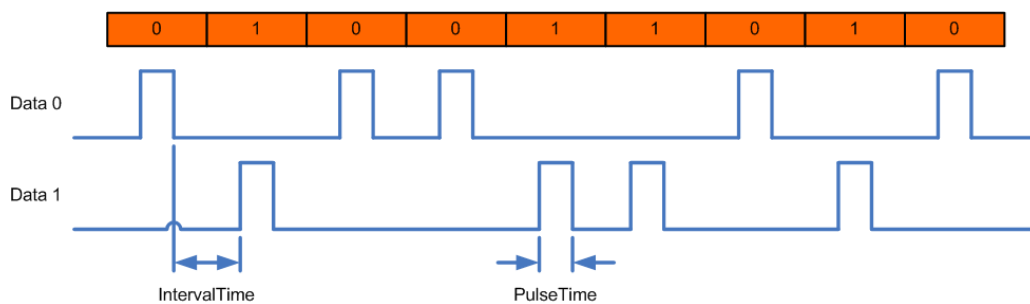
Parameters:

<code>int</code> GPIOData0	Specify the GPIO that shall be used to transmit zeros. Use one of the pre-defined constants GPIO0 through GPIO7 for specifying the GPIO.
<code>int</code> GPIOData1	Specify the GPIO that shall be used to transmit ones. Use one of the pre-defined constants GPIO0 through GPIO7 for specifying the GPIO.
<code>int</code> PulseTime	Specify the pulse duration in microseconds.
<code>int</code> IntervalTime	Specify the duration in microseconds between consecutive pulses.
<code>byte*</code> Bits	Reference to an array of bytes which represents a bit field which holds the data to be sent.
<code>int</code> BitCount	Specify the number of bits to be sent.

Return:

None.

See timing diagram below for details about how the timing values are used:



Example:

Here is an example which shows minimum code for doing a Wiegand output:

```
// Init Section:
// Use GPIO2 and GPIO3 for Wiegand interface
GPIOConfigureOutputs(GPIO2 | GPIO3,GPIO_PUPD_NOPULL,GPIO_OTYPE_PUSH_PULL);
// Enter idle level. In this case we have active low outputs
GPIOSetBits(GPIO2 | GPIO3);
// Prepare some Wiegand data:
byte Bits[4];
Bits[0] = 0x12;
Bits[1] = 0x34;
Bits[2] = 0x56;
Bits[3] = 0x78;
// Now send the bits
SendWiegand(GPIO2,GPIO3,100,1000,Bits,32);
```

Note:

- It is up to the App to complete Wiegand data with parity bits and decide number of bits. In this way the App is fully flexible regarding data to be sent.
- The idle level of the Wiegand interface is determined by state of the outputs before calling SendWiegand. It must be setup by a separate call to GPIOSetBits or GPIOClearBits depending on the requirements of the underlying hardware.
- The GPIOs might need additional circuitry against shortcut or voltage level depending on the intended application.

6.1.3.2 Send Data in Omron Format

Use this function to send a bit stream via a software-emulated Omron interface. An Omron interface uses two lines for data transmission, one for clock and one for the data bit stream. Each GPIO can be individually configured to act as data or clock line. Note that the integrated API LED-functions are working with GPIO0 to GPIO2 by default, so the Omron interface lines should be selected carefully.

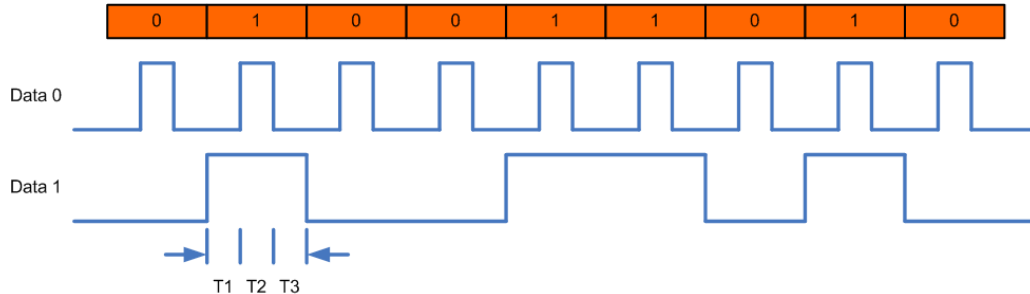
```
void SendOmron(int GPIOClock,int GPIOData,int T1,int T2,int T3,
               byte* Bits,int BitCount);
```

Parameters:

<code>int GPIOClock</code>	Specify the GPIO that shall be used for generating the clock signal. Use one of the predefined constants GPIO0 through GPIO7 for specifying the GPIO.
<code>int GPIOData</code>	Specify the GPIO that shall be used for data transmission. Use one of the predefined constants GPIO0 through GPIO7 for specifying the GPIO.
<code>int T1</code>	
<code>int T2</code>	
<code>int T3</code>	
<code>byte* Bits</code>	Reference to an array of bytes which represents a bit field which holds the data to be sent.
<code>int BitCount</code>	Specify the number of bits to be sent.

Return: None.

See timing diagram below for details about how the timing values are used:



Example:

Here is an example which shows minimum code for doing a clock/data output:

```
// Init Section:
// Use GPIO2 and GPIO3 for the clock/data interface
GPIOConfigureOutputs(GPIO2 | GPIO3,GPIO_PUPD_NOPULL,GPIO_OTYPE_PUSH_PULL);
// Enter idle level. In this case we have active low outputs
GPIOSetBits(GPIO2 | GPIO3);
// Prepare some data:
byte Bits[4];
Bits[0] = 0x12;
Bits[1] = 0x34;
Bits[2] = 0x56;
Bits[3] = 0x78;
// Now send the bits
SendOmron(GPIO2,GPIO3,500,1000,500,Bits,32);
```

Note:

- It is up to the App to complete data with parity bits and decide number of bits. In this way the App is fully flexible regarding data to be sent.
- The idle level of the clock/data interface is determined by state of the outputs before calling `Send0mron`. It must be setup by a separate call to `GPIOSetBits` or `GPIOClearBits` depending on the requirements of the underlying hardware.
- The GPIOs might need additional circuitry against shortcut or voltage level depending on the intended application.

6.2 Beeper

Use following function to sound a beep at the dedicated beeper port.

6.2.1 Duration controlled by function

```
void Beep(int Volume, int Frequency, int OnTime, int OffTime);
```

Parameters:

<code>int</code> Volume	Specify the volume in percent from 0 to 100.
<code>int</code> Frequency	Specify the frequency in Hertz from 500 to 10000.
<code>int</code> OnTime	Specify the duration of the beep in milliseconds from 0 to 10000000.
<code>int</code> OffTime	Specify the length of the pause after the beep in milliseconds from 0 to 10000000. This is useful for generating melodies. If this is not required, the parameter may have the value 0.

Return: None.

6.2.2 Duration controlled by App

Turn on beep with infinite length. The execution of the App is continued independently.

```
void BeepOn(int Volume, int Frequency);
```

Parameters:

<code>int</code> Volume	Specify the volume in percent from 0 to 100.
<code>int</code> Frequency	Specify the frequency in Hertz from 500 to 10000.

Return: None.

Turn off beep:

```
void BeepOff(void);
```

Parameters: None.

Return: None.

6.3 LEDs

6.3.1 General Purpose LED Functions

These functions are related for usage with TWN4 Desktop where the different LEDs have a dedicated connection scheme. The LEDs are connected as follows:

- GPIO0 → Red
- GPIO1 → Green
- GPIO2 → Yellow

6.3.1.1 Initialization

Use this function to initialize the respective GPIOs to drive LEDs.

```
void LEDInit(int LEDs);
```

Parameters:

`int LEDs` Specify the GPIOs that shall be configured for LED operation. Several GPIOs can be configured simultaneously by using the bitwise or-operator (|). Use the predefined constants REDLED, GREENLED or YELLOWLED for specifying the GPIOs.

Return: None.

6.3.1.2 Set LEDs On/Off

Use these functions to set one or several LEDs on/off.

```
void LEDOn(int LEDs);  
void LEDOff(int LEDs);
```

Parameters:

`int LEDs` Specify the LEDs that shall be set on/off. Several LEDs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants REDLED, GREENLED or YELLOWLED for specifying the LEDs.

Return: None.

6.3.1.3 Toggle LEDs

Use this function to toggle one or several LEDs.

```
void LEDToggle(int LEDs);
```

Parameters:

`int` LEDs Specify the LEDs that shall be toggled. Several LEDs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants REDLED, GREENLED or YELLOWLED for specifying the LEDs.

Return: None.

6.3.1.4 Blink LEDs

Use this function to let one or several LEDs blink.

```
void LEDBlink(int LEDs, int TimeOn, int TimeOff);
```

Parameters:

`int` LEDs Specify the LEDs that shall blink. Several LEDs can be handled simultaneously by using the bitwise or-operator (|). Use the predefined constants REDLED, GREENLED or YELLOWLED for specifying the LEDs.

`int` TimeOn Specify the on-time in milliseconds.

`int` TimeOff Specify the off-time in milliseconds.

Return: None.

6.3.2 Diagnostic LED

The TWN4 Core Module has one integrated LED that can be used for diagnostic purposes. There is no initialization necessary.

6.3.2.1 Set Diagnostic LED On/Off

Use these functions to set the diagnostic LED on or off.

```
void DiagLEDOn(void);  
void DiagLEDOff(void);
```

Parameters: None.

Return: None.

6.3.2.2 Toggle Diagnostic LED

Use this function to toggle the diagnostic LED.

```
void DiagLEDToggle(void);
```

Parameters: None.

Return: None.

6.3.2.3 Get LED State

Use this function to determine if the diagnostic LED is on or off.

```
bool DiagLEDIsOn(void);
```

Parameters: None.

Return: If the diagnostic LED is on, the return value is `true`, otherwise it is `false`.

7 Conversion Functions

7.1 Hexadecimal ASCII to Binary

7.1.1 Scan Hexadecimal Character

Convert an ASCII-character which represents a hexadecimal number into its binary representation.

```
int ScanHexChar  
(  
    byte Char  
);
```

Parameters:

byte Char ASCII-coded hexadecimal character. The input value may be one of the characters '0'-'9', 'a'-'f' or 'A'-'F'.

Return: If the character is a valid hexadecimal expression, the return value is the binary representation (a number between 0 and 15), else it is -1.

7.1.2 Scan Hexadecimal String

Convert an array of bytes containing ASCII characters which represents hexadecimal numbers into their binary representation. The conversion is done in place. This means that after successful conversion, number of valid bytes is half of the given count of ASCII characters (two hex digits represent one binary byte).

```
int ScanHexString  
(  
    byte* ASCII,  
    int ByteCount  
);
```

Parameters:

byte* ASCII Reference to an array of ASCII-coded hexadecimal characters. The array may contain the characters '0'-'9', 'a'-'f' or 'A'-'F'. The array is also the destination for the operation.

int ByteCount Number of (ASCII-) bytes to be converted.

Return: Number of successfully converted bytes.

7.2 Binary to Hexadecimal ASCII

Convert a number, which is given as a bit field into ASCII format, and store it in an array of bytes. The conversion is made in the following sequence:

1. Convert the binary data to a number of digits, which is determined by the parameter `MaxDigits`. If `MaxDigits` is smaller than `MinDigits`, the number of digits is equal to `MaxDigits`.
2. If the result of the conversion is less than the number of digits specified by `MinDigits`, precede the converted number with zeros according to `MinDigits`.

```
int ConvertBinaryToString
(
    const byte* SourceBits,
    int StartBit,
    int BitCnt,
    char* String,
    int Radix,
    int MinDigits,
    int MaxDigits
);
```

Parameters:

<code>const byte* SourceBits</code>	A reference to an array of bytes, which contains the bit field.
<code>int StartBit</code>	Index of the first bit to be converted.
<code>int BitCnt</code>	The number of bits, which are valid within the array of bytes.
<code>char* String</code>	A reference to an array of bytes, which receives the result of the conversion.
<code>int Radix</code>	Base for conversion, use: • 2 for binary conversion • 8 for octal conversion • 10 for decimal conversion • 16 for hexadecimal conversion
<code>int MinDigits</code>	Specifies the minimum number of digits, the output should contain. If <code>MinDigits</code> is 0, then at least 1 digit is sent. If <code>MinDigits</code> is greater than the actual width of the number to be converted, then the number is preceded by zeros.
<code>int MaxDigits</code>	Specifies the maximum number of digits, the output may contain. <code>MaxDigits</code> has higher priority than <code>MinDigits</code> .
<u>Return:</u>	The actual number of ASCII bytes, which has been stored in the array <code>String</code> .

8 I2C Functions

This chapter describes functions for accessing the I2C interface of TWN4. I2C is also known as TWI (Two-Wire Interface).

8.1 Initialization/Deinitialization

8.1.1 I2CInit

```
bool I2CInit(int Mode);
```

Parameters:

int Mode This value specifies the mode of operation.

Return: If the operation was successful, the return value is true, otherwise it is false.

8.1.2 I2CDeInit

```
void I2CDeInit(void);
```

Parameters: None.

Return: None.

8.1.3 Examples

```
// Initialize as master
I2CInit(I2CMODE_MASTER);

// Initialize as slave.
// I2CMODE_SLAVE: Setup interface as slave
// 0x30: Address of of this slave
// I2CMODE_CHANNEL: Do communication via channels (this is the
// only currently available option, therefore
// a must to be specified)
I2CInit(I2CMODE_SLAVE | 0x30 | I2CMODE_CHANNEL);
```

8.2 Communication (Master)

8.2.1 I2CMasterStart

Generate a I2C start sequence.

```
void I2CMasterStart(void);
```

Parameters: None.

Return: None.

8.2.2 I2CMasterStop

Generate a I2C stop sequence.

```
void I2CMasterStop(void);
```

Parameters: None.

Return: None.

8.2.3 I2CMasterTransmitByte

Transmit one byte to a slave.

```
void I2CMasterTransmitByte(byte Byte);
```

Parameters:

byte Byte The byte to be transmitted to the slave.

Return: None.

8.2.4 I2CMasterReceiveByte

Receive one byte from a slave.

```
byte I2CMasterReceiveByte(void);
```

Parameters: None.

Return: The byte read from the slave.

8.2.5 I2CMasterBeginWrite

Begin a write sequence. This will send the target slave address together with R/W-bit set to write.

```
void I2CMasterBeginWrite(int Address);
```

Parameters:

int Address The target slave address, a value from 0 to 127.

Return: None.

8.2.6 I2CMasterBeginRead

Begin a read sequence. This will send the target slave address together with R/W-bit set to read.

```
void I2CMasterBeginRead(int Address);
```

Parameters:

int Address The target slave address, a value from 0 to 127.

Return: None.

8.2.7 I2CMasterSetAck

Set ACK state of the master. This ACK will be sent after reception of one byte from the slave.

```
void I2CMasterSetAck(bool SetOn);
```

Parameters:

bool SetOn Set this value to true to turn acknowledge on or false to turn acknowledge off. Definitions ON or OFF may be used for better readability.

Return: None.

8.2.8 Examples

```
// This sample demonstrates transmission and reception of data
// to/from a I2C-slave

// This is the address of the slave
const int I2CAddress = 0x30;
// Init the I2C port
I2CInit(I2CMODE_MASTER);

// Send two bytes to the slave
I2CMasterStart();
I2CMasterBeginWrite(I2CAddress);
I2CMasterTransmitByte(0x12);
I2CMasterTransmitByte(0x34);
I2CMasterStop();

// Receive three bytes from the slave
byte Bytes[3];
I2CMasterStart();
I2CMasterBeginRead(I2CAddress);
// All bytes except last byte require an ACK to be sent
I2CMasterSetAck(ON);
Bytes[0] = I2CMasterReceiveByte();
Bytes[1] = I2CMasterReceiveByte();
// Turn off ACK before reading last byte
I2CMasterSetAck(OFF);
Bytes[2] = I2CMasterReceiveByte();
I2CMasterStop();
```

8.3 Communication (Slave)

Communication as a I2C slaves works with well-defined I2C packets, which must be sent between master and slave (TWN4).

The communication is performed via normal communication channels. Therefore, for transmitting and receiving data, the normal IO-functions must be used. These are WriteByte, ReadByte and so on. In case of communication via I2C, the channel 4 must be used. There is a definition for this channel, which is CHANNEL_I2C.

As a conclusion, TWN4 offers a easy method of changing communication from USB or RS232 to I2C just by changing the communication channel. Only care must be taken to avoid buffer overflow. This can be

achieved by calling appropriate IO-functions `TestEmpty` and `TestFull`. On the other hand many communication protocols avoid a buffer overflow by their inherent flow of communication (e.g. command/response protocol).

The specification for the format of the packets sent/reveived on the I2C bus is as follows:

8.3.1 Slave to Master

1 Byte	Address/Read
1 Byte	Buffer status: Bits 7..4 hold the number of bytes, which are available to be read from the slave. Bits 3..0 hold the maximum number of bytes, which may be sent to slave.
n Bytes	Payload, where n is 0..15. Note: Due to the fact, that ACK must be turned off one byte before the master receives last byte, it is useful to check buffer status and receive bytes in separate read operations.

8.3.2 Master to Slave

1 Byte	Address/Write
n Bytes	Payload, where n is 1..15

8.3.3 Examples

This is a implementation of a I2C master communication, which routes USB- or RS232-interface to the I2C-interface of a TWN4 Core Module. In order to test this example, two TWN4 Core Modules are required:

- 1 TWN4 Core Module, which is running as I2C slave
- 1 TWN4 Core Module, which is running as I2C master.

```
//
// TWN4 App: I2C master, which routes USB or RS232-traffic to I2C
//
#include "twn4.sys.h"
#include "apptools.h"

int main(void)
{
    const int I2CAddress = 0x30;
    // USB or RS232 depends on which cable is connected
    int HostChannel = GetHostChannel();

    I2CInit(I2CMODE_MASTER);
    while (true)
    {
        int I2CRXTXCount;
        int TransferCount;

        I2CMasterStart();
        I2CMasterBeginRead(I2CAddress);
        I2CMasterSetAck(OFF);
```

```

I2CRXTXCount = I2CMasterReceiveByte();
I2CMasterStop();

// *****
// ***** Direction Host -> I2C *****
// *****
TransferCount = MIN(GetByteCount(HostChannel,DIR_IN),
                    I2CRXTXCount & 0x0F);
if (TransferCount > 0)
{
    I2CMasterStart();
    I2CMasterBeginWrite(I2CAddress);
    while (TransferCount-- > 0)
        I2CMasterTransmitByte(ReadByte(HostChannel));
    I2CMasterStop();
}

// *****
// ***** Direction I2C -> Host *****
// *****
TransferCount = MIN(GetBufferSize(HostChannel,DIR_OUT)-
                    GetByteCount(HostChannel,DIR_OUT),
                    I2CRXTXCount >> 4);
if (TransferCount > 0)
{
    I2CMasterStart();
    I2CMasterBeginRead(I2CAddress);
    I2CMasterSetAck(ON);
    // Flush RX/TX byte count
    I2CMasterReceiveByte();
    // Read data except last byte
    while (TransferCount-- > 1)
        WriteByte(HostChannel,I2CMasterReceiveByte());
    // Turn off ACK before reading last byte
    I2CMasterSetAck(OFF);
    WriteByte(HostChannel,I2CMasterReceiveByte());
    I2CMasterStop();
}
}
}

```


9 SPI Functions

This chapter describes functions for accessing the SPI interface of TWN4. The SPI interface can be operated as master or slave.

9.1 Slave Mode

To operate the SPI interface in slave mode, it is only necessary to configure SPI for slave mode using the function `SPIInit`. All data transfer is done using the respective I/O functions via channel `CHANNEL_SPI` (Core/Nano based readers) or `CHANNEL_HOST` (Mini Reader).

9.1.1 Timing

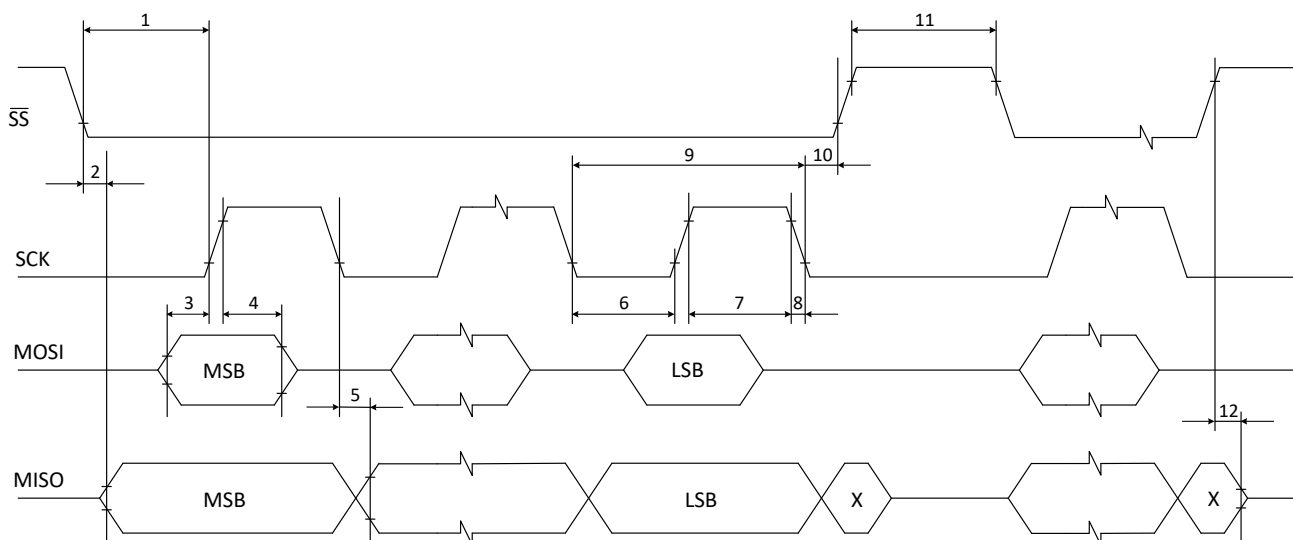


Figure 9.1: SPI Timing in Slave Mode

Marking	Description	Min	Max
1	$\overline{SS}$ low to start of clocking	2 μ s	
2	$\overline{SS}$ low to data output		100ns
3	Data input setup time	5ns	
4	Data input hold time	4ns	
5	SCK to data output		25ns
6	SCK low	34ns	
7	SCK high	34ns	
8	SCK rise/fall time		8ns
9	SCK period	68ns	
10	SCK to $\overline{SS}$ high	68ns	
11	$\overline{SS}$ inactive time	2 μ s	
12	$\overline{SS}$ high to tri-state	2ns	10ns

9.1.2 Communication Protocol

Communication with a TWN4 in SPI slave mode is conducted with well-defined SPI packets, which must be exchanged between master and slave. After a falling edge of the SS line, a communication frame can be initiated. The first exchanged byte is a buffer status byte which gives information about the communication buffer of the master and the slave respectively, see the sections below for details. A rising edge on SS signals the end of the communication frame.

9.1.2.1 Master to Slave

1 Byte Bits 3..0: Length of following payload.

n Bytes Payload, where n is 0..15

Note: It's possible to query the buffer status of a slave by sending 0x00 packets, this won't change the internal state of the slave but enforces the slave to reveal its internal SPI buffer state.

9.1.2.2 Slave to Master

1 Byte Buffer status: Bits 7..4 hold the number of bytes, which are available to be read from the slave. Bits 3..0 hold the maximum number of bytes, which may be sent to slave.

n Bytes Payload, where n is 0..15.

9.1.2.3 Example of a SPI Host Implementation

```
byte SPIReadWrite(byte Byte)
{
    int i;

    for (i=0; i<8; i++)
```

```

{
    if (Byte & 0x80)
        GPIO_MOSI_HIGH();
    else
        GPIO_MOSI_LOW();
    GPIO_SCK_HIGH();
    Byte <<= 1;
    if (GPIO_MISO_TEST())
        Byte |= 1;
    GPIO_SCK_LOW();
}
return Byte;
}

int main(void)
{
    GPIOConfigureOutputs(GPIO_MOSI | GPIO_SCK | GPIO_SS, GPIO_PUPD_NOPULL, GPIO_OTYPE_PUSH_PULL);
    GPIOConfigureInputs(GPIO_MISO, GPIO_PUPD_NOPULL);

    while (true)
    {
        // Query buffer status of SPI slave
        GPIO_SS_LOW();
        byte SPIRXTXCount = SPIReadWrite(0x00);
        GPIO_SS_HIGH();

        // Determine number of bytes to be sent to the slave, limit to 15 bytes
        byte TransferCountTX = MIN(GetByteCount(CHANNEL_USB, DIR_IN), SPIRXTXCount & 0x0F);

        // Send buffer status byte to the slave
        GPIO_SS_LOW();
        byte TransferCountRX = SPIReadWrite(TransferCountTX) >> 4;

        // Send and receive payload
        while (TransferCountTX || TransferCountRX)
        {
            byte Byte = SPIReadWrite(TransferCountTX ? ReadByte(CHANNEL_USB) : 0x00);
            if (TransferCountTX)
                TransferCountTX--;
            if (TransferCountRX)
            {
                WriteByte(CHANNEL_USB, Byte);
                TransferCountRX--;
            }
        }
        GPIO_SS_HIGH();
    }
}

```

9.2 Initialization/Deinitialization

9.2.1 SPINInit

Initialize communication via SPI interface.

```
bool SPINInit(const TSPIParameters *SPIParameters);
```

Parameters:

`const TSPIParameters` Pointer to a structure, which specifies mode of operation.
`*SPIParameters`

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

The members of structure `TSPIParameters` are defined as follows:

Members	Length (Bits)	Description
<code>byte</code> Mode	8	Mode of operation. Specify <code>SPI_MODE_MASTER</code> or <code>SPI_MODE_SLAVE</code> here.
<code>byte</code> CPOL	8	Polarity if clock signal (<code>SPI_SCK</code>). Specify <code>SPI_CPOL_LOW</code> for idle/inactive low or <code>SPI_CPOL_HIGH</code> for idle/inactive high.
<code>byte</code> CPHA	8	Active edge of <code>SPI_SCK</code> . Specify <code>SPI_CPHA_EDGE1</code> for first edge or <code>SPI_CPHA_EDGE2</code> for second edge. In conjunction with the polarity of the clock signal this leads to active edge, which is either rising or falling.
<code>byte</code> ClockRate	8	Specify clock rate of <code>SPI_SCK</code> . Valid values are <code>SPI_CLOCKRATE_117_KHZ</code> , <code>SPI_CLOCKRATE_234_KHZ</code> , <code>SPI_CLOCKRATE_469_KHZ</code> , <code>SPI_CLOCKRATE_938_KHZ</code> , <code>SPI_CLOCKRATE_1_88_MHZ</code> , <code>SPI_CLOCKRATE_3_75_MHZ</code> , <code>SPI_CLOCKRATE_7_5_MHZ</code> or <code>SPI_CLOCKRATE_15_MHZ</code>
<code>byte</code> BitOrder	8	Specify order of data bits on data lines (<code>SPI_MISO</code> and <code>SPI_MOSI</code>). Specify <code>SPI_FIRSTBIT_MSB</code> for idle/inactive low or <code>SPI_FIRSTBIT_LSB</code> for idle/inactive high.

Table 9.1: Definition of `TSPIParameters`

9.2.2 SPIDeInit

Deinitialize SPI interface.

```
void SPIDeInit(void);
```

Parameters: None.

Return: None.

9.3 Communication

9.3.1 SPIMasterBeginTransfer

Begin a transfer via SPI interface in master mode. This function sets signal `SPI_SS-` to active, thus low.

```
void SPIMasterBeginTransfer(void);
```

Parameters: None.

Return: None.

9.3.2 SPIMasterEndTransfer

End a transfer via SPI interface in master mode. This function sets signal SPI_SS- to inactive, thus high.

```
void SPIMasterEndTransfer(void);
```

Parameters: None.

Return: None.

9.3.3 SPITransceive

Send and receive a number of bytes to/from the slave. Background: SPI is a full duplex communication link. This allows to send and receive data at the same time. With every clock pulse, a bit is sent to the slave, another bit is received from the slave.

```
bool SPITransceive(const byte *TXData, byte *RXData, int ByteCount);
```

Parameters:

const byte *TXData Pointer to an array of bytes being transmitted to the slave.

byte *RXData Pointer to an array of bytes being received from the slave.

int ByteCount Number of bytes transferred in each direction.

Return: If the operation was successful, the return value is true, otherwise it is false.

9.3.4 SPITransmit

Send a number of bytes to the slave. Received bits are refused.

```
bool SPITransmit(const byte *TXData, int ByteCount);
```

Parameters:

const byte *TXData Pointer to an array of bytes being transmitted to the slave.

int ByteCount Number of bytes transmitted.

Return: If the operation was successful, the return value is true, otherwise it is false.

9.3.5 SPIReceive

Receive a number of bytes from the slave. Transmitted bits are set to zero.

```
bool SPIReceive(byte *RXData, int ByteCount);
```

Parameters:

byte *RXData Pointer to an array of bytes being received from the slave.
int ByteCount Number of bytes received.

Return: If the operation was successful, the return value is true, otherwise it is false.

9.4 Examples

```
#include "twn4.sys.h"

void FuncSPIInitMaster(void)
{
    const TSPIParameters Mode =
    {
        SPI_MODE_MASTER,
        SPI_CPOL_LOW,
        SPI_CPHA_EDGE1,
        SPI_CLOCKRATE_15_MHZ,
        SPI_FIRSTBIT_MSB
    };
    SPIInit(&Mode);
}

void FuncSPITransmitPacket(void)
{
    SPIMasterBeginTransfer();
    static const byte TXData[4] = { 'A','B','C','D' };
    SPITransmit(TXData,4);
    SPIMasterEndTransfer();
}

void FuncSPIReceivePacket(void)
{
    SPIMasterBeginTransfer();
    byte RXData[4];
    SPIReceive(RXData,4);
    SPIMasterEndTransfer();
}

void FuncSPITransceivePacket(void)
{
    SPIMasterBeginTransfer();
    static const byte TXData[4] = { 'A','B','C','D' };
    byte RXData[4];
    SPITransceive(TXData,RXData,4);
    SPIMasterEndTransfer();
}
```

10 RF Functions

10.1 SearchTag

Use this function to search a transponder in the reading range of TWN4. TWN4 is searching for all types of transponders, which have been specified via function SetTagTypes. If a transponder has been found, tag type, length of ID and ID data itself are returned.

```
bool SearchTag(int *TagType, int *IDBitCount, byte *ID, int MaxIDBytes);
```

<u>Parameters:</u>	None.
<code>int *TagType</code>	Pointer to an integer, which receives the type of tag, which has been found.
<code>int *IDBitCount</code>	Pointer to an integer, which receives the number of bits(!), the ID consists of.
<code>byte *ID</code>	Pointer to an array of bytes, which contain ID data, if a transponder has been found.
<code>int MaxIDBytes</code>	A value, which specifies the buffer size of ID. No more than this specified number of bytes will be copied to the location specified by ID.
<u>Return:</u>	If a transponder has been found, the function returns <code>true</code> , otherwise it returns <code>false</code> .

10.2 SetRFOff

Turn off RF field. If no further operations are required on a transponder found via function SearchTag you may use this command to minimize power consumption of TWN4.

```
void SetRFOff(void);
```

<u>Parameters:</u>	None.
<u>Return:</u>	None.

10.3 SetTagTypes

Use this function to configure the transponders, which are searched by function SearchTag.

```
void SetTagTypes(unsigned int LFTagTypes, unsigned int HFTagTypes);
```

<u>Parameters:</u>	
<code>unsigned int LFTagTypes</code>	Specifies transponder types at the frequency 125.0 kHz - 134.2 kHz.
<code>unsigned int HFTagTypes</code>	Specifies transponder types at the frequency 13.56 MHz.
<u>Return:</u>	None.

10.3.1 Supported Types of LF Tags (125 kHz - 134.2 kHz)

Definition	Frequency	Name	Remark	Active by Default
LFTAG_EM4102	125 kHz	EM4102 / CASI-RUSCO	Supported	Yes
LFTAG_HITAG1S	125 kHz	HITAG 1 / HITAG S	Supported	Yes
LFTAG_HITAG2	125 kHz	HITAG 2	Supported	Yes
LFTAG_EM4150	125 kHz	EM4x50	Supported	Yes
LFTAG_AT5555	125 kHz	AT5555 / AT5557 / AT5577 / Q5	Supported, delivers no ID	Yes
LFTAG_ISOFDX	134.2 kHz	ISO FDX-B / EM4105	Supported	No
LFTAG_EM4026	125 kHz	EM4026	On request	Yes
LFTAG_HITAGU	125 kHz	HITAG μ	On request	Yes
LFTAG_EM4305	125 kHz	EM4305	Supported	No
LFTAG_HIDPROX	125 kHz	HID Prox	Supported with option P	Yes
LFTAG_TIRIS	134.2 kHz	ISO HDX / TIRIS	Supported	No
LFTAG_COTAG	134.2 kHz	Cotag	Supported by option P	No
LFTAG_IOPROX	125 kHz	ioProx	Supported by option P	Yes
LFTAG_INDITAG	125 kHz	Indala	Supported by option P	Yes
LFTAG_HONEYTAG	125 kHz	NexWatch	Supported by option P	Yes
LFTAG_AWID	125 kHz	AWID	Supported	Yes
LFTAG_GPROX	125 kHz	G-Prox	Supported, read of hash value	Yes
LFTAG_PYRAMID	125 kHz	Pyramid	Supported	Yes
LFTAG_KERI	125 kHz	Keri	Supported, read of raw data	Yes
LFTAG_DEISTER	125 kHz	Deister	Supported, read of raw data	No
LFTAG_CARDAX	125 kHz	Cardax	Supported, read of hash value	No
LFTAG_NEDAP	125 kHz	Nedap	Supported, read of hash value	Yes
LFTAG_PAC	134.2 kHz	PAC	Supported, read of hash value	No
LFTAG_IDTECK	125 kHz	IDTECK	Supported, read of raw data	No
LFTAG_ULTRAPROX	125 kHz	UltraProx	Supported, read of raw data	No
LFTAG_ICT	125 kHz	ICT	Supported, read of hash data	No
LFTAG_ISONAS	125 kHz	Isonas	Supported, read of raw data	No

10.3.2 Supported Types of HF Tags (13.56 MHz, Bluetooth)

Definition	Frequency	Name	Remark	Active by Default
HFTAG_MIFARE	13.56 MHz	ISO14443A / MIFARE	Supported	Yes
HFTAG_ISO14443B	13.56 MHz	ISO14443B	Supported	Yes
HFTAG_ISO15693	13.56 MHz	ISO15693 / Tag-it	Supported	Yes
HFTAG_LEGIC	13.56 MHz	LEGIC	Supported by TWN4 LEGIC	Yes
HFTAG_HIDICLASS	13.56 MHz	HID iCLASS	Supported, read of UID, read of PAC with option I	Yes
HFTAG_FELICA	13.56 MHz	FeliCa	Supported, read of UID only	Yes
HFTAG_SRX	13.56 MHz	SRX	Supported	Yes
HFTAG_TOPAZ	13.56 MHz	Topaz	Not supported by TWN4 LEGIC	Yes
HFTAG_CTS	13.56 MHz	CTS	Not supported by TWN4 LEGIC	Yes
HFTAG_BLELC	2.4 GHz	LEGIC Connect (BLE)	Supported by TWN4 Slim LEGIC	No
HFTAG_BLE	2.4 GHz	BLE (Bluetooth Low Energy)	Supported by TWN4 MultiTech 2 BLE	No

To search for more than one type of transponder, several types can be combined.

Note:

The use of the predefined macro TAGMASK is mandatory, even if only one type of tag is specified. Here is an example which is searching for EM4102 and HITAG 1 at LF and for MIFARE at HF:

Example:

```
SetTagTypes(TAGMASK(LFTAG_EM4102) | TAGMASK(LFTAG_HITAG1S),
TAGMASK(HFTAG_MIFARE));
```


10.4 GetTagTypes

This function returns the transponder types currently being searched for by function SearchTag separated by frequency (LF and HF).

```
void GetTagTypes(unsigned int *LFTagTypes, unsigned int *HFTagTypes);
```

Parameters:

`unsigned int *LFTagTypes` Pointer to a value, which receives the LF tag types.

`unsigned int *HFTagTypes` Pointer to a value, which receives the HF tag types.

Return: None.

10.5 GetSupportedTagTypes

This function returns the transponder types, which are actually supported by the individual TWN4 separated by frequency (LF and HF). Also the P-option is taken into account. This means, if the specific TWN4 has no option P, the appropriate transponders are not returned as supported type of transponder.

```
void GetSupportedTagTypes(unsigned int *LFTagTypes,  
                          unsigned int *HFTagTypes);
```

Parameters:

`unsigned int *LFTagTypes` Pointer to a value, which receives the LF tag types.

`unsigned int *HFTagTypes` Pointer to a value, which receives the HF tag types.

Return: None.

11 EM4x02-Specific Transponder Operations

This chapter describes one function for accessing EM4x02 transponders. EM4x02 is a broadly known type of transponder, which is known under several names, such as EM4002, EM4102, Unique and several other brands.

11.1 Function

11.1.1 EM4102_GetTagInfo

Get detailed information regarding transponder type LFTAG_EM4102 being found by function SearchTag.

```
int EM4102_GetTagInfo(void)
```

Parameters: None.

Return: One of the following pre-defined values: EM4102_BITRATE_UNKNOWN or EM4102_BITRATE_F64 or EM4102_BITRATE_F32.

12 HITAG 1- and HITAG S-Specific Transponder Operations

This chapter describes functions for accessing HITAG 1 and HITAG S transponders. HITAG 1 and HITAG S are very similar. Therefore, same set of functions can be used for both types.

HITAG 1 and HITAG S transponder are available with different memory sizes. Due to this, the maximum address, which can be specified depends also on the specific type of transponder:

Type	Memory Size (Bits)	Memory Size (Bytes)	Valid Address Range
HITAG 1	2048	256	0-63
HITAG S 2048	2048	256	0-63
HITAG S 256	256	32	0-7

12.1 Read/Write Data

12.1.1 Hitag1S_ReadPage

Read one page (4 bytes) from the transponder.

```
bool Hitag1S_ReadPage(int PageAddress, byte *Page);
```

Parameters:

int PageAddress	Specifies the address of the page to be read.
byte *Page	Pointer to an array of 4 bytes where page data is stored after a successful operation.

Return: If the operation was successful, the return value is true, otherwise it is false.

12.1.2 Hitag1S_WritePage

Write one page (4 bytes) to the transponder.

```
bool Hitag1S_WritePage(int PageAddress, const byte *Page);
```

Parameters:

int PageAddress	Specifies the address of the page to be written.
byte *Page	Pointer to an array of 4 bytes which are written to the transponder.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

12.1.3 Hitag1S_ReadBlock

Read 1 to 4 consecutive pages (4 to 16 bytes) from the transponder. The number of pages depends on the specified address: The read process is stopped as soon as the read address reaches a block boundary, which is a multiple of 4. If BlockAddress already specifies a block boundary, the maximum of 4 pages will be read.

```
bool Hitag1S_ReadBlock(int BlockAddress,  
                      byte *Block,int *BytesRead);
```

Parameters:

int BlockAddress	Specifies the first page address of the block to be read.
byte *Page	Pointer to an array of 4 to 16 bytes which are read from the transponder.
int *BytesRead	Pointer to an integer, which receives the number of actually read bytes.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

12.1.4 Hitag1S_WriteBlock

Write 1 to 4 consecutive pages (4 to 16 bytes) to the transponder. The number of pages depends on the specified address: The write process is stopped as soon as the write address reaches a block boundary, which is a multiple of 4. If BlockAddress already specifies a block boundary, the maximum of 4 pages will be written.

```
bool Hitag1S_WriteBlock(int BlockAddress,const byte *Block,  
                      int *BytesWritten);
```

Parameters:

int BlockAddress	Specifies the first page address of the block to be written.
byte *Page	Pointer to an array of 4 to 16 bytes which are written to the transponder.
int *BytesWritten	Pointer to an integer, which receives the number of actually written bytes.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

12.2 Hitag1S_Halt

This functions will halt a currently selected transponder. The transponder will not participate in any further transponder communication till the RF field is turned off and on again.

```
bool Hitag1S_Halt(void);
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is false.

13 HITAG 2-Specific Transponder Operations

This chapter describes functions for accessing HITAG 2 transponders.

HITAG 2 is a transponder with a memory size of 256 bits, thus 32 bytes. It stores data organized in pages, where one page is 4 bytes. There are 8 pages, which can be accessed with addresses in the range from 0 to 7.

HITAG 2 can be operated in two modes: Password mode and crypto mode.

Note:

TWN4 supports password mode of HITAG 2 only.

13.1 Read/Write Data

13.1.1 Hitag2_ReadPage

Read one page (4 bytes) from the transponder.

```
bool Hitag2_ReadPage(int PageAddress, byte *Page);
```

Parameters:

byte PageAddress	Specifies the address of the page to be read.
byte *Page	Pointer to an array of 4 bytes where page data is stored after a successful operation.

<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.
----------------	---

13.1.2 Hitag2_WritePage

Write one page (4 bytes) to the transponder.

```
bool Hitag2_WritePage(byte PageAddress, const byte *Page);
```

Parameters:

byte PageAddress	Specifies the address of the page to be written.
byte *Page	Pointer to an array of 4 bytes which are written to the transponder.

<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.
----------------	---

13.1.3 Hitag2_SetPassword

During search for HITAG 2, TWN4 is using a password for doing a login to the transponder. The default password after a reset is 0x4D, 0x49, 0x4B, 0x52. This is the well-known default password for HITAG 2.

```
void Hitag2_SetPassword(const byte *Password);
```

Parameters:

const byte *Password Pointer to an array of 4 bytes, which contains the new password.

Return: None.

13.2 Hitag2_Halt

This functions will halt a currently selected transponder. The transponder will not participate in any further transponder communication till the RF field is turned off and on again.

```
bool Hitag2_Halt(void);
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is false.

14 EM4x50-Specific Transponder Operations

This chapter describes functions for accessing EM4x50 transponders. There are several chips, which are compatible to each other within this family. These are: EM4050, EM4150, EM4450, EM4550. According to the datasheet of the EM4x50 transponder, one word is meant to be 4 bytes.

14.1 Functions

Perform a login operation to the transponder.

14.1.1 EM4150_Login

```
bool EM4150_Login(const byte *Password)
```

Parameters:

const byte *Password Pointer to an array of 4 bytes which contains the password.

Return: If the operation was successful, the return value is true, otherwise it is false.

14.1.2 EM4150_ReadWord

Read one word (4 bytes) from the transponder.

```
bool EM4150_ReadWord(int Address, byte *Word)
```

Parameters:

int Address Specifies the address of the page to be read. The valid address range is from 0 to 33.

byte *Word Pointer to an array of 4 bytes which contains data read from the transponder.

Return: If the operation was successful, the return value is true, otherwise it is false.

14.1.3 EM4150_WriteWord

Write one word (4 bytes) to the transponder.

```
bool EM4150_WriteWord(int Address, const byte *Word)
```


Parameters:

`int Address` Specifies the address of the page to be written.

`const byte *WordData` Pointer to an array of 4 bytes which contains data to be written to the transponder.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

14.1.4 EM4150_WritePassword

Change the password stored on a transponder.

```
bool EM4150_WritePassword(const byte *ActualPassword, const byte *NewPassword)
```

Parameters:

`const byte *ActualPassword` Pointer to an array of 4 bytes which specifies the current password of the transponder.

`const byte *NewPassword` Pointer to an array of 4 bytes which specifies the password to be written to the transponder.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

14.1.5 EM4150_GetTagInfo

Get detailed information regarding transponder type LFTAG_EM4150 being found by function SearchTag.

```
int EM4150_GetTagInfo(void)
```

Parameters: None.

Return: One of the following pre-defined values: `EM4150_BITRATE_UNKNOWN` or `EM4150_BITRATE_F64` or `EM4150_BITRATE_F40`.

15 EM4305-Specific Transponder Operations

This chapter describes functions for accessing the EM4305 transponder. The available system functions are according to the command set of EM4305. For additional information please also consult the data sheet of the manufacturer.

Notes:

- EM4305 is a versatile chip, which allows several types of modulation and bit rates in direction from transponder to the reader. Even though, TWN4 allows to switch EM4305 into any type of modulation, it only supports to read data from the transponder, if the transponder is in default configuration (Manchester modulation, bit rate RF/64)
- If EM4305 is configured to emulate FDX-B, it will be found by the system function `SearchTag` as tag type `LFTAG_ISOFDX`.
- If EM4305 is configured to emulate EM4x02, it will be found by the system function `SearchTag` as tag type `LFTAG_EM4102`.

How to return EM4305 to default configuration:

If EM4305 was programmed to different kind of modulation or bit rate (e.g. due to emulation of FDX), following sequence is proposed to establish communication with transponder:

1. Call `EM4305_Begin`
2. Call `EM4305_Login` with valid password (if needed)
3. Call `EM4305_Write`, write `0xFA 0x01 0x80 0x00` (the default configuration) to address 4
4. Call `EM4305_Begin`
5. Continue with desired further operations

15.1 Functions

15.1.1 EM4305_Begin

If EM4305 in question is already configured to other than Manchester, RF/64, it cannot be found by system function `SearchTag`.

For this reason and in order to work with EM4305 independent of its configuration, there is the system function `EM4305_Begin`. Instead of searching the transponder in question, it just prepares TWN4 for operation with EM4305 (thus, turning on RF field).

`void EM4305_Begin(void)`

Parameters: None.

Return: None.

15.1.2 EM4305_Read

```
bool EM4305_Read(int Address, byte *Data)
```

Read one word (4 bytes) from the transponder.

Parameters:

`int Address` Specifies the address of the word to be read. The valid address range is from 0 to 15.

`byte *Data` Pointer to an array of 4 bytes which contains data read from the transponder.

Return: Return is `false`, if operation failed or `true`, if operation was successful. Success is determined if response from transponder contains valid parity information.

15.1.3 EM4305_Write

Write one word (4 bytes) to the transponder.

```
bool EM4305_Write(int Address, const byte *Data)
```

Parameters:

`int Address` Specifies the address of the word to be written. The valid address range is from 0 to 15.

`const byte *Data` Pointer to an array of 4 bytes which contains data to be written to the transponder.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. Success is determined by if 0x0A was received from the transponder, once operation is completed.

15.1.4 EM4305_Login

Login to the transponder with a given password (4 bytes).

```
bool EM4305_Login(const byte *Password)
```

Parameters:

`const byte *Password` Pointer to an array of 4 bytes which contains the password.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. Success is determined by if 0x0A was received from the transponder, once operation is completed.

15.1.5 EM4305_Protect

Protect words on the transponder by specification of a protection word (4 bytes, 2 bytes are actually used). Also see data sheet of transponder.

```
bool EM4305_Protect(const byte *Data)
```

Parameters:

`const byte *Data` Pointer to an array of 4 bytes which contains the protection data.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`. Success is determined by if 0x0A was received from the transponder, once operation is completed.

15.1.6 EM4305_Disable

Disable modulation of transponder currently in the RF field.

```
bool EM4305_Disable(void)
```

Parameters:

None.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`. The function signals success, as long as transponder does not return anything or return answer is different from 0x01.

16 AT55xx-Specific Transponder Operations

This chapter describes functions for accessing AT55xx transponders. There are several chips, which are compatible to each other within this family. These are: e5550, e5551, T5555, T5555B, T5556, T5557, ATA5567, ATA5577. Note: T5552 and T5558 are not supported by this API.

16.1 Control Functions

16.1.1 AT55_Begin

The function AT55_Begin must be used before subsequent read or write access to the transponder in question.

```
void AT55_Begin(void);
```

Parameters: None.

Return: None.

Background:

Normally, in order to begin any read/write access to a transponder, the TWN4 system provides the function SearchTag. This function searches for a transponder and keeps the RF in appropriate condition to allow subsequent read- and write access.

This sequence is not applicable for the AT55xx family of transponders for two reasons:

- The transponder does not contain a serial number
- The transponder does not send data in a well-known standard format

The way out of this situation is the function AT55_Begin, which does not return any transponder data but turns on RF field for subsequent read-/write operations.

16.2 Read Data

Requirements:

The firmware of TWN4 supports read of data only, if the modulation of the transponder is configured to manchester coding with a bitrate of RF/128 up to RF/8.

Furthermore, TWN4 can be set up to support sequence terminator turned on or off.

The default setup is RF/64 with sequence terminator turned off. In order to choose a different configuration the function SetParameters must be used. Here is an example of how use of RF/32 is programmed:

```
const byte MyRF32Config[] = { AT55_BITRATE, 1, 32, TLV_END };  
SetParameters(MyRF32Config, sizeof(MyRF32Config));
```

Example of how to turn on sequence terminator on and use RF/40:

```
const byte MyRF40Config[] =
{
    AT55_OPTIONS, 1, AT55_OPT_SEQUENCETERMINATOR,
    AT55_BITRATE, 1, 40,
    TLV_END
};
SetParameters(MyRF40Config, sizeof(MyRF40Config));
```

16.2.1 AT55_ReadBlock

Read one block (4 bytes) from the transponder.

```
bool AT55_ReadBlock(int Address, byte *Data);
```

Parameters:

int Address	Specifies the address of the page to be read.
byte *Data	Pointer to an array of 4 bytes which contains data read from the transponder.

<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.
----------------	---

16.2.2 AT55_ReadBlockProtected

Read one block (4 bytes) from a password-protected transponder.

```
bool AT55_ReadBlockProtected(int Address, byte *Data, const byte *Password);
```

Parameters:

int Address	Specifies the address of the page to be read.
byte *Data	Pointer to an array of 4 bytes which contains data read from the transponder.
const byte *Password	Pointer to an array of 4 bytes which contains the password.

<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.
----------------	---

16.3 Write Data

16.3.1 AT55_WriteBlock

Write one block (4 bytes) to the transponder.

```
bool AT55_WriteBlock(int Address, const byte *Data);
```

Parameters:

int Address	Specifies the address of the page to be written.
const byte *Data	Pointer to an array of 4 bytes which contains data to be written to the transponder.

<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.
----------------	---

16.3.2 AT55_WriteBlockProtected

Write one block (4 bytes) to a password-protected transponder.

```
bool AT55_WriteBlockProtected(int Address, const byte *Data, const byte *Password);
```

Parameters:

int Address	Specifies the address of the page to be written.
const byte *Data	Pointer to an array of 4 bytes which contains data to be written to the transponder.
const byte *Password	Pointer to an array of 4 bytes which contains the password.

Return: If the operation was successful, the return value is true, otherwise it is false.

16.3.3 AT55_WriteBlockAndLock

Write one block (4 bytes) to a transponder and lock the written data. Locking data means, that it is not possible to modify data contained in this block.

```
bool AT55_WriteBlockAndLock(int Address, const byte *Data);
```

Parameters:

int Address	Specifies the address of the page to be written.
const byte *Data	Pointer to an array of 4 bytes which contains data to be written to the transponder.

Return: If the operation was successful, the return value is true, otherwise it is false.

16.3.4 AT55_WriteBlockProtectedAndLock

Write one block (4 bytes) to a password-protected transponder and lock the written data. Locking data means, that it is not possible to modify data contained in this block.

```
bool AT55_WriteBlockProtectedAndLock(int Address, const byte *Data, const byte *Password);
```

Parameters:

int Address	Specifies the address of the page to be written.
const byte *Data	Pointer to an array of 4 bytes which contains data to be written to the transponder.
const byte *Password	Pointer to an array of 4 bytes which contains the password.

Return: If the operation was successful, the return value is true, otherwise it is false.

17 TILF (TIRIS) Functions

This chapter describes functions for accessing Texas Instruments Low Frequency transponders (TILF). This type of transponder was formerly also known as TIRIS.

Note:

It is highly recommended to also study datasheets of according transponders. Datasheets are available from Texas Instruments.

17.1 Search Function

17.1.1 TILF_SearchTag

Search for a TILF tag. This function can be used directly instead of the general search function `SearchTag`. The function doing a search for a TILF tag in two different ways: First, a tag is search via a call of function `TILF_ChargeOnlyRead`. Second, a tag is searched via function `TILF_MUGeneralReadPage`, address 3.

```
bool TILF_SearchTag(int *IDBitCount, byte *ID, int MaxIDBytes);
```

Parameters:

<code>int *IDBitCount</code>	A pointer to an integer, which receives the number of actually read bits(!). Due to the nature of the functions <code>TILF_ChargeOnlyRead</code> and <code>TILF_MUGeneralReadPage</code> , the number of received bits is either 32 or 64.
<code>byte *ID</code>	A pointer to an array of bytes, which receives the read ID. Due to the nature of the functions <code>TILF_ChargeOnlyRead</code> and <code>TILF_MUGeneralReadPage</code> , the number of received bytes is either 4 or 8.
<code>int MaxIDBytes</code>	The maximum number of bytes, which will be copied to ID

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

17.2 Single-Page Read/Write Function

17.2.1 TILF_ChargeOnlyRead

Search for a single page transponder. This might be a read-only or a read/write transponder. Only transponders are detected, where ID is stored under use of a CCITT CRC. If a transponder is programmed in a different way, consider using `TILF_ChargeOnlyReadLo`, which allows to read entire content of transponder W/O CRC check.

```
bool TILF_ChargeOnlyRead(byte *ReadData);
```


Parameters:

byte *ReadData A pointer to an array of 8 bytes, which receives checked ID data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.2.2 TILF_ChargeOnlyReadLo

Search for a single page transponder. This might be a read-only or a read/write transponder. No CRC check is performed, thus allowing to read also custom programmed tags. The interpretation of data should be known by the solution builder.

```
bool TILF_ChargeOnlyReadLo(byte *ReadData);
```

Parameters:

byte *ReadData A pointer to an array of 16 bytes, which receives unchecked ID data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.2.3 TILF_SPPProgramPage

Write data to a single-page read/write transponder by using CCITT CRC.

```
bool TILF_SPPProgramPage(const byte *WriteData, byte *ReadData);
```

Parameters:

const byte *WriteData A pointer to an array of 8 bytes, which will be written to the transponder.

byte *ReadData A pointer to an array of 8 bytes, which receives checked response from the transponder.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.2.4 TILF_SPPProgramPageLo

Write data to a single-page read/write transponder.

```
bool TILF_SPPProgramPageLo(const byte *WriteData, byte *ReadData);
```

Parameters:

const byte *WriteData A pointer to an array of 10 bytes, which will be written to the transponder.

byte *ReadData A pointer to an array of 16 bytes, which receives unchecked response from the transponder.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3 Multi-Page Read/Write Function

17.3.1 TILF_MPGeneralReadPage

General read of data from a multi-page transponder (MPT).

```
bool TILF_MPGeneralReadPage(int Address, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
byte *ReadData	A pointer to an array of 8 bytes, which receives data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.2 TILF_MPSelectiveReadPage

Selective read of data from a multi-page transponder (SAMPT or SAMPTS).

```
bool TILF_MPSelectiveReadPage(
    int Address, const byte *SelectiveAddress, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.

byte *ReadData	A pointer to an array of 8 bytes, which receives data.
----------------	--

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.3 TILF_MPProgramPage

Program one page to a multi-page transponder (MPT).

```
bool TILF_MPProgramPage(
    int Address, const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *WriteData	A pointer to an array of 8 bytes, which will be programmed.

byte *ReadData	A pointer to an array of 8 bytes, which receives data.
----------------	--

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.4 TILF_MPSelectiveProgramPage

Selective program of one page to a multi-page transponder (SAMPT or SAMPTS).

```
bool TILF_MPSelectiveProgramPage(
    int Address, const byte *SelectiveAddress,
    const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.
const byte *WriteData	A pointer to an array of 8 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 8 bytes, which receives data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.5 TILF_MPLockPage

Lock one page on a multi-page transponder (MPT).

```
bool TILF_MPLockPage(int Address, byte *ReadData);
```

Parameters:

int Address	The page address, which will be locked.
byte *ReadData	A pointer to an array of 8 bytes, which receives data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.6 TILF_MPSelectiveLockPage

Selective lock one page on a multi-page transponder (SAMPT or SAMPTS).

```
bool TILF_MPSelectiveLockPage(
    int Address, const byte *SelectiveAddress, byte *ReadData);
```

Parameters:

int Address	The page address, which will be locked.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.
byte *ReadData	A pointer to an array of 8 bytes, which receives data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.3.7 TILF_MPGeneralReadPageLo

General read of data from a multi-page transponder (MPT) W/O CRC-check.

```
bool TILF_MPGeneralReadPageLo(int Address, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.3.8 TILF_MPSelectiveReadPageLo

Selective read of data from a multi-page transponder (SAMPT or SAMPTS) W/O CRC-check.

```
bool TILF_MPSelectiveReadPageLo(
    int Address, const byte *SelectiveAddress, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.3.9 TILF_MPProgramPageLo

Program one page to a multi-page transponder (MPT) W/O CRC-check.

```
bool TILF_MPProgramPageLo(
    int Address, const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *WriteData	A pointer to an array of 10 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.3.10 TILF_MPSelectiveProgramPageLo

Selective program of one page to a multi-page transponder (SAMPT or SAMPTS) W/O CRC-check.

```
bool TILF_MPSelectiveProgramPageLo(
    int Address, const byte *SelectiveAddress,
    const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.
const byte *WriteData	A pointer to an array of 10 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.3.11 TILF_MPLockPageLo

Lock one page on a multi-page transponder (MPT) W/O CRC-check.

```
bool TILF_MPLockPageLo(int Address, byte *ReadData);
```

Parameters:

int Address	The page address, which will be locked.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.3.12 TILF_MPSelectiveLockPageLo

Selective lock one page on a multi-page transponder (SAMPT or SAMPTS) W/O CRC-check.

```
bool TILF_MPSelectiveLockPageLo(
    int Address, const byte *SelectiveAddress, byte *ReadData);
```

Parameters:

int Address	The page address, which will be locked.
const byte *SelectiveAddress	Pointer to an array of 3 bytes (24 bits) which provides the selective address.
byte *ReadData	A pointer to an array of 16 bytes, which receives data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4 Multi-Usage Read/Write Function**17.4.1 TILF_MUGeneralReadPage**

General read of one page from a multi-usage transponder (MUSA).

```
bool TILF_MUGeneralReadPage(int Address, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.2 TILF_MUSelectiveReadPage

Selective read of one page from a multi-usage transponder (MUSA).

```
bool TILF_MUSelectiveReadPage(  
    int Address, int SelectiveAddress, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
int SelectiveAddress	A value which specifies the 8-bit selective address.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.3 TILF_MUSpecialReadPage

Special read of one page from a multi-usage transponder (MUSA).

```
bool TILF_MUSpecialReadPage(  
    int Address, const byte *SpecialAddress1,  
    const byte *SpecialAddress2, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be read from.
const byte *SpecialAddress1	Pointer to an array of 5 bytes (40 bits) which provides the special address 1.
const byte *SpecialAddress2	Pointer to an array of 3 bytes (24 bits) which provides the special address 2.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.4 TILF_MUProgramPage

Program one page to a multi-usage transponder (MUSA).

```
bool TILF_MUProgramPage(int Address, const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *WriteData	A pointer to an array of 5 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.5 TILF_MUSelectiveProgramPage

Selective program of one page to a multi-usage transponder (MUSA).

```
bool TILF_MUSelectiveProgramPage(
    int Address, int SelectiveAddress,
    const byte *WriteData, byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
int SelectiveAddress	A value which specifies the 8-bit selective address.
const byte *WriteData	A pointer to an array of 5 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.6 TILF_MUSpecialProgramPage

Special program of one page to a multi-usage transponder (MUSA).

```
bool TILF_MUSpecialProgramPage(
    int Address, const byte *SpecialAddress1,
    const byte *SpecialAddress2, const byte *WriteData,
    byte *ReadData);
```

Parameters:

int Address	The page address, where data will be programmed to.
const byte *SpecialAddress1	Pointer to an array of 5 bytes (40 bits) which provides the special address 1.
const byte *SpecialAddress2	Pointer to an array of 3 bytes (24 bits) which provides the special address 2.
const byte *WriteData	A pointer to an array of 5 bytes, which will be programmed.
byte *ReadData	A pointer to an array of 7 bytes, which receives page data.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

17.4.7 TILF_MULockPage

Lock one page of a multi-usage transponder (MUSA).

```
bool TILF_MULockPage(int Address, byte *ReadData);
```

Parameters:

int Address The page address, which will be locked.

byte *ReadData A pointer to an array of 7 bytes, which receives page data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.4.8 TILF_MUSelectiveLockPage

Selective lock of one page of a multi-usage transponder (MUSA).

```
bool TILF_MUSelectiveLockPage(  
    int Address, int SelectiveAddress, byte *ReadData);
```

Parameters:

int Address The page address, which will be locked.

int SelectiveAddress A value which specifies the 8-bit selective address.

byte *ReadData A pointer to an array of 7 bytes, which receives page data.

Return: If the operation was successful, the return value is true, otherwise it is false.

17.4.9 TILF_MUSpecialLockPage

Special lock of one page of a multi-usage transponder (MUSA).

```
bool TILF_MUSpecialLockPage(  
    int Address, const byte *SpecialAddress1,  
    const byte *SpecialAddress2, byte *ReadData);
```

Parameters:

int Address The page address, which will be locked.

const byte *SpecialAddress1 Pointer to an array of 5 bytes (40 bits) which provides the special address 1.

const byte *SpecialAddress2 Pointer to an array of 3 bytes (24 bits) which provides the special address 2.

byte *ReadData A pointer to an array of 7 bytes, which receives page data.

Return: If the operation was successful, the return value is true, otherwise it is false.

18 ISO14443 Transponder Operations

This chapter handles specific operations for transparent access of ISO14443A/B compliant transponders. Before these functions can be used, the transponder must have been selected using the function `SearchTag(...)`.

18.1 ISO14443A

18.1.1 Get ATQA

This function delivers the ATQA (Answer To Request TypeA) of the last detected ISO14443A compliant transponder.

```
bool ISO14443A_GetATQA(byte* ATQA);
```

Parameters:

`byte* ATQA` After successful completion of this function, the buffer referred by this parameter holds the ATQA of the transponder. The function returns two bytes of data.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

18.1.2 Get SAK

This function delivers the SAK (Select Acknowledge) of the last detected ISO14443A compliant transponder.

```
bool ISO14443A_GetSAK(byte* SAK);
```

Parameters:

`byte* SAK` After successful completion of this function, the buffer referred by this parameter holds the SAK of the transponder. The function returns one byte of data.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

18.1.3 Get ATS

This function delivers the ATS (Answer To Select) of a ISO14443A layer 4 transponder.

```
bool ISO14443A_GetATS
(
    byte* ATS,
    int* ATSByteCnt,
    int MaxATSByteCnt
);
```

Parameters:

<code>byte* ATS</code>	After successful completion of this function, the buffer referred by this parameter holds the ATS which was read from the transponder. Take care for adequate dimensioning.
<code>int* ATSByteCnt</code>	After successful completion of this function, this parameter holds the number of bytes, the ATS contains.
<code>int MaxATSByteCnt</code>	This parameter holds the array-size of ATS in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

18.2 ISO14443B

18.2.1 Get ATQB

This function delivers the ATQB (Answer To Request TypeB) of the last detected ISO14443B compliant transponder.

Note: This function cannot be called on TWN4 MultiTech Legic.

```
bool ISO14443B_GetATQB(byte* ATQB, int* ATQBByteCnt, int MaxATQBByteCnt);
```

Parameters:

<code>byte* ATQB</code>	After successful completion of this function, the buffer referred by this parameter holds the ATQB of the transponder. Take care for adequate dimensioning, the ATQB usually has 12 or 13 bytes in length.
<code>int* ATQBByteCnt</code>	After successful completion of this function, this parameter holds the number of bytes of ATQB.
<code>int MaxATQBByteCnt</code>	This parameter holds the array-size of ATQB in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

18.2.2 Get Answer to ATTRIB

This function delivers the transponder's answer to the ATTRIB command, which is sent automatically during selection process by the reader.

Note: This function cannot be called on TWN4 MultiTech Legic.

```
bool ISO14443B_GetAnswerToATTRIB
(
    byte* AnswerToATTRIB,
    int* AnswerToATTRIBByteCnt,
    int MaxAnswerToATTRIBByteCnt
);
```

Parameters:

<code>byte*</code> AnswerToATTRIB	After successful completion of this function, the buffer referred by this parameter holds the AnswerToATTRIB of the transponder. Take care for adequate dimensioning, AnswerToATTRIB can have one or more bytes in length.
<code>int*</code> AnswerToATTRIBByteCnt	After successful completion of this function, this parameter holds the number of bytes of AnswerToATTRIB.
<code>int</code> MaxAnswerToATTRIBByteCnt	This parameter holds the array-size of AnswerToATTRIB in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

18.3 Check Presence

This function can be used to probe if a ISO14443-4 transponder is still in reading range. The internal state of the transponder remains unchanged.

Note: This function cannot be called on TWN4 MultiTech Legic.

```
bool ISO14443_4_CheckPresence(void);
```

Parameters: None.

Return: If the transponder is still in range, the return value is `true`, otherwise it is `false`.

18.4 ISO14443-3 Transparent Data Exchange

This function can be used for transparent exchange of data between reader and ISO14443-3 transponders. The function does not calculate any CRC or other overhead by itself, so if necessary this has to be conducted on host side.

```
bool ISO14443_3_TDX
(
    byte* TXRX,
    int TXByteCnt,
    int* RXByteCnt,
    int MaxRXByteCnt,
    int Timeout
);
```

Parameters:

<code>byte*</code> TXRX	This buffer holds the byte-string that shall be transmitted to the transponder. The response of the transponder is also returned by this parameter. Take care for adequate dimensioning.
<code>int</code> TXByteCnt	This parameter holds the number of bytes that shall be transmitted to the transponder.
<code>int*</code> RXByteCnt	After successful completion of this function, this parameter holds the number of bytes that the transponder response contains.
<code>int</code> MaxRXByteCnt	This parameter holds the array-size of TXRX in bytes.
<code>int</code> Timeout	Response timeout in milliseconds.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

18.5 ISO14443-4 Transparent Data Exchange

This function can be used for transparent exchange of data between reader and ISO14443-4 transponders. All framing of layer 4 subset is already done by the reader, so only the payload needs to be passed to the function.

```
bool ISO14443_4_TDX
(
    byte* TXRX,
    int TXByteCnt,
    int* RXByteCnt,
    int MaxRXByteCnt
);
```

Parameters:

<code>byte*</code> TXRX	This buffer holds the byte-string that shall be transmitted to the transponder. The response of the transponder is also returned by this parameter. Take care for adequate dimensioning.
<code>int</code> TXByteCnt	This parameter holds the number of bytes that shall be transmitted to the transponder.
<code>int*</code> RXByteCnt	After successful completion of this function, this parameter holds the number of bytes that the transponder response contains.
<code>int</code> MaxRXByteCnt	This parameter holds the array-size of TXRX in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

18.6 Multiple Tag Handling

TWN4 is capable of handling multiple ISO14443A tags that are simultaneously present in the RF field. Use the following functions to inventorize the field and select one of the discovered transponders for subsequent operations.

18.6.1 Search for Transponders

Use this function to search the RF field for ISO14443A transponders. The result is a list of the UID of the respective transponders.

```
bool ISO14443A_SearchMultiTag
(
    int* UIDCnt,
    int* UIDListByteCnt,
    byte* UIDList,
    int MaxUIDListByteCnt
);
```

Parameters:

- | | |
|------------------------------------|--|
| <code>int*</code> UIDCnt | This parameter holds the number of found transponder UIDs. |
| <code>int*</code> UIDListByteCnt | This parameter holds the number of valid bytes of the UID list. |
| <code>byte*</code> UIDList | This parameter holds the list of found UIDs. Every entry is preceded by a single byte representing the respective UID length, e.g. the two transponder IDs 11223344 and 00010203040506 would be coded as follows: 0411223340700010203040506. |
| <code>int</code> MaxUIDListByteCnt | This parameter holds the array-size of UIDList in bytes. |

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

18.6.2 Select Transponder

Use this function to select one of the discovered transponders for further operations.

```
bool ISO14443A_SelectTag(const byte* UID, int UIDByteCnt);
```

Parameters:

- | | |
|------------------------------|---|
| <code>const byte*</code> UID | Specify the UID of the transponder to be selected. |
| <code>int</code> UIDByteCnt | This parameter holds the byte count of the specified UID. |
| <u>Return:</u> | If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> . |

19 MIFARE Classic Specific Transponder Operations

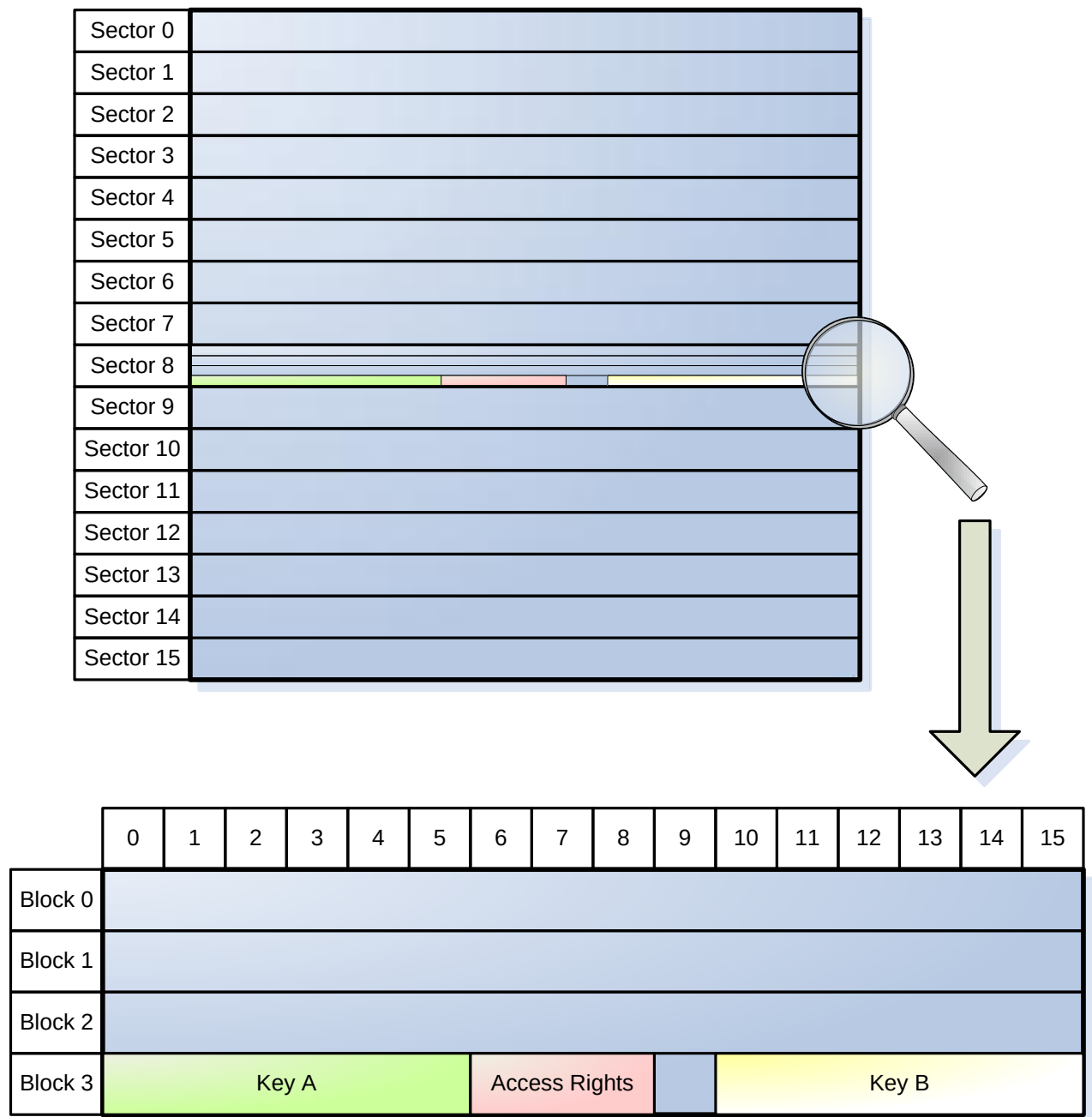


Figure 19.1: Memory layout of a MIFARE Classic 1K transponder

The memory of MIFARE Classic transponders is organized in sectors and blocks. In case of MIFARE Classic 1K, the memory is divided into 16 sectors, each sector holds 4 blocks. Each block holds 16 bytes of data. Each sector is secured by two keys, Key A and Key B which are always located in the last block of a sector (sector trailer). In order to access the respective sector, a login using one of the two keys has to be performed. Once logged in, the data blocks are accessible for read-, write- or value-operations. Each key may be equipped with certain access rights, the access rights are coded in byte 6, 7 and 8 of the sector trailer. Byte 9 is available for data storage.

In case of MIFARE Classic 4K, the memory layout of sector addresses 0 to 31 is compatible to the 1K version, from sector 32 to 39, each sector holds 16 data blocks.

In any case, block 0 of sector 0 is called manufacturer block, and cannot be overwritten. Within this block, the UID is stored and some manufacturer specific data.

19.1 Login

In order to do any operation on a sector of a MIFARE Classic transponder, a login to the respective sector has to be performed. Each sector holds two keys, *Key A* and *Key B*. Depending on the access conditions of the sector, the appropriate key shall be used for the desired operation. Both the keys and the access conditions are stored in the sector trailer.

```
bool MifareClassic_Login
(
    const byte* Key,
    byte KeyType,
    int Sector
);
```

Parameters:

<code>const byte* Key</code>	Pointer to an array of bytes, which has to contain six bytes. These bytes represent the key for the login process.
<code>byte KeyType</code>	Specifies, with which key the operation has to be performed. This is one of the defined constants <code>KEYA</code> or <code>KEYB</code> .
<code>int Sector</code>	Specifies the sector for the login.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

Key (hex)	Description
FF FF FF FF FF FF	Default Transport Key A/B (NXP)
A0 A1 A2 A3 A4 A5	Default Transport Key A (Infineon)
B0 B1 B2 B3 B4 B5	Default Transport Key B (Infineon)
D3 F7 D3 F7 D3 F7	Default key for NDEF-formatted tags

Table 19.1: Well-known keys for MIFARE Classic transponders

19.2 Read/Write Data

19.2.1 Read Data Block

Read 16 bytes of data from a data-block of the transponder. Please note: If a sector trailer is read, the respective key which was used for login is represented by zeros.

```
bool MifareClassic_ReadBlock
(
    int Block,
    byte* Data
);
```

Parameters:

int Block Specify the address of the block to be read. The valid range of this parameter is between 0 and 255.

byte* Data This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 16 bytes of data, so the minimum array size of Data must be at least 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

19.2.2 Write Data Block

Write 16 bytes of data to a data-block of the transponder. Special care must be taken when writing to a sector trailer as a faulty setting of the access conditions can make the sector inaccessible.

```
bool MifareClassic_WriteBlock
(
    int Block,
    const byte* Data
);
```

Parameters:

int Block Specify the address of the block to be written. The valid range of this parameter is between 0 and 255.

const byte* Data This parameter holds the data which shall be written to the tag. Note that this function always writes 16 bytes of data, so the minimum array size of Data shall be at least 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

19.3 Handling of Value Blocks

19.3.1 Read Value Block

Read the value stored in a MIFARE Classic compliant value block.

```
bool MifareClassic_ReadValueBlock
(
    int Block,
    int* Value
);
```

Parameters:

`int` Block Specify the address of the block to be read. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.

`int*` Value This parameter holds the value which was read from the tag if the operation was successful.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: This function checks if the block has a valid value block format. If this is not the case, the function returns `false`.

19.3.2 Write Value Block

Format a data block to a MIFARE Classic compliant value block and assign an initial value.

```
bool MifareClassic_WriteValueBlock
(
    int Block,
    int Value
);
```

Parameters:

`int` Block Specify the address of the block to be formatted. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.

`int` Value This parameter holds the initial value of the value block.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

19.3.3 Increment Value Block

Credit a value block with a given increment value.

```
bool MifareClassic_IncrementValueBlock
(
    int Block,
    int Value
);
```

Parameters:

int Block Specify the address of the block to be incremented. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.

int Value This parameter holds the increment value.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

19.3.4 Decrement Value Block

Debit a value block with a given decrement value.

```
bool MifareClassic_DecrementValueBlock
(
    int Block,
    int Value
);
```

Parameters:

int Block Specify the address of the block to be decremented. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.

int Value This parameter holds the decrement value.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

19.3.5 Copy Value Block

Copy a value block within a sector.

```
bool MifareClassic_CopyValueBlock
(
    int SourceBlock,
    int DestBlock
);
```

Parameters:

int SourceBlock Specify the address of the source block.

int DestBlock Specify the address of the destination block.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20 MIFARE Plus Specific Transponder Operations

MIFARE Plus is mostly compatible to MIFARE Classic, but comes with several enhancements regarding security and functionality. The memory layout is compatible to MIFARE Classic. MIFARE Plus transponders incorporate four different levels of operation, these are called Security Level (SL).

Blank transponders are usually sold in SL0, which is used for personalisation of the transponder. Within this level, the keys and data blocks can be written. When the personalisation process has finished, the transponder has to be switched to a higher security level.

In usual cases, this is SL1 where the transponder is compatible to Mifare Classic, this means the login process, memory layout and read/write operations are the same. In this case refer to the API description of MIFARE Classic.

In case of MIFARE Plus X, the transponder may be switched from SL1 to SL2 where an additional AES authentication becomes necessary before any memory operation is possible. All subsequent Crypto1 operations are then depending on this authentication, as a session key is calculated and the Crypto1 key is diversified for this session. So, after AES authentication, the API functions for MIFARE Classic have to be used for accessing the memory.

MIFARE Plus S or X can be switched to SL3, where a AES authentication is necessary to access the transponder memory. In case of MIFARE Plus X all operations are done fully encrypted, in case of MIFARE Plus S all operations are done in plain but with computation of an additional MAC. For memory operations in SL3, the API functions described in the following chapters shall be used.

Please note, once a MIFARE Plus transponder has been switched to a higher security level, it cannot be switched back again.

20.1 Personalisation

Personalisation can only be done if the transponder is operating in SL0. As all communication is done in plain, this process should be conducted at a secure place. When all personalisation data has been written, the personalisation must be finished by issuing the function Commit Personalisation. After that, the personalisation becomes valid and the transponder is switched to SL1.

20.1.1 Write Personalisation

Use this function to write any personalisation data to a specific block of the transponder.

```
bool MFP_WritePerso(int BlockNr, const byte* Data);
```

Parameters:

<code>int</code> BlockNr	Specify the block number to be written. This can either be the number of a sector block or a AES key.
<code>const byte*</code> Data	Specify the data to be written with this parameter. The function expects always 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.1.2 Commit Personalisation

This function shall be used to switch the transponder to SL1 when all personalisation has been finished. After calling this function, the transponder has to be reselected in order to access it again.

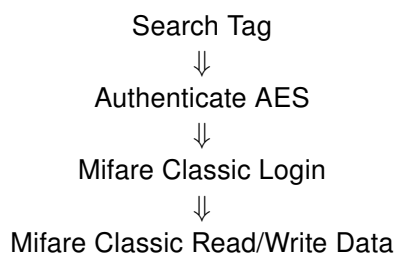
```
bool MFP_CommitPerso(void);
```

Parameters: None

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.2 Authenticate AES

Use this function to do a mutual authentication in AES with the transponder. The key may either be a sector key or a special one like a level switch key. In case of MIFARE Plus running in SL2, a preceding AES authentication is necessary before any following memory operations which are conducted in Crypto1. A typical transaction flow looks like this:



```
bool MFP_Authenticate(int CryptoEnv, int KeyBNr, const byte* Key);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. All consecutive operations with the transponder shall be done using the specified environment.
<code>int</code> KeyBNr	Specify the key number that shall be used for authentication. This can either be a sector key or a special key like a level switch key.
<code>const byte*</code> Key	Specify the key that shall be used for authentication. For AES, the key must have a key length of 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.3 Security Level 3

In Security Level 3 all memory related operations require a preceding AES authentication with the respective key. Please note, MIFARE Plus S does not support all the functionality of a MIFARE Plus X, e.g. handling of value blocks is not supported here.

20.3.1 Read/Write Data

20.3.1.1 Read Data Block

Use this function to read a data block from a MIFARE Plus transponder.

```
bool MFP_ReadBlock(int CryptoEnv, int Block, byte* Data);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int Block</code>	Specify the number of the block that shall be read.
<code>byte* Data</code>	This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 16 bytes of data, so the minimum array size of Data must be at least 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.3.1.2 Write Data Block

Use this function to write data to a block of a MIFARE Plus transponder.

```
bool MFP_WriteBlock(int CryptoEnv, int Block, const byte* Data);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int Block</code>	Specify the number of the block that shall be written.
<code>const byte* Data</code>	This parameter holds the data which shall be written to the tag. Note that this function always writes 16 bytes of data, so the minimum array size of Data must be at least 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.3.2 Handling of Value Blocks

20.3.2.1 Read Value Block

Use this function to read the value stored in a MIFARE compliant value block.

```
bool MFP_ReadValueBlock(int CryptoEnv, int Block, int* Value);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int</code> Block	Specify the address of the block to be read. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.
<code>int*</code> Value	This parameter holds the value which was read from the tag if the operation was successful.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: This function checks if the block has a valid value block format. If this is not the case, the function returns `false`.

20.3.2.2 Write Value Block

Format a data block to a MIFARE compliant value block and assign an initial value.

```
bool MFP_WriteValueBlock(int CryptoEnv, int Block, int Value);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int</code> Block	Specify the address of the block to be formatted. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.
<code>int</code> Value	This parameter holds the initial value of the value block.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

20.3.2.3 Increment Value Block

Credit a value block with a given increment value.

```
bool MFP_IncrementValueBlock(int CryptoEnv, int Block, int Value);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int</code> Block	Specify the address of the block to be incremented. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.
<code>int</code> Value	This parameter holds the increment value.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

20.3.2.4 Decrement Value Block

Debit a value block with a given decrement value.

```
bool MFP_DecrementValueBlock(int CryptoEnv, int Block, int Value);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int</code> Block	Specify the address of the block to be decremented. The valid range of this parameter is between 0 and 255. Note that this function does not work with sector trailers.
<code>int</code> Value	This parameter holds the decrement value.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

20.3.2.5 Copy Value Block

Copy a value block within a sector.

```
bool MFP_CopyValueBlock(int CryptoEnv, int SourceBlock, int DestBlock);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int</code> SourceBlock	Specify the address of the source block.
<code>int</code> DestBlock	Specify the address of the destination block.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

21 MIFARE Ultralight/Ultralight C/Ultralight EV1 Specific Transponder Operations

21.1 Authentication (Ultralight C only)

Depending on the security settings of the transponder, a login with the valid transponder key might be necessary prior performing any further operation.

21.1.1 Authentication with given Key

Use this function to authenticate at a Mifare Ultralight C transponder with a given key.

```
bool MifareUltralightC_Authenticate(const byte* Key);
```

Parameters:

`const byte* Key` Pointer to an array of bytes, which has to contain 16 bytes. These bytes represent the key for the authentication process.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Key (hex)	Description
49 45 4D 4B 41 45 52 42 21 4E 41 43 55 4F 59 46	Default Transport Key

Table 21.1: Well-known key for MIFARE Ultralight C transponders

21.1.2 Authentication using SAM Card

Use this function to authenticate at a Mifare Ultralight C transponder with a key stored on a SAM card. Depending on the security settings of the SAM card, an additional authentication between reader and SAM might be necessary prior issuing this command.

```
bool MifareUltralightC_SAMAuthenticate
(
    int KeyNo,
    int KeyVersion,
    const byte* DIVInput,
    int DIVByteCnt
);
```

Parameters:

<code>int KeyNo</code>	Specify the number of the SAM key entry that shall be used for authentication.
<code>int KeyVersion</code>	Specify the key version of the SAM key entry that shall be used for authentication.
<code>const byte* DIVInput</code>	Specify optional diversification input used for authentication.
<code>int DIVByteCnt</code>	Specify the number of bytes for diversification input. Valid values are 0 to 31.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.2 Write Key from SAM to Transponder Key Storage Area

Use this function to transfer a key from a SAM card to the key storage area of a Mifare Ultralight C transponder. Please note that the key stored on the SAM card must be dumpable. Depending on the security settings of the SAM card, an additional authentication between reader and SAM might be necessary prior issuing this command.

```
bool MifareUltralightC_WriteKeyFromSAM
(
    int KeyNo,
    int KeyVersion,
    const byte* DIVInput,
    int DIVByteCnt
);
```

Parameters:

<code>int</code> KeyNo	Specify the number of the SAM key entry that shall be transfered.
<code>int</code> KeyVersion	Specify the key version of the SAM key entry.
<code>const byte*</code> DIVInput	Specify optional diversification input.
<code>int</code> DIVByteCnt	Specify the number of bytes for diversification input. Valid values are 0 to 31.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

21.3 Read/Write Data

21.3.1 Read Page

Though the page size of this transponder family is 4 bytes, the transponder always returns 16 bytes of data. This is achieved by reading four consecutive data pages, e.g. if page 4 is to be read, the transponder also returns the content of page 5, 6 and 7. The transponder incorporates an integrated roll-back mechanism if reading is done beyond the last physical available page address. E.g., in case of reading page 14 of MIFARE Ultralight this would result in reading page 14, 15, 0, 1.

```
bool MifareUltralight_ReadPage
(
    int Page,
    byte* Data
);
```

Parameters:

<code>int</code> Page	Specify the address of the page to be read. The valid range of this parameter is between 0 and 15 (Ultralight) or 0 and 43 (Ultralight C).
<code>byte*</code> Data	This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 16 bytes of data, so the minimum array size of <code>Data</code> must be at least 16 bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

21.3.2 Write Page

Write 4 bytes of data to a data-page of the transponder. Compared to the read-function, this function processes only one page at once.

```
bool MifareUltralight_WritePage
(
    int Page,
    const byte* Data
);
```

Parameters:

`int` Page Specify the address of the page to be written. The valid range of this parameter is between 2 and 15 (Ultralight) or 2 and 47 (Ultralight C).

`const byte*` Data This parameter holds the data which shall be written to the tag. Note that this function always writes 4 bytes of data, so the minimum array size of Data must be at least 4 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4 Mifare Ultralight EV1

21.4.1 Fast Read

The Fast Read function reads a number of pages beginning at a starting page from the transponder.

```
bool MifareUltralightEV1_FastRead(int StartPage, int NumberOfPages, byte* Data);
```

Parameters:

`int` StartPage Specify the address of the starting page.

`int` NumberOfPages Specify the number of pages to be read.

`byte*` Data This buffer holds the received data from the tag. Take care for proper dimensioning, the buffer size must be at least `NumberOfPages * 4`.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.2 Increment Counter

Use this function to increment of the 3 one-way counters.

```
bool MifareUltralightEV1_IncCounter(int CounterAddr, int IncrValue);
```

Parameters:

`int` CounterAddr Specify the address of the counter to be incremented.

`int` IncrValue Specify the increment value.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.3 Read Counter

Use this function to read the value of one of the 3 one-way counters.

```
bool MifareUltralightEV1_ReadCounter(int CounterAddr, int* CounterValue);
```

Parameters:

`int CounterAddr` Specify the address of the counter to be read.

`int* CounterValue` This parameter holds the returned counter value.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.4 Read ECC Signature

Use this function to read the factory programmed 32 byte ECC signature, to verify NXP Semiconductors as the silicon vendor.

```
bool MifareUltralightEV1_ReadSig(byte* ECCSig);
```

Parameters:

`byte* ECCSig` This buffer holds the returned ECC signature. The required buffer size is 32 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.5 Get Transponder Information

Use this function to retrieve information about the transponder such as product version or storage size.

```
bool MifareUltralightEV1_GetVersion(byte* Version);
```

Parameters:

`byte* Version` This buffer holds 8 bytes of version information.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.6 Password Authentication

Use this function for password authentication. For authentication, a 4 bytes password and a 2 bytes acknowledgment are required.

```
bool MifareUltralightEV1_PwdAuth(const byte* Password, const byte* PwdAck);
```

Parameters:

`const byte* Password` The 4 bytes password is specified by this parameter.

`const byte* PwdAck` This buffer holds 2 bytes of Password Acknowledge.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

21.4.7 Check Tearing Event

Use this function to check if a tearing event has happened at a specific counter.

```
bool MifareUltralightEV1_CheckTearingEvent(int CounterAddr, byte* ValidFlag);
```

Parameters:

<code>int</code> CounterAddr	Specify the address of the counter to be checked.
<code>byte*</code> ValidFlag	The ValidFlag is returned by this parameter. If no tearing event has happened, the returned value is 0xBD.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

22 NTAG Specific Transponder Operations

22.1 Read/Write Data

22.1.1 Read Page

Though the page size of this transponder family is 4 bytes, the transponder always returns 16 bytes of data. This is achieved by reading four consecutive data pages, e.g. if page 4 is to be read, the transponder also returns the content of page 5, 6 and 7. The transponder incorporates an integrated roll-back mechanism if reading is done beyond the last physical available page address. The function is available for all members of the NTAG transponder family.

```
bool NTAG_Read(int Page, byte* Data);
```

Parameters:

int Page Specify the address of the page to be read. The valid range of this parameter depends on the transponder type.

byte* Data This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 16 bytes of data, so the minimum array size of Data must be at least 16 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

22.1.2 Write Page

Write 4 bytes of data to a data-page of the transponder. Compared to the read-function, this function processes only one page at once. The function is available for all members of the NTAG transponder family.

```
bool NTAG_Write(int Page, const byte* Data);
```

Parameters:

int Page Specify the address of the page to be written. The valid range of this parameter depends on the transponder type.

const byte* Data This parameter holds the data which shall be written to the tag. Note that this function always writes 4 bytes of data, so the minimum array size of Data must be at least 4 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

22.1.3 Fast Read

The Fast Read function reads a number of pages beginning at a starting page from the transponder. The function is supported by NTAG21x and NT3H1xxx transponders.

```
bool NTAG_FastRead(int StartPage, int NumberOfPages, byte* Data);
```

Parameters:

<code>int StartPage</code>	Specify the address of the starting page.
<code>int NumberOfPages</code>	Specify the number of pages to be read.
<code>byte* Data</code>	This buffer holds the received data from the tag. Take care for proper dimensioning, the buffer size must be at least <code>NumberOfPages * 4</code> .

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

22.2 Miscellaneous functions

22.2.1 Read Counter

This function reads the value of the one-way counter. The function is supported by NTAG21x transponders. Please note that the `NFC_CNT_EN` bit in `ACCESS` configuration byte must be set in order to make this function work.

```
bool NTAG_ReadCounter(int* CounterValue);
```

Parameters:

<code>int* CounterValue</code>	This parameter holds the returned counter value.
--------------------------------	--

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

22.2.2 Read ECC Signature

Use this function to read the factory programmed 32 byte ECC signature, to verify NXP Semiconductors as the silicon vendor. The function is supported by NTAG21x transponders.

```
bool NTAG_ReadSig(byte* ECCSig);
```

Parameters:

<code>byte* ECCSig</code>	This buffer holds the returned ECC signature. The required buffer size is 32 bytes.
---------------------------	---

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

22.2.3 Get Transponder Information

Use this function to retrieve information about the transponder such as product version or storage size. The function is supported by NTAG21x and NT3H1xxx transponders.

```
bool NTAG_GetVersion(byte* Version);
```

Parameters:

`byte* Version` This buffer holds 8 bytes of version information.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

22.2.4 Password Authentication

Use this function for password authentication. For authentication, a 4 bytes password and a 2 bytes acknowledgement are required. The function is supported by NTAG21x transponders.

```
bool NTAG_PwdAuth(const byte* Password, const byte* PwdAck);
```

Parameters:

`const byte* Password` The 4 bytes password is specified by this parameter.

`const byte* PwdAck` This buffer holds 2 bytes of Password Acknowledge.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

22.2.5 Select Sector

Use this function to perform a sector select in order to switch between different memory sectors of a NT3H1XXX.

```
bool NTAG_SectorSelect(int Sector);
```

Parameters:

`int Sector` Specify the sector to be selected.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23 DESFire Specific Transponder Operations

The memory of a DESFire transponder is organized as a flexible file system. The transponder can hold up to 28 applications and each application may contain up to 32 files of different type and size. Each application can be secured by up to 14 cryptographic keys which are stored in the applications's internal key file. Applications are identified by a number, which must be unambiguous on the transponder. The same rule applies to files within applications, these are identified by numbers which must be unambiguous within the application.

By default, there exists a root-application with the identifier 0x000000 which defines the so-called transponder level. This application cannot hold any files, it is intended to be used for basic administration of the transponder.

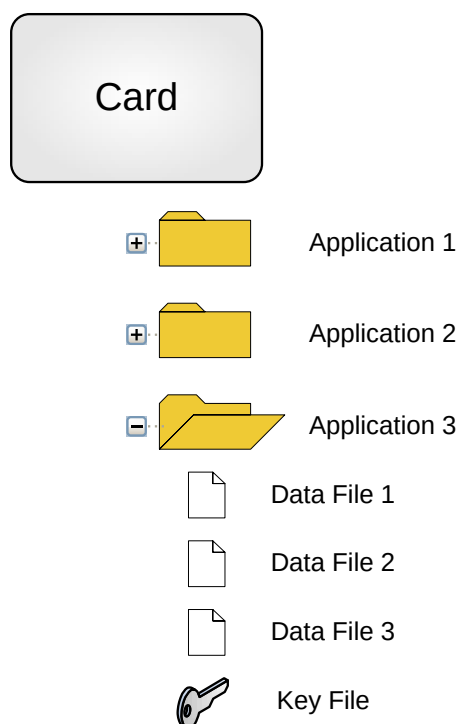


Figure 23.1: DESFire memory layout

A simple use-case could be: Search for a transponder, select the desired application, perform an authentication with the respective key (if required), access data file for read or write operation.

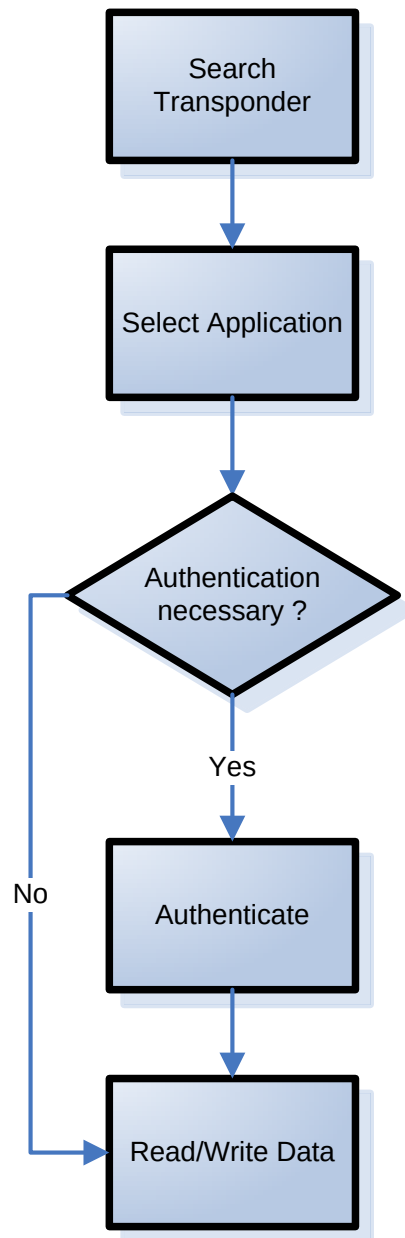


Figure 23.2: Simple way to gain access to the file system

23.1 Security Related Operations

23.1.1 Authenticate

This function shall be used to perform a mutual three pass authentication between reader and transponder. The function supports both 3DES, 3K3DES and AES cryptography. In order to support both the DESFire EV1 transponder family and the older DESFire MF3ICD40, the function incorporates a so-called *Compatible Mode*. In case a DESFire EV2/EV3 transponder is presented to the reader, it's possible to operate the EV2/EV3 Secure Messaging by selecting the *EV2 Mode*.

After successful authentication, a session-key is generated which is used for all further cryptographic operations. The authenticated state is invalidated in case of selecting an application, changing the key which was used for the current authentication or a failed authentication.

On transponder level, depending on the security configuration, an authentication with the transponder master key may be required to perform specific operations:

- Gather information on the transponder
- Change the transponder master key
- Change the transponder master key settings
- Create/delete applications

On application level, depending on the configuration, an authentication may be required to perform specific operations:

- Gather information about the application
- Change the keys of the application
- Create/delete files within the application
- Change access rights
- Access data files

```
bool DESFire_Authenticate
(
    int CryptoEnv,
    int KeyNoTag,
    const byte* Key,
    int KeyByteCount,
    int KeyType,
    int Mode
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. All consecutive operations with the transponder shall be done using the specified environment.
<code>int</code> KeyNoTag	Specify the key number that shall be used for authentication. On transponder level, only key 0 is valid for authentication. On application level, one can specify up to 14 keys which can be used for authentication. Both on transponder and application level, key 0 identifies the respective master key.
<code>const byte*</code> Key	Specify the key that shall be used for authentication. For 3DES/AES, the key must have a key length of 16 bytes, for 3K3DES the key must have a key length of 24 bytes.
<code>int</code> KeyByteCount	Specify the key length of the key. Use one of the predefined constants DESF_KEYLEN_3DES, DESF_KEYLEN_3K3DES or DESF_KEYLEN_AES.
<code>int</code> KeyType	Specify the type of the specified key. Use one of the predefined constants DESF_KEYTYPE_3DES, DESF_KEYTYPE_3K3DES or DESF_KEYTYPE_AES. The authentication will be performed according to the specified key type.
<code>int</code> Mode	Select either DESFire EV2/EV3 Secure Messaging, EV1 ISO-mode authentication or the compatible native DESFire authentication scheme. Use one of the predefined constants DESF_AUTHMODE_COMPATIBLE, DESF_AUTHMODE_EV1 or DESF_AUTHMODE_EV2. Note that 3K3DES or AES cryptography cannot be used in compatible mode, EV2 mode only supports AES cryptography.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: By default, the initial value of any key is all zeros. E.g. after creation of an application, all keys have this initial value.

Example:

```
// Perform AES-authentication using key 0

const byte Key[16] =
{
    0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
    0x88, 0x99, 0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF
};

if (DESFire_Authenticate(
    CRYPTO_ENV0,
    0,
    Key,
    DESF_KEYLEN_AES,
    DESF_KEYTYPE_AES,
    DESF_AUTHMODE_EV1))
{
    DoSomething();
}
```

23.1.2 Get Key Version

This function can be used to read the current key version of any key that is stored on the transponder. If the selected application is 0x000000, the command applies to the transponder master key and therefore only key number 0 is valid for querying the key version.

```
bool DESFire_GetKeyVersion
(
    int CryptoEnv,
    int KeyNo,
    byte* KeyVer
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int KeyNoTag</code>	Specify the key number that shall be queried.
<code>byte* KeyVer</code>	The key version information is returned as one byte by this parameter.

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

Example:

```
// Query key version of key 0
byte KeyVer;

if (DESFire_GetKeyVersion(CRYPTO_ENV0,0,&KeyVer))
{
    DoSomething();
}
```

23.1.3 Get Key Settings

This function allows to get information on the transponder- or application key settings. Depending on the key settings, a preceding authentication with the respective master key may be required.

```
bool DESFire_GetKeySettings
(
    int CryptoEnv,
    TDESFireMasterKeySettings* MasterKeySettings
);
```

Parameters:`int CryptoEnv`

Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

TDESFireMasterKey
Settings*

This structure receives the queried master key settings.

MasterKeySettings

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

Members	Length (Bits)	Description
TDESFireKeySettings KeySettings	8	This member holds the settings of the master key.
<code>int</code> NumberOfKeys	32	This member holds the number of available keys. The valid range is 0 to 14.
<code>int</code> KeyType	32	This member holds the type of keys. Possible value is one of the predefined constants DESF_KEYTYPE_3DES, DESF_KEYTYPE_3K3DES or DESF_KEYTYPE_AES.

Table 23.1: Definition of TDESFireMasterKeySettings

Members	Length (Bits)	Description
<code>byte AllowChangeMasterKey</code>	1	If set to 1 the master key is changeable, otherwise it cannot be changed any more.
<code>byte FreeDirectoryList</code>	1	If set to 1 no preceding authentication with the master key is required to perform the operations <code>GetFileIDs</code> , <code>GetFileSettings</code> , <code>GetKeySettings</code> (application level) or <code>GetApplicationIDs</code> , <code>GetKeySettings</code> (transponder level). If set to 0, an authentication with the master key is required.
<code>byte FreeCreateDelete</code>	1	If set to 1 no preceding authentication with the master key is required to perform the operations <code>CreateFile/DeleteFile</code> (application level) or <code>CreateApplication/DeleteApplication</code> (transponder level). If set to 0, an authentication with the master key is required.
<code>byte ConfigurationChangeable</code>	1	If set to 1 the configuration is changeable if authenticated with the master key. If set to 0, the configuration cannot be changed any more.
<code>byte ChangeKeyAccessRights</code>	4	This member holds the access rights for changing keys. On transponder level this member is set to 0. 0x0: Authentication with the master key is necessary to change any key. 0x1...0xD: Authentication with the specified key is necessary to change any key. The specified key and the master key can only be changed after authentication with the master key. 0xE: Authentication with the key to be changed is necessary to change the key. 0xF: All keys except the master key are frozen.

Table 23.2: Definition of `TDESFireKeySettings`Example:

```
// Query key settings of application 0x123456

TDESFireMasterKeySettings MasterKeySettings;

if (DESFire_SelectApplication(0x123456))
{
    if (DESFire_GetKeySettings(CRYPTO_ENV0,&MasterKeySettings))
    {
        DoSomething(MasterKeySettings);
    }
}
```

23.1.4 Change Key Settings

This function allows to change the transponder- or application master key settings. The respective master key settings can only be changed, if the bit `ConfigurationChangeable` of the current key settings was not

cleared before. In order to change the key settings, a preceding authentication with the respective master key is required in general.

```
bool DESFire_ChangeKeySettings
(
    int CryptoEnv,
    const TDESFireMasterKeySettings* MasterKeySettings
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>const TDESFireMasterKeySettings* MasterKeySettings</code>	This structure holds the new master key settings. See chapter <i>Get Key Settings</i> for details.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.1.5 Change Key

This function allows to change a key. The respective key settings define (see chapter *Get Key Settings*) whether changing of a key is permitted or not and which key must be used for authentication before calling this function.

```
bool DESFire_ChangeKey
(
    int CryptoEnv,
    int KeyNo,
    const byte* OldKey,
    int OldKeyByteCount,
    const byte* NewKey,
    int NewKeyByteCount,
    byte KeyVersion,
    const TDESFireMasterKeySettings* MasterKeySettings
);
```


Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> KeyNo	Specify the key number that shall be changed.
<code>const byte*</code> OldKey	Specify the old key.
<code>int</code> OldKeyByteCount	Specify the length of the old key in bytes.
<code>const byte*</code> NewKey	Specify the new key.
<code>int</code> NewKeyByteCount	Specify the length of the new key in bytes.
<code>byte</code> KeyVersion	Specify the key version of the new key.
<code>const</code> TDESFireMasterKey Settings* MasterKeySettings	This structure holds the current master key settings. See chapter <i>Get Key Settings</i> for details.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

Example:

```
// Change key 1 of application 0x123456

const byte oldKey[16] =
{
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
const byte newKey[16] =
{
    0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
    0x88, 0x99, 0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF
};
TDESFireMasterKeySettings MasterKeySettings;

if (!DESFire_SelectApplication(0x123456))
{
    return; // Error selecting application
}
if (!DESFire_GetKeySettings(CRYPTO_ENV0, &MasterKeySettings))
{
    return; // Error gathering key settings
}
if (MasterKeySettings.KeySettings.ChangeKeyAccessRights == 0)
{
    // Authenticate with master key
    if (!DESFire_Authenticate(
        CRYPTO_ENV0,
        0,
        oldKey,
        DESF_KEYLEN_AES,
        DESF_AUTHMODE_EV1))
    {
        return; // Authentication error
    }
    if (!DESFire_ChangeKey(
```

```

        CRYPTO_ENV0,
        1,
        oldKey,
        DESF_KEYLEN_AES,
        newKey,
        DESF_KEYLEN_AES,
        0x20,
        &MasterKeySettings))
    {
        return; // Error changing key 1
    }
}

```

23.2 Transponder Related Operations

23.2.1 Create Application

This function allows to create a new application on the transponder. Depending on the security settings of the transponder, a preceding authentication with the transponder master key may be required, see chapter *Get Key Settings* for details.

```

bool DESFire_CreateApplication
(
    int CryptoEnv,
    int AID,
    const TDESFireMasterKeySettings* MasterKeySettings
);

```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int AID</code>	Specify the Application ID of the new application to be created. The AID consists of 24 bit, its value must be unique on the transponder. The value 0x000000 is reserved for the root application.
<code>const TDESFireMasterKeySettings* MasterKeySettings</code>	This structure holds the master key settings of the new application. See chapter <i>Get Key Settings</i> for details.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Example:

```

// Create application 0x123456

TDESFireMasterKeySettings MasterKeySettings;

MasterKeySettings.KeySettings.AllowChangeMasterKey    = true;
MasterKeySettings.KeySettings.FreeDirectoryList       = true;
MasterKeySettings.KeySettings.FreeCreateDelete       = true;
MasterKeySettings.KeySettings.ConfigurationChangeable = true;

```

```

MasterKeySettings.KeySettings.ChangeKeyAccessRights    = 0x0;
MasterKeySettings.NumberOfKeys = 2;
MasterKeySettings.KeyType      = DESF_KEYTYPE_AES;

if (DESFire_CreateApplication(
    CRYPTO_ENV0,
    0x123456,
    &MasterKeySettings))
{
    DoSomething();
}

```

23.2.2 Delete Application

This function allows to delete an existing application on the transponder. Depending on the security settings of the transponder, a preceding authentication with the transponder master key may be required, see chapter *Get Key Settings* for details.

```

bool DESFire_DeleteApplication
(
    int CryptoEnv,
    int AID
);

```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int AID</code>	Specify the Application ID of the application that shall be deleted. The AID consists of 24 bit. The value 0x000000 is reserved for the root application hence this AID cannot be deleted.

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

23.2.3 Get Application IDs

This function allows to list all application IDs that exist on the transponder. Depending on the security settings of the transponder, a preceding authentication with the transponder master key may be required, see chapter *Get Key Settings* for details.

```

bool DESFire_GetApplicationIDs
(
    int CryptoEnv,
    int* AIDs,
    int* NumberOfAIDs,
    int MaxAIDCnt
);

```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int*</code> AIDs	After successful completion of this function, this parameter holds a list of the retrieved application IDs.
<code>int*</code> NumberOfAIDs	This parameter holds the number of retrieved application IDs.
<code>int</code> MaxAIDCnt	Specify the maximum number of application IDs, that can be stored in the array AIDs. Note: Up to 28 applications can be stored on a DESFire transponder, so take care for proper dimensioning of the array AIDs.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Example:

```
// List applications stored on the transponder

int AIDList[28];
int NumberOfAIDs;

if (DESFire_GetApplicationIDs(
    CRYPTO_ENV0,
    AIDList,
    &NumberOfAIDs,
    sizeof(AIDList)/sizeof(int)))
{
    DoSomething(AIDList,NumberOfAIDs);
}
```

23.2.4 Select Application

This function is used to select an application in order to perform further operations such as reading or writing. Depending on the security settings of the selected application, an authentication with one of the application's keys may be required after selection.

```
bool DESFire_SelectApplication
(
    int CryptoEnv,
    int AID
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> AID	This parameter holds the application ID of the application to be selected.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.5 Format Transponder

Calling this function results in formatting the transponder. This means, all applications including their files and keys are destroyed and the occupied memory space is released for future use. For proper usage, a preceding authentication with the transponder master key is required.

```
bool DESFire_FormatTag
(
    int CryptoEnv
);
```

23.2.6 Get Transponder Information

This function can be used to gather detailed information about the DESFire transponder regarding hardware and software version.

```
bool DESFire_GetVersion
(
    int CryptoEnv,
    TDESFireVersion* Version
);
```

Parameters:

`int CryptoEnv`

Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

`TDESFireVersion* Version`

This structure receives the queried manufacturing related information.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

Members	Length (Bits)	Description
TDESFireTagInfo HWInfo	80	This member holds the hardware related version information.
TDESFireTagInfo SWInfo	80	This member holds the software related version information.
TDESFireProdInfo ProdInfo	112	This member holds manufacturing specific information.

Table 23.3: Definition of TDESFireVersion

Members	Length (Bits)	Description
<code>byte VendorID</code>	8	Codes the vendor ID (0x04 stands for NXP).
<code>byte Type</code>	8	Codes the type (here 0x01).
<code>byte SubType</code>	8	Codes the subtype (here 0x01).
<code>byte VersionMajor</code>	8	Codes the major version number.
<code>byte VersionMinor</code>	8	Codes the minor version number.
<code>uint32_t StorageSize</code>	32	Size of EEPROM in bytes.
<code>byte CommunicationProtocol</code>	8	Codes the communication protocol type (here 0x05 means ISO14443-3 and -4).

Table 23.4: Definition of TDESFireTagInfo

Members	Length (Bits)	Description
<code>byte UID[7]</code>	56	This member holds the unique serial number. If the transponder is configured to Random ID, the UID is set to 0x00.
<code>byte ProdBatchNumber[5]</code>	40	Codes the production batch number.
<code>byte CalendarWeekOfProduction</code>	8	Codes the calendar week of production.
<code>byte YearOfProduction</code>	8	Codes the year of production.

Table 23.5: Definition of TDESFireProdInfo

23.2.7 Get Available Memory Space

This function allows to gather the available memory space of the transponder. A preceding authentication is not required.

```
bool DESFire_FreeMem
(
    int CryptoEnv,
    int* FreeMemory
);
```

Parameters:

`int CryptoEnv`

Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

`int* FreeMemory`

After successful completion of this function, the available memory size in bytes is returned by this parameter.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.8 Get Card UID

This function allows to retrieve the card UID in case of random ID. A preceding authentication with any key is required prior calling this function.

```
bool DESFire_GetUID
(
    int CryptoEnv,
    byte* UID,
    int* Length,
    int BufferSize
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>byte*</code> UID	After successful completion of this function, the real card UID is returned by this parameter. Note: The UID usually occupies 7 bytes, so take care for proper dimensioning of the array UID.
<code>int*</code> Length	The length in bytes of the UID is returned by this parameter.
<code>int</code> BufferSize	This parameter specifies the size of the array UID in bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.9 Set Transponder Configuration

23.2.9.1 Disable Format Tag

When this function is called, formatting the transponder is not possible any more (see chapter *Format Transponder*). A preceding authentication with the transponder master key is required prior calling this function. Note: Disabling tag formatting cannot be reset any more.

```
bool DESFire_DisableFormatTag
(
    int CryptoEnv
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
----------------------------	--

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.9.2 Enable Random ID

When this function is called, the transponder is turned into Random ID mode, this means the real UID can only be retrieved by authenticating to the transponder and calling the function *Get Card UID*. A preceding authentication with the transponder master key is required prior calling this function. Note: Setting the transponder to Random ID mode cannot be reset any more.

```
bool DESFire_EnableRandomID
(
    int CryptoEnv
);
```

Parameters:

int CryptoEnv Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.9.3 Set Default Key

This function can be used to specify the default key, which is applied when e.g. a new application is created on the transponder. By default, keys are initialized to 0x00. A preceding authentication with the transponder master key is required prior calling this function.

```
bool DESFire_SetDefaultKey
(
    int CryptoEnv,
    const byte* Key,
    int KeyByteCount,
    byte KeyVersion
);
```

Parameters:

int CryptoEnv Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

const byte* Key This parameter specifies the new default key.

int KeyByteCount This parameter specifies the length of the new default key in bytes. Use one of the predefined constants DESF_KEYLEN_3DES, DESF_KEYLEN_3K3DES or DESF_KEYLEN_AES.

byte KeyVersion This parameter specifies the default key version.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.2.9.4 Set User-defined Answer To Select (ATS)

This function can be used to specify a user-defined Answer To Select (ATS) which is returned by the transponder after RATS. Changing the ATS to a non-default value shall only be carried out by experts as a ATS longer than 16 bytes could cause problems with readers that support only frame sizes of max. 16 bytes. The ATS must be formatted as follows: TL T0 TA TB TC + Historical bytes. The default ATS of DESFire EV1 is TL=0x06, T0=0x75, TA=0x77, TB=0x81, TC=0x02, Historical Bytes=0x80.

```
bool DESFire_SetATS
(
    int CryptoEnv,
    const byte* ATS,
    int Length
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>const byte* ATS</code>	This parameter specifies the new ATS.
<code>int Length</code>	This parameter specifies the length of the new ATS in bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.3 Application Related Operations

This section deals with file handling within an application of a DESFire transponder. An application can hold three different basic file types: Data files, Value files and Record Files. Data files are available either with or without integrated backup-mechanism, Value files and Record files always incorporate integrated backup. There exist two types of record files: Linear record files and Cyclic Record Files. Some functions for file handling are using the data structure `TDESFireFileSettings` which defines all relevant file settings. See the following tables for reference:

Coding of access rights:

Every file holds four different access rights, each access right is coded in one nibble. These four nibbles are concatenated and form the 16 bit variable `AccessRights`.

One nibble codes 16 possible values. If it codes a number between 0 and 13, this references a certain key number within the application.

If the number is 14, this means "free" access so there is no authentication necessary to perform the respective operation on the file. In case of coding the number 15, this means "deny" access.

23.3.1 Create File

This section deals with the creation of new files within applications. Depending on the specified file type, the file is either created with or without integrated backup-mechanism. Each file requires an unambiguous identifier which is coded in one byte in the range from 0x00 to 0x1F. During creation of the file, the level of security is defined in the communication settings. Communication can be either plain, secured by MAC or

Members	Length (Bits)	Description
<code>byte</code> FileType	8	This member defines the file type. Possible values are: DESF_FILETYPE_STDDATAFILE, DESF_FILETYPE_BACKUPDATAFILE, DESF_FILETYPE_VALUEFILE, DESF_FILETYPE_LINEARRECORDFILE, DESF_FILETYPE_CYCLICRECORDFILE.
<code>byte</code> CommSet	8	This member defines the communication settings between reader and transponder when the file is accessed. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC
<code>uint16_t</code> AccessRights	16	This member holds the access rights.
<code>union</code> TDESFireSpecificFileInfo SpecificFileInfo	32 to 128	This member holds file type specific information.

Table 23.6: Definition of TDESFireFileSettings

15...12	11...8	7...4	3...0
Read Access	Write Access	Read/Write Access	Change Access Rights

Table 23.7: Coding of AccessRights

Members	Length (Bits)	Description
<code>struct</code> TDESFireDataFileSettings DataFileSettings	32	Definition of data file settings.
<code>struct</code> TDESFireValueFileSettings ValueFileSettings	128	Definition of value file settings.
<code>struct</code> TDESFireRecordFileSettings RecordFileSettings	96	Definition of record file settings.

Table 23.8: Definition of `union` TDESFireSpecificFileInfo

Members	Length (Bits)	Description
<code>uint32_t</code> FileSize	32	Definition of the data file size.

Table 23.9: Definition of `struct` TDESFireDataFileSettings

fully enciphered. Furthermore, the access rights are assigned to certain keys held by the application. Depending on the security settings of the application, a preceding authentication with the application master key may be required, see chapter *Get Key Settings* for details.

Members	Length (Bits)	Description
uint32_t LowerLimit	32	Definition of the lower limit which must not be passed by a debit operation.
uint32_t UpperLimit	32	Definition of the upper limit which must not be passed by a credit operation.
uint32_t LimitedCreditValue	32	Definition of the initial value of the file at file creation.
TValueFileOptions ValueFileOptions	32	Specific options for value files.

Table 23.10: Definition of `struct TDESFireValueFileSettings`

Members	Length (Bits)	Description
byte LimitedCreditEnable	1	Limited Credit feature enabled or disabled.
byte FreeGetValue	1	Free read access enabled or disabled.

Table 23.11: Definition of `struct TValueFileOptions`

Members	Length (Bits)	Description
uint32_t RecordSize	32	Definition of the size of one single record in bytes.
uint32_t MaxNumberOfRecords	32	Definition of the maximum number of records.
uint32_t CurrentNumberOfRecords	32	Definition of the current number of records. This member is ignored at file creation.

Table 23.12: Definition of `struct TDESFireRecordFileSettings`

```

bool DESFire_CreateDataFile
(
    int CryptoEnv,
    int FileNo,
    const TDESFireFileSettings* FileSettings
);

bool DESFire_CreateValueFile
(
    int CryptoEnv,
    int FileNo,
    const TDESFireFileSettings* FileSettings
);

bool DESFire_CreateRecordFile
(
    int CryptoEnv,
    int FileNo,
    const TDESFireFileSettings* FileSettings
);

```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the file ID. If the ID already exists within the application, this results in an error.
<code>const</code> TDESFireFileSettings* FileSettings	This member holds the file settings. See description of TDESFireFileSettings for details.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

Example:

```
// Create new standard data file (without backup)
// in application 0x123456

TDESFireFileSettings FileSettings;
int FileID;

if (DESFire_SelectApplication(0x123456))
{
    // We create a standard data file
    FileSettings.FileType = DESF_FILETYPE_STDDATAFILE;
    // Communication between reader and tag is fully enciphered
    FileSettings.CommSet = DESF_COMMSET_FULLY_ENC;
    // Read Access      : Key 1
    // Write Access     : Key 2
    // Read/Write       : Key 3
    // Change Settings  : Key 4
    FileSettings.AccessRights = 0x1234;
    // File size shall be 512 bytes
    FileSettings.SpecificFileInfo.DataFileSettings.FileSize = 512;
    // Assign an identifier to the file
    FileID = 0x12;
    if (DESFire_CreateDataFile(CRYPTO_ENV0, FileID, &FileSettings))
    {
        DoSomething();
    }
}
```

23.3.2 Delete File

This function allows to permanently deactivate a file within an application. This means, the allocated memory is not released for further usage, only the file number can be re-used for creating a new file. In order to re-use the memory of deleted files, this requires formatting the transponder but this leads to permanent loss of any application data. Depending on the security settings of the application, a preceding authentication with the application master key may be required, see chapter *Get Key Settings* for details.

```
bool DESFire_DeleteFile
(
    int CryptoEnv,
    int FileNo
```

```
);
```

Parameters:

int CryptoEnv Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

int FileNo Specify the ID of the file which shall be deleted. If the ID doesn't exist within the application, this results in an error.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.3.3 Get File IDs

This function allows to list all file IDs that exist within the currently selected application. Each file ID is coded in one byte in the range from 0x00 to 0x1F. Duplicate values are not possible as each file must have an unambiguous identifier. Depending on the security settings of the application, a preceding authentication with the application master key may be required, see chapter *Get Key Settings* for details.

```
bool DESFire_GetFileIDs
(
    int CryptoEnv,
    byte* FileIDList,
    int* FileIDCount,
    int MaxFileIDCount
);
```

Parameters:

int CryptoEnv Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

byte* FileIDList After successful completion of this function, this parameter holds a list of the retrieved file IDs.

int* FileIDCount This parameter holds the number of retrieved file IDs.

int MaxFileIDCount Specify the maximum number of file IDs, that can be stored in the array FileIDList. Note: Up to 32 files can be stored within an application, so take care for proper dimensioning of the array FileIDList.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Example:

See chapter *Get File Settings* for a comprehensive example.

23.3.4 Get File Settings

This function allows to query the file settings of an existing file within an application. The returned information depends on the type of the file. Depending on the security settings of the application, a preceding authentication with the application master key may be required, see chapter *Get Key Settings* for details.

```
bool DESFire_GetFileSettings
(
    int CryptoEnv,
    int FileNo,
    TDESFireFileSettings* FileSettings
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int FileNo</code>	Specify the file ID which shall be queried.
<code>TDESFireFileSettings* FileSettings</code>	This member holds the returned file settings. See description of TDESFireFileSettings for details.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

Example:

```
// Query file settings of all files in application 0x123456
```

```
TDESFireFileSettings FileSettings;
```

```
// An application can hold up to 32 files
```

```
byte FileIDList[32];
```

```
int FileIDCount;
```

```
int i;
```

```
if (DESFire_SelectApplication(0x123456))
{
```

```
    // Gather a list of present file IDs
```

```
    if (DESFire_GetFileIDs(
        CRYPTO_ENV0,
        FileIDList,
        &FileIDCount,
        sizeof(FileIDList)))
    {
```

```
        for (i=0; i<FileIDCount; i++)
        {
```

```
            // Query the settings of each file
```

```
            if (DESFire_GetFileSettings(
                CRYPTO_ENV0,
                FileIDList[i],
                &FileSettings))
            {
```

```
                switch(FileSettings.FileType)
                {
```

```
                    case DESF_FILETYPE_STDDATAFILE:
                        DoSomething();
                        break;
```

```
                    case DESF_FILETYPE_VALUEFILE:
                        DoSomethingElse();
                        break;
```

```
                }
```

```
            }
```

```

    }
}

```

23.3.5 Change File Settings

This function allows to change the access parameters such as communication settings and access rights of an existing file. Depending on the actual change access rights of the file, authentication with the respective key has to be performed before calling this function. Furthermore, the change access right must be different from "deny". See *Coding of Access Rights* for details.

```

bool DESFire_ChangeFileSettings
(
    int CryptoEnv,
    int FileNo,
    int NewCommSet,
    int OldAccessRights,
    int NewAccessRights
);

```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int FileNo</code>	Specify the file ID whose settings shall be changed.
<code>int NewCommSet</code>	Specify the new communication settings. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.
<code>int OldAccessRights</code>	Specify the current Access Rights of the file.
<code>int NewAccessRights</code>	Specify the new Access Rights of the file.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4 File Related Operations

23.4.1 Data Files

23.4.1.1 Read Data

This function shall be used to access a standard or backup data file in order to read from it. Depending on the file's access rights, a preceding authentication with the read or read/write key has to be done, see *Coding of Access Rights* for details. The function allows segmented access, this means the user is able to either read the entire file or only a part starting at a user-defined offset.

```

bool DESFire_ReadData
(
    int CryptoEnv,

```

```

int FileNo,
byte* Data,
int Offset,
int Length,
int CommSet
);

```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int FileNo</code>	Specify the ID of the file that shall be read.
<code>byte* Data</code>	After successful completion of this function, the buffer referred by this parameter holds the data which was read from the transponder. Take care for adequate dimensioning.
<code>int Offset</code>	Specify the starting address for reading. The valid range of this parameter is 0x000000 to FileSize - 1. In case of address-range violation, the function returns with an error.
<code>int Length</code>	Specify the length of data that shall be read. The valid range of this parameter is FileSize - Offset. In case of address-range violation, the function returns with an error.
<code>int CommSet</code>	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Example:

```

// Read data file 0x12 which is located in application 0x123456

TDESFireFileSettings FileSettings;

int ReadAccess;

// This is the buffer that receives the data to be read
byte Data[512];

// If an authentication is necessary, we assume this would be
// the key that gives read access
const byte KeyRead[16] =
{
    0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
    0x88, 0x99, 0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF
};

if (!DESFire_SelectApplication(CRYPTO_ENV0, 0x123456))
    return;    // Error

// Gather file settings
if (!DESFire_GetFileSettings(CRYPTO_ENV0, 0x12, &FileSettings))
    return;    // Error

```



```

// Read access rights are located in the highest nibble of
// FileSettings.AccessRights
ReadAccess = (FileSettings.AccessRights >> 12) & 0x000F;

switch (ReadAccess)
{
case 15:    // Access denied
    return;
case 14:    // Free access
    break;
default:
    // Authenticate with the "reading-key"
    if (!DESFire_Authenticate(
        CRYPTO_ENVO,
        ReadAccess,
        KeyRead,
        DESF_KEYLEN_AES,
        DESF_KEYTYPE_AES,
        DESF_AUTHMODE_EV1))
        return;    // Error
}

// Check size of reading buffer
if (FileSettings.SpecificFileInfo.DataFileSettings.FileSize >
    sizeof(Data))
    return;    // Buffer size not enough

// Read entire data file
if (DESFire_ReadData(
    CRYPTO_ENVO,
    0x12,
    Data,
    0,
    FileSettings.SpecificFileInfo.DataFileSettings.FileSize,
    FileSettings.CommSet))
{
    DoSomething();
}

```

23.4.1.2 Write Data

This function shall be used to access a standard or backup data file in order to write to it. Depending on the file's access rights, a preceding authentication with the write or read/write key has to be done, see *Coding of Access Rights* for details. The function allows segmented access, this means the user is able to either rewrite the entire file or only a part starting at a user-defined offset.

```

bool DESFire_WriteData
(
    int CryptoEnv,
    int FileNo,
    const byte* Data,
    int Offset,
    int Length,
    int CommSet

```

```
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int FileNo</code>	Specify the ID of the file that shall be written.
<code>const byte* Data</code>	The buffer referred by this parameter holds the data which is written to the file.
<code>int Offset</code>	Specify the starting address for writing. The valid range of this parameter is 0x000000 to FileSize - 1. In case of address-range violation, the function returns with an error.
<code>int Length</code>	Specify the length of data that shall be written. The valid range of this parameter is FileSize - Offset. In case of address-range violation, the function returns with an error.
<code>int CommSet</code>	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.
<u>Return:</u>	If the operation was successful, the return value is true, otherwise it is false.

Remark: If data is written to a Backup Data File, it is necessary to validate the written data with the function *Commit Transaction*. Calling the function *Abort Transaction* will invalidate all changes.

Example:

```
// Write to data file 0x12 which is located in application 0x123456

TDESFireFileSettings FileSettings;

int WriteAccess;

// This is the buffer that holds the data to be written
const byte Data[] =
{
    0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08
};

// If an authentication is necessary, we assume this would be
// the key that gives write access
const byte KeyWrite[16] =
{
    0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
    0x88, 0x99, 0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF
};

if (!DESFire_SelectApplication(CRYPTO_ENV0, 0x123456))
    return;    // Error

// Gather file settings
if (!DESFire_GetFileSettings(CRYPTO_ENV0, 0x12, &FileSettings))
    return;    // Error
```

```

// Write access rights are located in bits 11...8 of
// FileSettings.AccessRights
WriteAccess = (FileSettings.AccessRights >> 8) & 0x000F;

switch (WriteAccess)
{
case 15:    // Access denied
    return;
case 14:    // Free access
    break;
default:
    // Authenticate with the "writing-key"
    if (!DESFire_Authenticate(
        CRYPTO_ENVO,
        WriteAccess,
        KeyWrite,
        DESF_KEYLEN_AES,
        DESF_KEYTYPE_AES,
        DESF_AUTHMODE_EV1))
        return;    // Error
}

// Check size of file
if (FileSettings.SpecificFileInfo.DataFileSettings.FileSize <
    sizeof(Data))
    return;    // File size not enough

// Write to data file
if (DESFire_WriteData(
    CRYPTO_ENVO,
    0x12,
    Data,
    0,
    sizeof(Data),
    FileSettings.CommSet))
{
    DoSomething();
}

```

23.4.2 Value Files

23.4.2.1 Get Value

This function allows to read the current value from a Value File. Depending on the file's access rights, a preceding authentication with the read, write or read/write key has to be done, see *Coding of Access Rights* for details.

```

bool DESFire_GetValue
(
    int CryptoEnv,
    int FileNo,
    int* Value,
    int CommSet
);

```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the Value File whose value shall be queried.
<code>int*</code> Value	After successful completion of this function, this parameter holds the value which was read from the file.
<code>int</code> CommSet	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4.2.2 Debit

This function allows to decrease a value stored in a Value File. The function requires a preceding authentication with the read, write or read/write key, see *Coding of Access Rights* for details. The value modifications of *Credit*, *Debit* and *Limited Credit* functions are cumulated until the function *Commit Transaction* is called.

If the *Limited Credit* feature is enabled, the new limit for a subsequent *Limited Credit* function call is set to the sum of *Debit* modifications within one transaction before calling *Commit Transaction*. This assures, that a *Limited Credit* can not re-book more values than a debiting transaction deducted before.

```
bool DESFire_Debit
(
    int CryptoEnv,
    int FileNo,
    const int Value,
    int CommSet
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the Value File that shall be debited.
<code>const int</code> Value	The value stored in the value file will be decreased by this parameter.
<code>int</code> CommSet	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: After modifying value files, it is necessary to validate the transaction with the function *Commit Transaction*. Calling the function *Abort Transaction* will invalidate all changes.

23.4.2.3 Credit

This function allows to increase a value stored in a Value File. The function requires a preceding authentication with the read/write key, see *Coding of Access Rights* for details. The value modifications of *Credit*, *Debit* and *Limited Credit* functions are cumulated until the function *Commit Transaction* is called.

If the *Limited Credit* feature is enabled, this function cannot be used. Use the function *Limited Credit* instead.

```
bool DESFire_Credit
(
    int CryptoEnv,
    int FileNo,
    const int Value,
    int CommSet
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int FileNo</code>	Specify the ID of the Value File that shall be credited.
<code>const int Value</code>	The value stored in the value file will be increased by this parameter.
<code>int CommSet</code>	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: After modifying value files, it is necessary to validate the transaction with the function *Commit Transaction*. Calling the function *Abort Transaction* will invalidate all changes.

23.4.2.4 Limited Credit

This function allows a limited increase of a value stored in a Value File without having full read/write permissions to the file. This feature can only be used if it has been enabled during file creation. The function requires a preceding authentication with the write or read/write key, see *Coding of Access Rights* for details. The value modifications of *Credit*, *Debit* and *Limited Credit* functions are cumulated until the function *Commit Transaction* is called.

After calling this function, the new limit is set to 0, regardless of the amount which has been re-booked. Hence, this function can only be used once after a Debit transaction.

```
bool DESFire_LimitedCredit
(
    int CryptoEnv,
    int FileNo,
    const int Value,
    int CommSet
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the Value File that shall be credited.
<code>const int</code> Value	The value stored in the value file will be increased by this parameter. It is limited to the sum of Debit operations on this value file within the most recent transaction containing at least one Debit.
<code>int</code> CommSet	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: After modifying value files, it is necessary to validate the transaction with the function *Commit Transaction*. Calling the function *Abort Transaction* will invalidate all changes.

23.4.3 Record Files**23.4.3.1 Read Records**

Use this function to read out a set of complete records from a Record File. The function requires a preceding authentication with the read or read/write key, see *Coding of Access Rights* for details.

```
bool DESFire_ReadRecords
(
    int CryptoEnv,
    int FileNo,
    byte* RecordData,
    int* RecDataByteCnt,
    int Offset,
    int NumberOfRecords,
    int RecordSize,
    int CommSet
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the file that shall be read.
<code>byte*</code> RecordData	After successful completion of this function, the buffer referred by this parameter holds the data which was read from the transponder. Take care for adequate dimensioning.
<code>int*</code> RecDataByteCnt	The total number of bytes read from the transponder is represented by this parameter.
<code>int</code> Offset	Specify the offset of the newest record to be read out. The valid range of this parameter is 0x000000 to number of existing records - 1. In case of 0x000000 the latest record is read out.
<code>int</code> NumberOfRecords	Specify the number of records to be read out.
<code>int</code> CommSet	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4.3.2 Write Record

Use this function to write data to a Record File. The function requires a preceding authentication with the write or read/write key, see *Coding of Access Rights* for details. In order to validate writing, a call of *Commit Transaction* becomes necessary. If writing is not validated, a new WriteRecord command writes to the already created record.

```
bool DESFire_WriteRecord
(
    int CryptoEnv,
    int FileNo,
    const byte* Data,
    int Offset,
    int Length,
    int CommSet
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the file that shall be read.
<code>const byte*</code> Data	This buffer holds the record data to be written.
<code>int</code> Offset	Specify the offset in bytes within one single record. The valid range of this parameter is 0x000000 to record size - 1.
<code>int</code> Length	Specify the length of data to be written. The parameter has to be in the range from 0x000001 to record size - offset.
<code>int</code> CommSet	Specify the communication settings. The communication settings must match to the actual settings of the file. Possible values are: DESF_COMMSET_PLAIN, DESF_COMMSET_PLAIN_MACED, DESF_COMMSET_FULLY_ENC.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4.3.3 Clear Record File

Use this function to reset a Record File to the empty state. The function requires a preceding authentication with the read/write key, see *Coding of Access Rights* for details. After execution of the function, a call of *Commit Transaction* becomes necessary.

```
bool DESFire_ClearRecordFile(int CryptoEnv, int FileNo);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.
<code>int</code> FileNo	Specify the ID of the file that shall be cleared.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4.4 Commit Transaction

This function allows to validate all previous modifications on files with integrated backup mechanism such as Backup Data Files, Value Files and Record Files. When a transaction has been finished, this is usually the last called function; if this step was omitted, any changes would be lost if a different application is selected or the transponder is removed from the RF-field.

```
bool DESFire_CommitTransaction
(
    int CryptoEnv
);
```


Parameters:

`int CryptoEnv` Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

23.4.5 Abort Transaction

This function allows to discard all previous modifications on files with integrated backup mechanism such as Backup Data Files, Value Files and Record Files.

```
bool DESFire_AbortTransaction
(
    int CryptoEnv
);
```

Parameters:

`int CryptoEnv` Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants. Usually the same environment is specified that was used for authentication.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

24 SAM AV1/AV2

Before using one of the following functions, a NXP SAM AV1/AV2 card must have been inserted into one of the available SAM slots. When powering up, TWN4 scans the slots for SAM cards, so a correctly inserted SAM card is detected automatically for later use.

24.1 Host Authentication

This function shall be used to perform a mutual three pass authentication between host (reader) and the SAM AV1/AV2 card. The function supports both 3DES and AES cryptography. Depending on security settings of the SAM card, the authentication might be necessary in order to perform different security related actions afterwards.

```
bool SAMAVx_AuthenticateHost
(
    int CryptoEnv,
    int KeyNo,
    const byte* Key,
    int KeyByteCount,
    int KeyType
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int KeyNo</code>	Specify the key number that shall be used for authentication.
<code>const byte* Key</code>	Specify the key that shall be used for authentication. For 3DES/AES, the key must have a key length of 16 bytes.
<code>int KeyByteCount</code>	Specify the key length of the key. Use one of the predefined constants DESF_KEYLEN_3DES or DESF_KEYLEN_AES.
<code>int KeyType</code>	Specify the type of the specified key. Use one of the predefined constants DESF_KEYTYPE_3DES or DESF_KEYTYPE_AES. The authentication will be performed according to the specified key type.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

24.2 Query Key Entry

Use this function to query information about a key entry on the SAM card.

```
bool SAMAVx_GetKeyEntry(int KeyNo, TSAMAVxKeyEntryData* KeyEntryData);
```

Parameters:

int KeyNo Specify the key number that shall be used for authentication.

TSAMAVxKeyEntryData* KeyEntryData The key entry is returned by this parameter.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Members	Length (Bits)	Description
byte VersionKeyA	8	This member holds the version of Key A.
byte VersionKeyB	8	This member holds the version of Key B.
byte VersionKeyC	8	This member holds the version of Key C.
uint32_t DF_AID	32	This member holds the associated DESFire AID.
byte DF_KeyNo	8	This member holds the associated DESFire key number.
byte KeyNoCEK	8	This member holds the key number of the change entry key.
byte KeyNoVCEK	8	This member holds the key version of the change entry key.
byte RefNoKUC	8	This member holds the number of the associated Key Usage Counter.
uint16_t SET	16	This member holds the configuration settings of the key entry.

Table 24.1: Definition of TSAMAVxKeyEntryData

25 ISO15693 Specific Transponder Operations

25.1 Generic ISO15693 Command

This function can be used for ISO15693 specific transponder operations which are not covered by high-level system functions.

```
bool ISO15693_GenericCommand
(
    byte Flags,
    byte Command,
    byte* Data,
    int* Length,
    int BufferSize
);
```

Parameters:

<code>byte</code> Flags	Specify the ISO15693 flags. Note: TWN4 Core Mifare based readers only support dual subcarrier mode at low datarate. All other readers support single subcarrier mode at high datarate.
<code>byte</code> Command	Command code.
<code>byte*</code> Data	This parameter works as Input/Output-buffer. All additional parameters which are sent to the transponder are passed within this buffer. This buffer is also used for data returned from the transponder. The reader internal send/receive buffer is 256 bytes.
<code>int*</code> Length	This parameter works as Input/Output-variable. It holds the payload-length of Data in the directions <i>Reader</i> → <i>Tag</i> and <i>Tag</i> → <i>Reader</i> .
<code>int</code> BufferSize	This parameter holds the array-size of Data in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

25.1.1 Example of creating a Standard ISO15693 Command

```
// Read Single Block Command
byte Flags = 0x11;           // Dual Subcarrier, low datarate, Select Flag is set
byte Command = 0x20;        // Command Code of READ_SINGLE_BLOCK
byte Data[16] = { 0x00 };    // Shared buffer for command arguments and tag response, we want to read block
int Length = 1;             // Payload length for input and output
if (ISO15693_GenericCommand(Flags, Command, Data, &Length, sizeof(Data)))
{
    DoSomething();
}
```

Members	Length (Bits)	Description
<code>byte</code> DSFID_Present	1	Set to 1 if DSFID is present
<code>byte</code> AFI_Present	1	Set to 1 if AFI is present
<code>byte</code> VICC_Memory_Size_Present	1	Set to 1 if BlockSize and Number_of_Blocks are present
<code>byte</code> IC_Reference_Present	1	Set to 1 if IC_Reference is present
<code>byte</code> Res1	4	Reserved for future use
<code>byte</code> UID[8]	64	Unique Identifier
<code>byte</code> DSFID	8	Data Storage Format Identifier
<code>byte</code> AFI	8	Application Family Identifier
<code>byte</code> BlockSize	8	Size of one data block in bytes
<code>uint16_t</code> Number_of_Blocks	16	Number of available blocks
<code>byte</code> IC_Reference	8	Meaning defined by the IC manufacturer

Table 25.1: Definition of TISO15693_SystemInfo

25.2 Gather Tag Specific Information

25.2.1 Get System Information

This function returns more in-depth information of the tag. The function is available in two versions (Protocol Extension flag set or reset), as some tag types like ST 24LR16/64 require the Protocol Extension flag to be set for proper operation.

```
bool ISO15693_GetSystemInformation
(
    TISO15693_SystemInfo* SystemInfo
);

bool ISO15693_GetSystemInformationExt
(
    TISO15693_SystemInfo* SystemInfo
);
```

Parameters:

`TISO15693_SystemInfo* SystemInfo` Pointer to the structure which receives the System Information.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: As the GetSystemInformation command is no mandatory ISO15693 command, it is not implemented in all tag types available on the market.

Definition	Value	Manufacturer	Tag Type
ISO15693_TAGTYPE_ICODESL2	0x00	NXP	ICode SL2
ISO15693_TAGTYPE_ICODESL2S	0x01		ICode SL2S
ISO15693_TAGTYPE_UNKNOWNNXP	0x0F		Unknown
ISO15693_TAGTYPE_TAGITHFIPLUSINLAY	0x10	TI	Tag-It HFI Plus Inlay
ISO15693_TAGTYPE_TAGITHFIPLUSCHIP	0x11		Tag-It HFI Plus Chip
ISO15693_TAGTYPE_TAGITHFISTD	0x12		Tag-It HFI Standard
ISO15693_TAGTYPE_TAGITHFIPRO	0x13		Tag-It HFI Pro
ISO15693_TAGTYPE_UNKNOWNNTI	0x1F		Unknown
ISO15693_TAGTYPE_UNKNOWNST	0x4F	ST	Unknown
ISO15693_TAGTYPE_SRF55V02P	0x50	Infineon	SRF55V02P
ISO15693_TAGTYPE_SRF55V10P	0x51		SRF55V10P
ISO15693_TAGTYPE_SRF55V02S	0x52		SRF55V02S
ISO15693_TAGTYPE_SRF55V10S	0x53		SRF55V10S
ISO15693_TAGTYPE_UNKNOWNNINFINEON	0x5F		Unknown
ISO15693_TAGTYPE_UNKNOWN	0xFF	Unknown	Unknown ISO15693

Table 25.2: Retrievable tag types from UID

25.2.2 Get Tag Type

The ISO15693 API incorporates two methods to determine the tag type, either by analysing the UID or the System Information structure.

25.2.2.1 Get Tag Type From UID

This function can be used to determine the tag type of ISO15693 compliant transponders if only the UID is available.

```
int ISO15693_GetTagTypeFromUID
(
    byte* UID
);
```

Parameters:

byte* UID

This parameter holds the UID. Watch for the correct byte order; UID[0] shall have the value 0xE0

Return:

The return-value is the determined tag-type which is represented by one of the constants in the table below.

25.2.2.2 Get Tag Type From System Information

This function can be used to determine the tag type of ISO15693 compliant transponders if the System Information is available.

```
int ISO15693_GetTagTypeFromSystemInfo  
(  
    TISO15693_SystemInfo* SystemInfo  
);
```

Parameters:

TISO15693_SystemInfo* Pointer to the structure which holds the System Information.
SystemInfo

Return: The return-value is the determined tag-type which is represented by one of the constants in the table below.

Definition	Value	Manufacturer	Tag Type
ISO15693_TAGTYPE_ICODESL2	0x00	NXP	ICode SL2
ISO15693_TAGTYPE_ICODESL2S	0x01		ICode SL2S
ISO15693_TAGTYPE_UNKNOWNNXP	0x0F		Unknown
ISO15693_TAGTYPE_TAGITHFIPLUSINLAY	0x10	TI	Tag-It HFI Plus Inlay
ISO15693_TAGTYPE_TAGITHFIPLUSCHIP	0x11		Tag-It HFI Plus Chip
ISO15693_TAGTYPE_TAGITHFISTD	0x12		Tag-It HFI Standard
ISO15693_TAGTYPE_TAGITHFIPRO	0x13		Tag-It HFI Pro
ISO15693_TAGTYPE_UNKNOWNNTI	0x1F		Unknown
ISO15693_TAGTYPE_MB89R118	0x20	Fuji	MB89R118
ISO15693_TAGTYPE_MB89R119	0x21		MB89R119
ISO15693_TAGTYPE_MB89R112	0x22		MB89R112
ISO15693_TAGTYPE_UNKNOWNFUJI	0x2F		Unknown
ISO15693_TAGTYPE_24LR16	0x30	ST	24LR16
ISO15693_TAGTYPE_24LR64	0x31		24LR64
ISO15693_TAGTYPE_LRI1K	0x40		LRI1K
ISO15693_TAGTYPE_LRI2K	0x41		LRI2K
ISO15693_TAGTYPE_LRIS2K	0x42		LRIS2K
ISO15693_TAGTYPE_LRIS64K	0x43		LRIS64K
ISO15693_TAGTYPE_UNKNOWNST	0x4F		Unknown
ISO15693_TAGTYPE_SRF55V02P	0x50	Infineon	SRF55V02P
ISO15693_TAGTYPE_SRF55V10P	0x51		SRF55V10P
ISO15693_TAGTYPE_SRF55V02S	0x52		SRF55V02S
ISO15693_TAGTYPE_SRF55V10S	0x53		SRF55V10S
ISO15693_TAGTYPE_UNKNOWNINFINEON	0x5F		Unknown
ISO15693_TAGTYPE_UNKNOWN	0xFF	Unknown	Unknown ISO15693

Table 25.3: Retrievable tag types from System Information

25.3 Read/Write Data

25.3.1 Read Single Block

Read a single data block from the transponder. The function is available in two versions (Protocol Extension flag set or reset), as some tag types like ST 24LR16/64 require the Protocol Extension flag to be set for proper operation.

```
bool ISO15693_ReadSingleBlock
(
    int BlockNumber,
    byte* BlockData,
    int* Length,
    int BufferSize
);

bool ISO15693_ReadSingleBlockExt
(
    int BlockNumber,
    byte* BlockData,
    int* Length,
    int BufferSize
);
```

Parameters:

<code>int</code> BlockNumber	This parameter holds the number of the block to be read.
<code>byte*</code> BlockData	This parameter holds the data which was read from the tag if the operation was successful. Note that the block size varies between different tag types, so the array size of BlockData should be set to a reasonable value.
<code>int*</code> Length	This parameter holds the length of data which was read from the tag in bytes.
<code>int</code> BufferSize	This parameter holds the array-size of BlockData in bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

25.3.2 Write Single Block

Write to a single data block of the transponder. The function is available in two versions (Protocol Extension flag set or reset), as some tag types like ST 24LR16/64 require the Protocol Extension flag to be set for proper operation.

```
bool ISO15693_WriteSingleBlock
(
    int BlockNumber,
    const byte* BlockData,
    int Length
);

bool ISO15693_WriteSingleBlockExt
(
    int BlockNumber,
    const byte* BlockData,
```

```
int Length  
);
```

Parameters:

`int` BlockNumber

This parameter holds the number of the block to be written.

`const byte*` BlockData

This parameter holds the data which shall be written to the tag.

`int` Length

This parameter holds the length of data which shall be written to the tag in bytes.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

26 LEGIC-Specific Functions

This chapter describes functions for accessing LEGIC functionality.

Notes:

- These functions are available at TWN4 MultiTech LEGIC only.
- The style of functions has been changed due to additional support of SM4500: All functions are starting with SM4X00 instead of SM4200. Old-style functions are supported via macros.

26.1 Direct Access of LEGIC Chip

TWN4 MultiTech LEGIC has a built-in LEGIC chip type SM4200 or SM4500. There are functions available to directly communicate with this chipset.

Note:

Due to license restrictions, this documentation only mentions the functions itself. In order to use full functionality of the LEGIC chip, appropriate documentation is required, which is available under NDA (none-disclosure agreement) only.

26.1.1 SM4X00_GenericRaw

Send a command and receive the response from SM4X00. Command and response are expected to include CRC. This function is intended to be used for end-to-end communication between SM4X00 and a host.

```
bool SM4X00_GenericRaw(const byte *TXData, int TXDataLength,
                       byte *RXData, int *RXDataLength,
                       int MaxRXDataLength, int Timeout);
```

Parameters:

const byte *TXData	Pointer to an array of bytes, which contains the command to be sent to SM4X00.
int TXDataLength	Number of bytes to be sent to SM4X00.
byte *RXData	Pointer to an array of bytes, which receives response from SM4X00
int *RXDataLength	Pointer to an integer, which receives the actually read number of bytes.
int MaxRXDataLength	A value, which specifies the maximum number of bytes, which can be received byte RXData, thus the buffer size.
int Timeout	Maximum time, the function should wait for a response from SM4X00. This value is specified in milliseconds.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

26.1.2 SM4X00_Generic

Send a command and receive the response from SM4X00. This function is intended to be used by standard-along applications.

```
bool SM4X00_Generic(const byte *TXData, int TXDataLength,
                    byte *RXData, int *RXDataLength,
                    int MaxRXDataLength, int Timeout);
```

Parameters:

const byte *TXData	Pointer to an array of bytes, which contains the command to be sent to SM4X00. The command has to be specified W/O leading length byte and W/O closing CRC value.
int TXDataLength	Number of bytes contained in TXData.
byte *RXData	Pointer to an array of bytes, which receives response from SM4X00. Received data is W/O length byte and W/O CRC value.
int *RXDataLength	Pointer to an integer, which receives length of the actually received payload.
int MaxRXDataLength	A value, which specifies the maximum number of bytes, which can be received by RXData, thus the buffer size.
int Timeout	Maximum time, the function should wait for a response from SM4X00. This value is specified in milliseconds.

Return: If the operation was successful, the return value is true, otherwise it is false.

26.1.3 SM4X00_StartBootloader

Start boot loader of SM4X00.

```
bool SM4X00_StartBootloader(byte *TLV, int *TLVLength, int MaxTLVLength)
```

Parameters:

```
byte *TLV
int *TLVLength
int MaxTLVLength
```

Return: If the operation was successful, the return value is true, otherwise it is false.

26.1.4 SM4X00_EraseFlash

Erase flash of SM4X00.

```
bool SM4X00_EraseFlash(void)
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is false.

26.1.5 SM4X00_ProgramBlock

Program one block of data into the flash of SM4X00.

```
bool SM4X00_ProgramBlock(byte *Data, bool *Done)
```

Parameters:

byte *Data	Pointer to an array of bytes.
bool *Done	Pointer to a boolean variable, which receives the status, if the last block was flashed.

Return: If the operation was successful, the return value is true, otherwise it is false.

27 iCLASS Specific Transponder Operations

This chapter handles specific operations with iCLASS transponders. Prerequisites for this functionality are:

- The reader must be the TWN4 MultiTech/MultiTech Nano version, LEGIC is not supported.
- An iCLASS SIO card must be inserted into one of the SAM slots.
- The I-Option must be activated.
- For iCLASS Seos support, the SIO card must have firmware 1.19 or higher.

27.1 Read PAC Bits

This function can be used to read the PAC (Physical Access Control) bits from an iCLASS transponder. The transponder must have been selected before this function can be called.

```
bool ICLASS_GetPACBits
(
    byte* PACBits,
    int* PACBitCnt,
    int MaxPACBytes
);
```

Parameters:

<code>byte* PACBits</code>	After successful completion of this function, the buffer referred by this parameter holds the PAC bits read from the transponder. Take care for adequate dimensioning.
<code>int* PACBitCnt</code>	After successful completion of this function, this parameter holds the number of bits, the PAC contains.
<code>int MaxPACBytes</code>	This parameter holds the maximum number of bytes which the buffer PACBits can hold.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

Remark: There are transponders available, that have not been configured to deliver the PAC bits. In this case, if an attempt is made to read these bits, the function returns `false`.

27.1.1 Example

The following example shows how to manually read the PAC from an iCLASS transponder using the built-in system functions.

```

byte ID[8];
int TagType;
int IDBitCnt;

byte PACBits[8];
int PACBitCnt;

// Search only for iCLASS transponders
SetTagTypes(0, TAGMASK(HFTAG_HIDICLASS));

while (true)
{
    // Search for transponders
    if (!SearchTag(&TagType,&IDBitCnt,ID,sizeof(ID)))
        continue;

    // Read the PAC bits
    if (!ICLASS_GetPACBits(PACBits, &PACBitCnt, sizeof(PACBits)))
        continue;

    // Output what was read from the card
    HostWriteHex(PACBits, PACBitCnt, (PACBitCnt+7)/8*2);
    HostWriteChar('\r');
}

```

27.2 Select Page

Use this function to select a transponder book and page. In case of success, the Configuration Block is returned. Note that this function is not supported by some iCLASS transponders, e.g. iCLASS 2k/2.

```

bool ICLASS_SelectPage
(
    int Book,
    int Page,
    TICLASS_ConfigBlock* ConfigBlock
);

```

Parameters:

<code>int Book</code>	Specify the book number to be selected.
<code>int Page</code>	Specify the page number to be selected.
<code>TICLASS_ConfigBlock* ConfigBlock</code>	The content of the Configuration Block of the selected page is returned by this buffer.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

27.3 Authenticate

Use this function to perform the mutual authentication procedure. Authentication may be required to have read-/write access to protected data blocks. Either a Debit or a Credit key can be used for Application Area

1 or 2 respectively. Before this function can be used, book and page have to be selected in advance.

```
bool ICLASS_Authenticate
(
    const byte* KeyReferenceOID,
    int KeyType
);
```

Parameters:

const byte* KeyReferenceOID Specify the OID of the key to be used for authentication. The OID is always 3 bytes in length, the first byte must be 0x03.

int KeyType Specify the type of key, use one of the predefined constants ICLASS_KEYTYPE_CREDIT or ICLASS_KEYTYPE_DEBIT.

Return: If the operation was successful, the return value is true, otherwise it is false.

27.4 Read Data Block

Use this function to read a single 8 byte block of an iCLASS transponder. If a read operation to a protected block is carried out, the function returns all 0xFF as data. So it is necessary to select a page and perform authentication prior read access.

```
bool ICLASS_ReadBlock
(
    int Block,
    byte* BlockData
);
```

Parameters:

int Block Specify the block number to be read.

byte* BlockData This buffer holds the block data read from the transponder. The block size is always 8 bytes so take care for proper dimensioning of the buffer size.

Return: If the operation was successful, the return value is true, otherwise it is false.

27.5 Write Data Block

Use this function to write a 8 byte data block to an iCLASS transponder. Before a write operation can be carried out, a page must be selected and authenticated.

```
bool ICLASS_WriteBlock
(
    int Block,
    const byte* BlockData
);
```


Parameters:

<code>int</code> Block	Specify the block number to be written.
<code>const byte*</code> BlockData	This buffer holds the block data to be written to the transponder. The block size is always 8 bytes.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

28 FeliCa

This chapter handles specific operations of contactless transponders that support FeliCa technology. Before one of the following functions can be used, the transponder must have been selected using the function `SearchTag(...)`.

28.1 Polling

Use this function to acquire a card by specifying a System Code. The transponder only answers if the specified System Code matches to a system stored on the card. By specifying a wildcard (0xFF) for either the upper or lower byte, a particular match of System Code can be achieved.

```
bool FeliCa_Poll(uint16_t SystemCode, byte* IDm, byte* PMm);
```

Parameters:

uint16_t SystemCode	Specify the two-byte System Code by this parameter.
byte* IDm	The Manufacture ID is returned by this buffer. The function always returns 8 bytes.
byte* PMm	The Manufacture Parameter is returned by this buffer. The function always returns 8 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

28.2 Request System Code

Use this function to acquire a list of System Codes which are available on the card. This function does not work with FeliCa Lite or FeliCa Plug ICs.

```
bool FeliCa_RequestSystemCode
(
    int* NumberOfSystemCodes,
    uint16_t* SystemCodeList,
    int MaxNumberOfSystemCodes
);
```

Parameters:

<code>int*</code> <code>NumberOfSystemCodes</code>	This parameter holds the number of retrieved System Codes.
<code>uint16_t*</code> <code>SystemCodeList</code>	This parameter holds the list of System Codes which are available on the card.
<code>int</code> <code>MaxNumberOfSystemCodes</code>	Specify the maximum number of System Codes, that can be stored in the array <code>SystemCodeList</code> .

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

28.3 Request Service

Use this function to verify the existence of Area and Service Codes. The function returns the Key Version of existing Area and System Codes. If the specified Area or System does not exist, the respective Key Version is 0xFFFF. This function does not work with FeliCa Lite or FeliCa Plug ICs.

```
bool FeliCa_RequestService
(
    int NumberOfServices,
    const uint16_t* ServiceCodeList,
    uint16_t* KeyVersionList
);
```

Parameters:

<code>int</code> <code>NumberOfServices</code>	This parameter specifies the size of <code>ServiceCodeList</code> .
<code>const</code> <code>uint16_t*</code> <code>ServiceCodeList</code>	This array holds the list of Service Codes that shall be queried.
<code>uint16_t*</code> <code>KeyVersionList</code>	The queried KeyVersions are returned by this array. It has the same size as <code>ServiceCodeList</code> , each KeyVersion is assigned to the order of appearance of <code>ServiceCodeList</code> .

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

28.4 Read Without Encryption

Use this function to read blocks of data from a authentication-not-required service. This function works with all transponders supporting FeliCa technology.

```
bool FeliCa_ReadWithoutEncryption
(
    int NumberOfServices,
    const uint16_t* ServiceCodeList,
    int NumberOfBlocks,
    const uint16_t* BlockList,
    byte* Data
);
```

Parameters:

<code>int</code> NumberOfServices	This parameter specifies the size of ServiceCodeList.
<code>const uint16_t*</code> ServiceCodeList	This array holds the list of Service Codes. Currently, one Service Code can be specified.
<code>int</code> NumberOfBlocks	This parameter specifies the number of blocks that shall be read. The current implementation allows reading of four blocks at a time.
<code>const uint16_t*</code> BlockList	This array holds the list of blocks that shall be read.
<code>byte*</code> Data	Block data which was read from the card is returned by this buffer. A block has always 16 bytes of data, so the buffer must be dimensioned depending on the number of blocks that shall be read. The block data is returned in the order of appearance of the values of BlockList.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

28.5 Write Without Encryption

Use this function to write blocks of data to a authentication-not-required service. This function works with all transponders supporting FeliCa technology.

```
bool FeliCa_WriteWithoutEncryption
(
    int NumberOfServices,
    const uint16_t* ServiceCodeList,
    int NumberOfBlocks,
    const uint16_t* BlockList,
    const byte* Data
);
```

Parameters:

<code>int</code> NumberOfServices	This parameter specifies the size of ServiceCodeList.
<code>const uint16_t*</code> ServiceCodeList	This array holds the list of Service Codes. Currently, one Service Code can be specified.
<code>int</code> NumberOfBlocks	This parameter specifies the number of blocks that shall be written. The current implementation allows writing of four blocks at a time.
<code>const uint16_t*</code> BlockList	This array holds the list of blocks that shall be written.
<code>const byte*</code> Data	Block data which shall be written to the card. A block has always 16 bytes of data, so the buffer must hold NumberOfBlocks multiplied by 16 bytes of data. The block data must be arranged in the order of appearance of the values of BlockList.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

28.6 Transparent Data Exchange

This function can be used for transparent exchange of data between reader and FeliCa transponders, e.g. for transponder commands which are not covered by the current implementation of the reader API.

```
bool FeliCa_TDX
(
    byte* TXRX,
    int TXByteCnt,
    int* RXByteCnt,
    int MaxRXByteCnt,
    byte MaximumResponseTime,
    byte NumberOfBlocks
);
```

Parameters:

<code>byte* TXRX</code>	This buffer holds the byte-string that shall be transmitted to the transponder. The response of the transponder is also returned by this parameter. Take care for adequate dimensioning.
<code>int TXByteCnt</code>	This parameter holds the number of bytes that shall be transmitted to the transponder.
<code>int* RXByteCnt</code>	After successful completion of this function, this parameter holds the number of bytes that the transponder response contains.
<code>int MaxRXByteCnt</code>	This parameter holds the array-size of TXRX in bytes.
<code>byte MaximumResponseTime</code>	This parameter holds the parameter byte which shall be used for calculation of the Maximum Response Time according to the calculation formula.
<code>byte NumberOfBlocks</code>	This parameter holds the value n which shall be used for calculation of the Maximum Response Time according to the calculation formula.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

29 Topaz Specific Transponder Operations

29.1 Read UID

Use this function to manually read byte 0 to 3 of the UID and the header ROM. The remaining bytes 4 to 6 of the UID can be read e.g. by using the function `Topaz_ReadByte`.

```
bool Topaz_RID(byte* HR0, byte* HR1, byte* UID);
```

Parameters:

`byte*` HR0 The byte HR0 of the Header ROM is returned by this parameter.

`byte*` HR1 The byte HR1 of the Header ROM is returned by this parameter.

`byte*` UID The UID bytes 0 to 3 are returned by this buffer.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

29.2 Read Data

In order to read data from memory of Topaz transponders, there are two functions available. You may choose between reading one single byte or even read the entire memory space.

29.2.1 Read Single Byte

Use this function to read one single byte from the memory of the transponder.

```
bool Topaz_ReadByte(const byte* UID, byte ADD, byte* Byte);
```

Parameters:

`const byte*` UID Specify byte 0 to 3 of the UID by this parameter.

`byte` ADD Specify the address in memory to be read from.

`byte*` Byte This parameter holds the byte which was read from the transponder.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

29.2.2 Read All Transponder Data

Use this function to read the entire memory of the transponder.

```
bool Topaz_ReadAllBlocks(const byte* UID, byte* HR0, byte* HR1, byte* Data);
```

Parameters:

<code>const byte* UID</code>	Specify byte 0 to 3 of the UID by this parameter.
<code>byte* HR0</code>	The byte HR0 of the Header ROM is returned by this parameter.
<code>byte* HR1</code>	The byte HR1 of the Header ROM is returned by this parameter.
<code>byte* Data</code>	The transponder memory data is returned by this buffer. The function returns 120 bytes, so take care for proper dimensioning.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

29.3 Write Data

In order to write data to the memory of Topaz transponders, there are two functions available. You may chose between programming of data with preceding erase or without erase. So if the variant without erase is selected, this results in logical ORing of data bits. In the initial state, all writeable data bytes of Topaz are 0x00.

29.3.1 Write Single Byte With Erase

Use this function to write one byte to the memory of the transponder. A preceding erase cycle is performed prior programming takes place.

```
bool Topaz_WriteByteWithErase(const byte* UID, byte ADD, byte Byte);
```

Parameters:

<code>const byte* UID</code>	Specify byte 0 to 3 of the UID by this parameter.
<code>byte ADD</code>	Specify the address in memory to be written to.
<code>byte Byte</code>	This parameter holds the byte to be written.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

29.3.2 Write Single Byte Without Erase

Use this function to write one byte to the memory of the transponder. As no preceding erase cycle is performed prior programming, the overall operation results in logical ORing of the existing data byte on the transponder and the byte to be written.

```
bool Topaz_WriteByteNoErase(const byte* UID, byte ADD, byte Byte);
```

Parameters:`const byte*` UID

Specify byte 0 to 3 of the UID by this parameter.

`byte` ADD

Specify the address in memory to be written to.

`byte` Byte

This parameter holds the byte to be written.

Return:If the operation was successful, the return value is `true`, otherwise it is `false`.

30 CTS Specific Transponder Operations

TWN4 offers support of CTS256B and CTS512B transponders. In order to read data from the memory, the respective transponder must be selected by issuing the function `SearchTag()`. After successful search, the transponder memory is ready for read and write operations. CTS transponders are organized in 16 or 32 blocks, each block contains 2 bytes of data.

Please note: CTS256B transponders remain silent after a write operation has been issued. As a consequence, the respective write function returns an error. In order to handle this, it is recommended to perform a verification read of the respective data block.

30.1 Read Block Data

Use this function in order to read a block of data from the transponder memory.

```
bool CTS_ReadBlock(int Block, byte* Data);
```

Parameters:

<code>int Block</code>	Specify the address of the block to be read. The valid range of this parameter depends on the transponder type.
<code>byte* Data</code>	This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 2 bytes of data, so the minimum array size of <code>Data</code> must be at least 2 bytes.

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

30.2 Write Block Data

Use this function to write data to a block of the transponder memory. This function performs a logical OR-ing of already present block data and new data to be written. If the block shall be overwritten regardless of its current content, the function `CTS_UpdateBlock` shall be used instead.

```
bool CTS_WriteBlock(int Block, const byte* Data);
```

Parameters:

- `int` Block Specify the address of the block to be written. The valid range of this parameter depends on the transponder type.
- `const byte*` Data This parameter holds the data which shall be written to the tag. Note that this function always writes 2 bytes of data, so the minimum array size of Data must be at least 2 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

30.3 Update Block Data

Use this function to write data to a block of the transponder memory. Data is written regardless of the current content of the specified data block. Please note: Prior execution of this function, the transponder system bits have to be refreshed in order to make this function work. This can be achieved by reading block 1 of the transponder.

```
bool CTS_UpdateBlock(int Block, const byte* Data);
```

Parameters:

- `int` Block Specify the address of the block to be written. The valid range of this parameter depends on the transponder type.
- `const byte*` Data This parameter holds the data which shall be written to the tag. Note that this function always writes 2 bytes of data, so the minimum array size of Data must be at least 2 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

31 SRX Specific Transponder Operations

TWN4 offers support of the ST SR transponder family like SRI, SRT or SRIX, called here SRX. In order to read data from the memory, the respective transponder must be selected by issuing the function `Search-Tag()`. After successful search, the transponder memory is ready for read and write operations. SRX transponders are organized in blocks, each block contains 4 bytes of data.

31.1 Read Block Data

Use this function in order to read a block of data from the transponder memory.

```
bool SRX_ReadBlock(int Block, byte* Data);
```

Parameters:

<code>int</code> Block	Specify the address of the block to be read. The valid range of this parameter depends on the transponder type.
<code>byte*</code> Data	This parameter holds the data which was read from the tag if the operation was successful. Note that this function always reads 4 bytes of data, so the minimum array size of Data must be at least 4 bytes.

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

31.2 Write Block Data

Use this function to write data to a block of the transponder memory.

```
bool SRX_WriteBlock(int Block, const byte* Data);
```

Parameters:

<code>int</code> Block	Specify the address of the block to be written. The valid range of this parameter depends on the transponder type.
<code>const byte*</code> Data	This parameter holds the data which shall be written to the tag. Note that this function always writes 4 bytes of data, so the minimum array size of Data must be at least 4 bytes.

<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .
----------------	---

32 BLE Functions

The reader TWN4 MultiTech 2 / 3 BLE supports LF / HF transponders and additionally BLE (Bluetooth Low Energy). This allows to connect to all devices with the Bluetooth Standard 4.0 or greater: Android mobile phones with Version 4.3 or greater, iPhones 4S, 5 or greater and PCs with Windows 8.1 / 10 and Bluetooth hardware.

The App in the TWN4 MultiTech 2 / 3 BLE control the BLE module. There are commands for initialization, setting connection parameters, do connection and e.g. reading / writing GATT data fields.

The TWB4 BLE API includes firmware support for several different BLE credential types, including Elatec's Mobile Badge BLE, HID Mobile Access, and PKOC (Public Key Open Credential). For these credentials, the TWN4 App only needs to call the function `BLEInit` with the appropriate parameter, and all further interaction between the reader and the remote device will be handled by the firmware.

For BLE credentials and use cases that are not supported directly by the firmware, the user has the ability to use the functions `BLETxRxModuleCommand` and `BLEGetEvent` to handle all interactions between the reader's BLE module and the remote device. The specifics will depend on the use case, but several example C files are included in the `Apps/Samples/BLE` folder of the DevPack to provide some guidance.

There are two BLE module "types" used in the TWN4 family of readers - the BGM1xx modules soldered to the reader board and listed as "1st Gen" in the datasheet, and the BGM220 available in a SAM card format and available in the "2nd Gen" BLE readers. Because of hardware constraints, these two module types use slightly different versions of the Silicon Labs "BGAPI" communication protocol, and so must be treated differently in TWN4 applications that could target either of the module types.

When designing a TWN4 App to support a custom use case, the user will need to refer to the Silicon Labs BGAPI documentation. For the BGM1xx family of modules, documentation can be found at: [1st Gen](#). The appropriate version of the BGAPI documentation for the BGM220 can be found at: [2nd Gen](#).

In addition to the `BLETxRxModuleCommand` and `BLEGetEvent` functions, there are also several "convenience" functions included in the API. These functions do not accomplish anything that couldn't be accomplished with the two functions mentioned above - they simply allow for more concise TWN4 App code.

The BLE Module on the TWN4 MultiTech BLE communicates serial with the main core. COM2 is reserved for the communication with the BLE Modul and GPIO7 is the reset of the BLE Module. So do not use COM2 and GPIO7 for other things on the hardware TWN4 MultiTech 2 / 3 BLE.

32.1 GATT Server Definition

UUID (hex)	Attr. Handle	Name	R/W	Type	Description
1800		General Access		Service	Service 1
2a00	7	Device name	R/W	Descriptor	xgatt_1800_2A00, fix "TWN4 BLE"
2a01	9	Appearance	R/W	Descriptor	xgatt_1800_2A01, fix 0000
180a		Device Information		Service	Service 2
2a29	12	Manufacturer Name	R	Descriptor	xgatt_180A_2A29, fix "OEM BLE"
2a24	14	Model Number	R	Descriptor	xgatt_180A_2A24, "TWN4 MultiTech"
2a25	16	Serial Number	R/W	Descriptor	xgatt_180A_2A25
2a26	18	Firmware Revision	R	Descriptor	xgatt_180A_2A26, f.e. "BT1.05EL"

Table 32.1: Read-only Bluetooth Standard-defined GATT Services

UUID Name	Attr. Handle	Name	R/W	Type	Description
UID1		Custom		Service	Service 3
UID2	21	SPP Data	Read, Write no resp., Notify, Indic.	Descriptor	xgatt_spp_data var. length=255, HEX
UID3	24	SP1 Data	Write no resp., Notify, Indicate	Descriptor	xgatt_sp1_data var. length=20, HEX
UID4	27	SP2 Data	Read, Notify, Indicate	Descriptor	xgatt_sp2_data var. length=20, HEX
UID5	30	SP3 Data	Read, Write, Notify, Indicate	Descriptor	xgatt_sp3_data var. length=255, HEX

Table 32.2: Custom GATT Services defined within "Gen 1" Firmware

UUID Name	User UUID (hex)	Description
UID1	"5a44c004-4112-4274-880e-cd9b3daedf8e"	SPP Service
UID2	"43c29edf-2f0a-4c43-aa22-489d169ec752"	xgatt_spp_data
UID3	"a897339f-adf0-4a2b-a2ef-4a57512e6eb6"	xgatt_sp1_data
UID4	"71f1ae4d-e1d1-438c-b05d-2c2c16abeaa7"	xgatt_sp2_data
UID5	"495f449c-fc60-4048-b53e-bdb3046d4495"	xgatt_sp3_data

Table 32.3: Definition of "Gen 1" Custom UUIDs

32.2 BLEPresetConfig

This function presets the individual configuration structure for the BLE module. The initialization command BLEInit is necessary after this - optional also the BLEPresetUserData.

```
bool BLEPresetConfig
(
    TBLEConfig* BLEConfig
);
```

Parameters:

TBLEConfig* Reference to the structure that holds the BLE configuration parameters. See the description of TBLEConfig for details.

Return:

If the operation was successful, the return value is true, otherwise it is false.

Members	Length (Bits)	Description
uint32_t ConnectTimeout	32	Timeout of an established connection in milliseconds.
byte Power	8	TX power in 0.1dBm steps in the range 0 to 80 (0.0dBm to 8.0dBm). Please note: It is not allowed to set the value more than 20 (2.0 dBm) for the TWN4 MultiTech 2 BLE desktop housing version. The value above the limit falls under complete integrator/user responsibility.
byte BondableMode	8	Bonding: 0 = Off, 1 = On. If additionally Bit5=1, transparent mode is deactivated. If additionally Bit6=1, discover mode is activated. If additionally Bit7=1, self defined UserData (with function BLEPresetUserData(..)) are used.
uint16_t AdvInterval	16	Advertisement interval: values 20ms to 10240ms
byte ChannelMap	8	Advertisement Bluetooth channels: 1 = CH37, 2 = CH38, 3 = CH37 + CH38, 4 = CH39, 5 = CH37 + CH39, 6 = CH38 + CH39, 7 = CH37 + CH38 + CH39.

byte DiscoverMode	8	Discoverable Modes, which dictate how the device is visible to other devices: 0 = LE_GAP_NON_DISCOVERABLE, 1 = LE_GAP_LIMITED_DISCOVERABLE, 2 = LE_GAP_GENERAL_DISCOVERABLE, 3 = LE_GAP_BROADCAST, 4 = LE_GAP_USER_DATA, Discover Mode to use when scanning for advertising devices: 20 = LE_GAP_DISCOVER_LIMITED, 21 = LE_GAP_DISCOVER_GENERIC, 22 = LE_GAP_DISCOVER_OBSERVATION.
byte ConnectMode	8	Connectable Modes for the LE (Low Energy) GAP (Generic Access Profile): 0 = LE_GAP_NON_CONNECTABLE, 1 = LE_GAP_DIRECTED_CONNECTABLE, 2 = LE_GAP_CONNECTABLE_SCANNABLE, 3 = LE_GAP_SCANNABLE_NON_CONNECTABLE, 4 = LE_GAP_CONNECTABLE_NON_SCANNABLE.
byte SecurityFlags	8	Security requirement bitmask: Bit 0 = 0: Allow bonding without MITM protection, Bit 0 = 1: Bonding requires MITM protection, Bit 1 = 0: Allow encryption without bonding, Bit 1 = 1: Encryption requires bonding, Bit 2 to 7: Reserved, Default value: 0x00.
byte IOCapabilities	8	Security Management related I/O capabilities: 0 = display only, 1 = display yes/no, 2 = keyboard only, 3 = no input / no output, 4 = keyboard / display
uint32_t Passkey	32	Passkey: If security is configured, the application needs to display or ask user to enter a passkey during the bonding process. See: BLE_EVENT_SM_PASSKEY_DISPLAY or BLE_EVENT_SM_PASSKEY_REQUEST.

Table 32.4: Definition of TBLEConfig

32.3 BLEPresetUserData

This function sets the user data string for advertising. F.e. the Apple company has introduced iBeacons to broadcast their identifier to nearby portable electronic devices. If you discover an iBeacon or common a Beacon, you get his UUID, Major and Minor values. With the TWN4 MultiTech 2 BLE, you can configure the reader to be a Beacon.

```
bool BLEPresetUserData
(
```

```

    byte ScanResp,
    const byte* UserData,
    int UserDataLength
);

```

Parameters:**byte** ScanResp

Selection the type showing user data:
 0 = advertise packets,
 1 = scan packets.

const byte* UserData

Reference to the byte buffer that holds the UserData parameters. See the description of UserData for details.

int UserDataLength

Length of the UserData. Maximum data length is 62 Bytes.

Return:

If the operation was successful, the return value is true, otherwise it is false.

Members	Length (Bits)	Value (f.e.)	Description
UserData[0]	8	0x02	Length of the Flags field - 2 bytes.
UserData[1]	8	0x01	Length of the Flags field - high byte.
UserData[2]	8	0x06	Length of the Flags field - low byte.
UserData[3]	8	0x1A	Length of the Manufacturer Data field - 26 bytes.
UserData[4]	8	0xFF	Data type / Manufacturer specific data / Type of the Manufacturer Data field.
UserData[5]	8	0x4C	Manufacturer data - high byte, Company ID field - 0x4C00 = Apple's Bluetooth SIG ID.
UserData[6]	8	0x00	Manufacturer data - low byte.
UserData[7]	8	0x02	Manufacturer data - high byte, Beacon Type field - 0x0215 = iBeacon.
UserData[8]	8	0x15	Manufacturer data - low byte.
UserData[9]	8	0xE2	UUID E2C56DB5-DFFB-48D2-B060-D0F5A71096-E0 (Apple AirLocate Service)
UserData[10]	8	0xC5	
UserData[11]	8	0x6D	
UserData[12]	8	0xB5	
UserData[13]	8	0xDF	
UserData[14]	8	0xFB	
UserData[15]	8	0x48	
UserData[16]	8	0xD2	
UserData[17]	8	0xB0	
UserData[18]	8	0x60	
UserData[19]	8	0xD0	
UserData[20]	8	0xF5	
UserData[21]	8	0xA7	

UserData[22]	8	0x10	
UserData[23]	8	0x96	
UserData[24]	8	0xE0	
UserData[25]	8	0x00	The Major high value, which is used to group related beacons.
UserData[26]	8	0x00	The Major low value.
UserData[27]	8	0x00	The Minor high value, which is used to specify individual beacon within a group.
UserData[28]	8	0x00	The Minor low value.
UserData[29]	8	0xC3	Signal power (calibrated RSSI) - See the iBeacon specification for measurement guidelines.

Table 32.5: Example of UserData

32.4 BLEInit

This function initialize the Bluetooth BLE Module on the reader. Different modes are possible: The custom mode makes individual operating modes possible - pre configured with BLEPresetConfig and BLEPresetUserData. The other modes are predefined modes for advertisement and Beacon.

```
bool BLEInit
(
  int Mode
);
```

Parameters:

int Mode Specify the initialization mode. See the definition of Mode for meaning of each member.

Return: If the operation was successful, the return value is true, otherwise it is false.

BLE_MODE_TRANSPARENT	17	BLE Custom mode for previously defined configuration with functions BLEPresetConfig and BLEPresetUserData, and using the BLETxRxModuleCommand, BLEGetEvent, and BLEStartAdvertisement API functions.
BLE_MODE_BEACON	2	BLE Beacon mode for mobile devices. Use BLEPresetUserData to set advertising data.
BLE_MODE_PACK_ID	4	BLE Beacon mode with connect (Special customer mode).
BLE_MODE_PKOC	18	BLE Mode for reading Public Key Open Credential BLE credentials after parameters are set with PKOC_SetParamsBLE.
BLE_MODE_MOBILE_BADGE_2_0	10	Enable "Mobile Badge 2.0" authentication using Search-Tag(..) function. The function returns Phone ID or other data, sent from the phone application, after established connection. Use for applications with narrow adjustment range.

BLE_MODE_MOBILE_BADGE_2_0_AT2	11	The same initialization as "BLE_MODE_MOBILE_BADGE_2_0". Use for applications with wide adjustment range.
BLE_MODE_OFF	255	BLE no longer active.

Table 32.6: Definition of Mode

32.5 BLETxRxModuleCommand

This function is used to interact directly with the BLE module - specifically to send BGAPI commands to the module and to retrieve the module's response to those commands.

```
bool BLETxRxModuleCommand
(
    int command,
    byte* TXData,
    int TXDataBytes,
    byte* RXData,
    int* RXDataBytes,
    int MaxRXDataBytes
);
```

Parameters:

<code>int command</code>	The formatted BGAPI command consisting of the byte 0x20 to indicate a command, the BGAPI message class byte, and the individual BGAPI command byte. Examples can be found in <code>twn4.sys.h</code> with names such as <code>CMD_SYSTEM_HELLO</code> . BGAPI commands not already included in <code>twn4.sys.h</code> can be constructed following the same pattern.
<code>byte* TXData</code>	The data that will be sent along with the BGAPI command, as required by the individual command.
<code>int TXDataBytes</code>	The number of bytes of data to be sent from TXData.
<code>byte* RXData</code>	Pointer to an array that will contain the response from the module to the sent command. Note that most BGAPI commands receive an instant response - usually <code>SL_STATUS_OK</code> (0x0000) - but that the completion of the action requested by the command is often indicated via related Events. This function does not wait for any such events. Rather, your application code must check for them using <code>BLECheckEvent</code> or <code>BLEGetEvent</code>
<code>int* RXDataBytes</code>	The number of bytes of response data received.
<code>int MaxRXDataBytes</code>	The size in bytes of the RXData array.

Return:

If the operation was successful, meaning the reader received a reply to the command within a 100 ms timeout, the return value is `true`, otherwise it is `false`.

32.6 BLEGetEvent

This function returns the event received from the BLE module. In addition, it allows retrieval of the Event Data received from the BLE module, for events where that additional data is required.

```
int BLEGetEvent
(
    byte* eventData,
    int maxDataLength,
    int* eventDataBytes
);
```

Parameters:

byte* eventData Pointer to an array to hold the data included in the received event.

int MaxDataLength The size in bytes of the eventData array.

int* eventDataBytes The number of bytes of event data received.

Return: Specify the event. See an incomplete list of definitions in the table below BLECheckEvent. All event definitions can be found in the appropriate BGAPI documentation.

32.7 BLECheckEvent

This function returns the event received from the BLE module. No additional data is returned, to be used when no additional event data is expected or needed.

```
int BLECheckEvent
(
    void
);
```

Parameters: None.

Return: Specify the event. See an incomplete list of definitions in the table below

BLE_EVENT_NONE	0x00	No event.
BLE_EVENT_ENDPOINT_CLOSING	0x21	This event indicates that an endpoint is closing or that the remote has terminated the connection.
BLE_EVENT_ENDPOINT_DATA	0x22	This event indicates incoming data from an endpoint.
BLE_EVENT_ENDPOINT_STATUS	0x23	This event indicates an endpoint's status.
BLE_EVENT_ENDPOINT_SYNTAX_ERROR	0x24	This event indicates that a protocol error to the BLE module was detected.
BLE_EVENT_GATT_MTU_EXCHANGED	0x45	This event indicates that a GATT MTU exchange procedure has been completed.
BLE_EVENT_GATT_SERVER_ATTRIBUTE_VALUE	0x51	This event indicates that the value of an attribute in the local GATT database has been changed by a remote GATT client.

BLE_EVENT_GATT_SERVER_CHARACTERISTIC_STATUS	0x52	This event indicates either that a local Client Characteristic Configuration descriptor has been changed by the remote GATT client, or that a confirmation from the remote GATT client was received upon a successful reception of the indication.
BLE_EVENT_GATT_CHARACTERISTIC	0x53	This event indicates that a GATT characteristic in the remote GATT database was discovered.
BLE_EVENT_GATT_CHARACTERISTIC_VALUE	0x54	This event indicates that the value of a characteristic in the remote GATT server was received.
BLE_EVENT_GATT_SERVICE	0x55	This event indicates that a GATT service in the remote GATT database was discovered.
BLE_EVENT_GATT_PROCEDURE_COMPLETED	0x56	This event indicates that the current GATT procedure has been completed successfully.
BLE_EVENT_CONNECTION_CLOSED	0x71	This event indicates that a connection was closed.
BLE_EVENT_CONNECTION_OPENED	0x72	This event indicates that a new connection was opened, whether the devices are already bonded, and what is the role of the Bluetooth device (Slave or Master). An open connection can be closed with the command BLERequestEndpointClose.
BLE_EVENT_CONNECTION_PARAMETERS	0x73	This event is triggered whenever the connection parameters are changed and at any time a connection is established.
BLE_EVENT_CONNECTION_RSSI	0x74	This event is triggered when an BLERequestRssi command has completed.
BLE_EVENT_LE_GAP_SCAN_RESPONSE	0x81	This event reports any advertisement packet that is received by the device's radio while in scanning mode.
BLE_EVENT_LE_GAP_EXTENDED_SCAN_RESPONSE	0x82	This event reports any advertisement or scan response packet that is received by the device's radio while in scanning mode.
BLE_EVENT_SM_BONDED	0x91	This event is triggered after the pairing or bonding procedure has been successfully completed.
BLE_EVENT_SM_BONDING_FAILED	0x92	This event is triggered if the pairing or bonding procedure has failed.
BLE_EVENT_SM_LIST_ALL_BONDINGS_COMPLETE	0x93	This event is triggered by the BLE_EVENT_SM_LIST_BONDING_ENTRY.

BLE_EVENT_SM_LIST_BONDING_ENTRY	0x94	This event is triggered if bondings exist in the local database.
BLE_EVENT_SM_PASSKEY_DISPLAY	0x95	This event indicates a request to display the passkey to the user.
BLE_EVENT_SM_PASSKEY_REQUEST	0x96	This event indicates a request for the user to enter the passkey displayed on the remote device.
BLE_EVENT_SM_CONFIRM_BONDING	0x99	This event indicates a request to display that new bonding request has been received to the user and for the user to confirm the request.
BLE_EVENT_SM_CONFIRM_PASSKEY	0x9A	This event indicates a request to display the passkey to the user and for the user to confirm the displayed passkey.
BLE_EVENT_SYSTEM_BOOT	0xA2	This event indicates the device has started and the radio is ready. This even carries the firmware build number and other SW and HW identification codes saved in the function BLEGetVersion.
BLE_EVENT_DTM_COMPLETED	0xF0	This event indicates that the RF section has processed a test start or end command.

Table 32.7: Definition of Event

32.8 BLEStartAdvertisement

This is a convenience function that allows the user to begin advertising when the module has been initialized via BLE_MODE_TRANSPARENT, and when advertising data has been set using BLEPresetUserData.

```
bool BLEStartAdvertisement
(
    uint16_t AdvertiseInterval,
    byte ConnectableMode
);
```

Parameters:

uint16_t AdvertiseInterval The advertising interval for the module to use. Default value is 100 ms.

byte ConnectableMode The advertising mode to use. Possible values are:

- 0 = Non-Connectable, Non-Scannable
- 1 = Directed Connectable
- 2 = Connectable, Scannable
- 3 = Non-Connectable, Scannable
- 4 = Connectable, Non-Scannable

Return: If the operation was successful, the return value is true, otherwise it is false.

32.9 BLEGetAddress

This function returns the device address from the BLE module, the remote address from the connected device and the address type of the remote address.

```
bool BLEGetAddress
(
    byte* DeviceAddress,
    byte* RemoteAddress,
    byte* Type
);
```

Parameters:

<code>byte* DeviceAddress</code>	The device address of the BLE module in 6 bytes hex is returned by this parameter.
<code>byte* RemoteAddress</code>	The remote address of the connected device in 6 bytes hex is returned, if the remote device is successfull connected. For additional information of the remote address see the Type parameter.
<code>byte* Type</code>	The type of the remote address is returned by this parameter. Possible values are: 0 = public address, 1 = random address, 2 = public identity address resolved by stack, 3 = random identity address resolved by stack, 4 = Classic Bluetooth address.

Return: If the operation was successful, the return value is true, otherwise it is false.

32.10 BLEGetVersion

This function returns on the one hand the version string of the BLE module firmware in ASCII format and on the other the boot string of the BLE hardware. Requirement: BLEInit() with any parameter is called first.

```
bool BLEGetVersion
(
    byte* HWVersion,
    byte* BootString
);
```

Parameters:

<code>byte*</code> HWVersion	The firmware version string (16 bytes) in ASCII code is returned by this parameter. Example: "V1.03,14.11.2016"
<code>byte*</code> BootString	The boot string of the BLE hardware is returned. The information is binary coded in 12 bytes with the following information: Byte 0 : Major release version, Byte 1 : HW-Device (0 = any BGM device, 1 = BGM111, 2 = BGM121, 3 = BGM11S, 4 = BGM113), For this selection the BLE Chipversion V1.06 or higher and Dev-Pack 3.22 or higher is needed! Byte 2 - 3: Minor release version, Byte 4 - 5: Patch release number, Byte 6 - 7: Build number, Byte 8 - 9: Bootloader version, Byte 10 - 11: Hardware type.

Return: If the operation was successful, the return value is `true`, otherwise it is returned `false`.

32.11 BLEGetEnvironment

This function can be used to ask for connection environment on a connected device.

```
bool BLEGetEnvironment
(
    byte* DeviceRole,
    byte* SecurityMode,
    byte* Rssi
);
```

Parameters:

<code>byte*</code> DeviceRole	The device role of the connection is returned: 0 = Slave, 1 = Master.
<code>byte*</code> SecurityMode	The security mode of the established connection is returned. Possible values are: 0 = No security. 1 = Unauthenticated pairing with encryption. 2 = Authenticated pairing with encryption. 3 = Authenticated Secure Connections pairing with encryption using a 128-bit strength encryption key.
<code>byte*</code> Rssi	RSSI (field strength) of the BLE connection Range: -127 to +20. Units: dBm.

Return: If the operation was successful, the return value is `true`, otherwise it is returned `false`.

32.12 BLEGetGattServerAttributeValue

This function returns the data of a GATT attribute handle. If the event "BLE_EVENT_GATT_SERVER_ATTRIBUTE_VALUE" is called, call the function with the AttrHandle and Bit 15 set to 1.

```
bool BLEGetGattServerAttributeValue
(
    int AttrHandle,
    byte *Data,
    int *Len,
    int MaxLen
);
```

Parameters:

<code>int AttrHandle</code>	Specify the attribute handle of the GATT who is selected to read. Additional set Bit15 to 1 if the event "BLE_EVENT_GATT_SERVER_ATTRIBUTE_VALUE" is called.
<code>byte *Data</code>	The read data of the given attribute handle is returned by this parameter.
<code>int *Len</code>	This parameter holds the length of data which was read from the GATT.
<code>int MaxLen</code>	Maximum number of characters, the specified byte array can receive excluding the 0-termination.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.13 BLESetGattServerAttributeValue

This function writes data to an attribute handle. Notice that the GATT attribute must be writeable.

```
bool BLESetGattServerAttributeValue
(
    int AttrHandle,
    int Offset,
    const byte *Data,
    int Len
);
```

Parameters:

<code>int AttrHandle</code>	Specify the attribute handle of the GATT for writing data. If Bit 15 of the AttrHandle is set, the GATT field is written and a notification is sent to the connected slave.
<code>int Offset</code>	Specify the starting address for writing to data. The valid range of this parameter is 0 to Len-1.
<code>byte *Data</code>	The write data buffer to the attribute handle with the specified offset.
<code>int Len</code>	This parameter holds the length of data which shall be written to the GATT.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.14 BLERequestRssi

This function calls a request for the latest RSSI of a Bluetooth connection. The value of the RSSI is returned by an event BLE_EVENT_CONNECTION_RSSI with function BLECheckEvent. The function makes only sense if there is a established connection with a remote device. If the module is in discovery mode, the function BLECheckEvent returns the current RSSI of the scanned device.

```
bool BLERequestRssi
(
    void
);
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.15 BLERequestEndpointClose

This function closes a connection with the remote device. If the connection is closed, the function BLECheckEvent returns the event BLE_EVENT_CONNECTION_CLOSED for successfull closing.

```
bool BLERequestEndpointClose
(
    void
);
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.16 BLEGetGattServerCharacteristicStatus

This function can be used after the event BLE_EVENT_GATT_SERVER_CHARACTERISTIC_STATUS to ask for GATT field characteristic change through a client. With the same function ask after the event BLE_EVENT_GATT_SERVER_ATTRIBUTE_VALUE for the GATT AttrHandle number.

```
bool BLEGetGattServerCharacteristicStatus
(
    int *AttrHandle,
    int *AttrStatusFlag,
    int *AttrConfigFlag
);
```

Parameters:

<code>int *AttrHandle</code>	GATT characteristic handle.
<code>int *AttrStatusFlag</code>	Describes whether Client Characteristic Configuration was changed or if confirmation was received: 1 = Characteristic client configuration has been changed. 2 = Characteristic confirmation has been received.
<code>int *AttrConfigFlag</code>	This field carries the new value of the Client Characteristic Configuration: 0 = Disable notifications and indications. 1 = Notification. 2 = Indication.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.17 BLEFindGattServerAttribute

This command can be used to find attributes of certain type from a local GATT database. Type is usually given as 16-bit (2 byte) or 128-bit (16 byte) UUID.

```
bool BLEFindGattServerAttribute
(
    const byte *UUID,
    int UUIDLength,
    int *AttrHandle
);
```

Parameters:

<code>const byte *UUID</code>	16-bit or 128-bit UUID of the local GATT database.
<code>int UUIDLength</code>	Length of the UUID in bytes (e.g. 2 or 16).
<code>int *AttrHandle</code>	The GATT characteristic attribute handle is returned by this parameter.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

32.18 BLEDiscover

This function starts with discovering devices, services and characteristics - depending on the Discover-Mode parameter. Unnecessary parameters of the function are assigned zero.

```
bool BLEDiscover
(
    int DiscoverMode,
    unsigned long GattHandle,
    const TBLEUUID *BLEUUID
);
```

Parameters:

`int DiscoverMode` Mode parameter to discover devices, services and characteristics. See the DiscoverMode table below.

`unsigned long GattHandle` Depending on parameter DiscoverMode the handle of a GATT field.

`const TBLEUUID *BLEUUID` Depending on parameter DiscoverMode the UUID of a GATT field.

Return: If the operation was successful, the return value is true, otherwise it is returned false.

Members	Length (Bytes)	Description
byte UUIDLength	1	Length of a UUID.
byte UUID[16]	16	UUID of a GATT field.

Table 32.8: Definition of TBLEUUID

BLE_DISC_START_PHY_1M	0	Start discovering with PHY "1M". No special parameter contents are required: Set these parameters (GattHandle and BLEUUID) to zero. If a device is found, the event "BLE_EVENT_LE_GAP_SCAN_RESPONSE" is generated. This event can be parsed with BLECheckDiscovered-String(). BLEGetAddress() gives the remote address and type of the device found.
BLE_DISC_STOP_PHY_1M	1	Stop discovering regarding to PHY "1M". No special parameter contents are required: Set these parameters (GattHandle and BLEUUID) to zero.
BLE_DISC_SERVICE	10	Start discovering all services on a connected device. No special parameter contents are required: Set these parameters (GattHandle and BLEUUID) to zero. The generated events for discovered services are "BLE_EVENT_GATT_SERVICE" and at the end "BLE_EVENT_GATT_PROCEDURE_COMPLETED". The service event data are displayed using BLE-GattGetAttribute(..) function.
BLE_DISC_SERVICE_WITH_UUID	11	Starts to discover a service with a well-known UUID on a connected device. Parameter BLEUUID is needed and GattHandle not (set it to zero). The generated event "BLE_EVENT_GATT_PROCEDURE_COMPLETED" indicates the success of the found UUID.

BLE_DISC_CHARAC	20	Start discovering all characteristics with service handle on a connected device. The service handle is needed and the BLEUUID not. Set it to zero. The generated events for discovered characteristic are "BLE_EVENT_GATT_CHARACTERISTIC" and at the end "BLE_EVENT_GATT_PROCEDURE_COMPLETED". The characteristic event data are displayed using BLEGattGetAttribute(..) function.
BLE_DISC_CHARAC_WITH_UUID	21	Starts to discover a characteristic with well-known service handle and characteristic UUID on a connected device. The generated event "BLE_EVENT_GATT_PROCEDURE_COMPLETED" indicates the success.
BLE_SET_CHARAC_DISABLE_NOTIF_INDIC	30	This command can be used to disable the notifications and indications being sent from a remote GATT server.
BLE_SET_CHARAC_NOTIFICATION	31	This command can be used to enable notifications being sent from a remote GATT server. The generated event is BLE_EVENT_GATT_CHARACTERISTIC_VALUE".
BLE_SET_CHARAC_INDICATION	32	This command can be used to enable indications being sent from a remote GATT server. The generated event is BLE_EVENT_GATT_CHARACTERISTIC_VALUE". The needed confirmation for the received event is automatically send.

Table 32.9: Definition of DiscoverMode

32.19 BLECheckDiscoveredString

In discover mode, the event "BLE_EVENT_LE_GAP_SCAN_RESPONSE" or "BLE_EVENT_LE_GAP_EXTENDED_SCAN_RESPONSE" indicates the response of discovered devices. Each time the event occurs, it is recommended to parse the discovered string with this function.

```
bool BLECheckDiscoveredString
(
    int CheckMode,
    const byte* CompareString,
    int Length
);
```

Parameters:

`int CheckMode` Mode parameter for different comparisons. See the CheckMode table below.

`const byte *CompareString` Compare string for different CheckModes.

`int Length` Length of the CompareString in bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

BLE_CHECK_NAME	0	Compares the discovered device name with the passed parameter CompareString (LOCAL NAME, TYPE 0x08 and 0x09).
BLE_CHECK_UUID	3	Compares the discovered UUID with the passed parameter CompareString (SERVICE UUID, TYPE 0x02 to 0x07 (16bit, 32 bit, 128 bit UUID)).
BLE_CHECK_TX_POWER_LEVEL	4	Compares the discovered power level with the passed parameter CompareString (TX POWER LEVEL, TYPE 0x0A).
BLE_CHECK_SIMPLE_PAIRING	5	Compares the discovered simple pairing with the passed parameter CompareString (SIMPLE PAIRING, TYPE 0x0D to 0x0F).
BLE_CHECK_SERVICE_DATA	6	Compares the discovered service data with the passed parameter CompareString (SERVICE DATA, TYPE 0x16).
BLE_CHECK_MANUFACTURER_DATA	7	Compares the discovered manufacturer string with the passed parameter CompareString (MANUFACTURER DATA, TYPE 0xFF).

Table 32.10: Definition of CheckMode

32.20 BLEConnectToDevice

This function connects a device with Address (MAC) and AdressType. The Address and Type can be requested with the function BLEGetAddress(..) described above.

```
bool BLEConnectToDevice
(
    const byte *Address,
    byte AddressType
);
```

Parameters:

`const byte *Address` The address of a device to connect to with 6 bytes Hex.

`byte AddressType` The type of the address. Possible values are:
 0 = public address,
 1 = random address,
 4 = Classic Bluetooth address.

Return: If the operation was successful, the return value is true, otherwise it is false.

32.21 BLEDisconnectFromDevice

This function disconnects from a connected device.

```
bool BLEDisconnectFromDevice
(
    void
)
```

Parameters: None.

Return: If the operation was successful, the return value is true, otherwise it is false.

32.22 BLEGattGetAttribute

This function reads GATT attributes at specific event:

The relevant data of "BLE_EVENT_GATT_SERVICE" or "BLE_EVENT_GATT_CHARACTERISTIC" event passes to this function, when discovering services or characteristics is started.

```
bool BLEGattGetAttribute
(
    TBLEUUID *BLEUUID,
    unsigned long *GattHandle
)
```

Parameters:

`TBLEUUID *BLEUUID` Returns the UUID of the discovered field.

`byte *Handle` Returns the handle of the GATT field that was found.

Return: If the operation was successful, the return value is true, otherwise it is false.

Members	Length (Bytes)	Description
byte UUIDLength	1	Length of the UUID.
byte UUID[16]	16	UUID of the GATT field.

Table 32.11: Definition of TBLEUUID

32.23 BLEGattGetValue

This function returns the read data of a GATT field.

```
bool BLEGattGetValue
(
    int ReadMode,
    unsigned long GattHandle,
    const TBLEUUID *BLEUUID,
    byte *AttrOpcode,
    byte *Data,
    int *Len,
    int MaxLen
)
```

Parameters:

<code>int ReadMode</code>	Mode parameter for different read modes. See table ReadMode below.
<code>unsigned long GattHandle</code>	Depending on the read mode, specify a characteristic or service handle of the GATT field.
<code>const TBLEUUID *BLEUUID</code>	Depending on the read mode, specify the UUID of the GATT field.
<code>byte *AttrOpcode</code>	Returns the Attribute Opcode of the read GATT field. See table AttrOpcode below.
<code>byte *Data</code>	The read data is returned by this parameter.
<code>int *Len</code>	This parameter contains the length of data from the read GATT field. If the length is 0, the Data are not valid.
<code>int MaxLen</code>	The max length of the read data.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

BLE_READ_CHARAC	0	Read a GATT field with a specific characteristic handle.
BLE_READ_SERVICE_WITH_UUID	1	Read a GATT field with a specific service handle and characteristic UUID.
BLE_READ_CHARAC_WITH_OFFSET	2	Read a GATT field with a specific characteristic handle. The offset is fix 0. Longer data is merged.

BLE_READ_CHARAC_WITH_VALUE_EVENT	3	Read a GATT field with a specific characteristic handle. Requirement: The requested characteristic has to support Notify attribute. The activation of the notification has to be performed prior execution of this function. This is accomplished with the function "BLEDiscover(BLE_SET_CHARAC_NOTIFICATION, ...)". The activation is possible only after a successful connection with the server. Note: When the characteristic is changed from the GATT server side (written with notification), the event "BLE_EVENT_GATT_CHARACTERISTIC_VALUE" is generated on the GATT client side. The event data can be checked with parameter "AttrOpcode". This value should be the GattHandle of the Data.
----------------------------------	---	---

Table 32.12: Definition of ReadMode

gatt_read_by_type_request	8	Read by type request
gatt_read_by_type_response	9	Read by type response
gatt_read_request	10	Read request
gatt_read_response	11	Read response
gatt_read_blob_request	12	Read blob request
gatt_read_blob_response	13	Read blob response
gatt_read_multiple_request	14	Read multiple request
gatt_read_multiple_response	15	Read multiple response
gatt_write_request	18	Write request
gatt_write_response	19	Write response
gatt_write_command	82	Write command
gatt_prepare_write_request	22	Prepare write request
gatt_prepare_write_response	23	Prepare write response
gatt_execute_write_request	24	Execute write request
gatt_execute_write_response	25	Execute write response
gatt_handle_value_notification	27	Notification
gatt_handle_value_indication	29	Indication

Table 32.13: Values of AttrOpcode

32.24 BLEGattSetValue

This function writes data to a characteristic handle.

```
bool BLEGattSetValue
(
    int WriteMode,
    unsigned long GattHandle,
    int Offset,
    const byte *Data,
    int DataLength
)
```

Parameters:

int WriteMode Mode parameter for different write modes. See table WriteMode below.

unsigned long GattHandle Specify the characteristic handle of the GATT for writing data.

int Offset Specify the starting address for writing data. The valid range of this parameter is 0 to Len-1.

const byte *Data The passed write data buffer.

int DataLength This parameter holds the length of data which shall be written to the GATT.

Return: If the operation was successful, the return value is true, otherwise it is false.

BLE_WRITE_CHARAC	0x00	Write data to a GATT field with a specific characteristic handle.
BLE_WRITE_CHARAC_WITHOUT_RESP	0x01	Write GATT field with a specific characteristic handle without response. This command does not generate any event. All failures on the server are ignored silently. For example, if an error is generated in the remote GATT server and the given value is not written into database no error message will be reported to the local GATT client. Note that this command cannot be used to write long data!

Table 32.14: Definition of WriteMode

32.25 BLECommand

This command can be used to set and request an connection.

```
int BLECommand
(
    int CommandCode,
    int Parameter
)
```

Parameters:

`int CommandCode` Mode parameter for a connection. See table CommandCode below.

`int Parameter` For more information, see the CommandCode table.

Return: The return value is specified in the CommandCode table.

BLE_CMD_SET_GATT_MTU	0	This command can be used to set the maximum size of ATT Message Transfer Units (MTU). If the given value is too large, the system will select the maximal possible value as the maximum ATT MTU. If maximum ATT MTU is larger than 23, MTU is exchanged automatically after a Bluetooth connection has been established. -> Parameter Flag: New MTU value. Range 23 to 250
BLE_CMD_REQUEST_RSSI	1	This command is used to verify the working communication between host and device. After this request has been issued, the RSSI value can be called up with BLEGetEnvironment(..). -> Parameter Flag: Not used.
BLE_CONN_STREAM_AVAILABLE	2	Communication in Streaming Mode: GATT Server Mode: Function return <code>true</code> if the GATT Server on the reader is successfully connected, otherwise it is <code>false</code> . GATT Client Mode: Flag = 0: Function return <code>true</code> if a connection is established, otherwise it is <code>false</code> . Flag = 1: Function return <code>true</code> if a connection / discovering is established and notification is set, otherwise it is <code>false</code> .

BLE_CMD_REQUEST_DTM_TEST_END	3	This command can be used to end a transmitter or a receiver test. When the command is processed by the radio and the test has ended, a BLE_EVENT_DTM_COMPLETED event is triggered. -> Parameter Flag: Not used.
BLE_CMD_REQUEST_DISABLE_CONNECTION	4	This command can be used to stop a established streaming connection. -> Parameter Flag: Not used.
BLE_CMD_ENABLE_MOBILE_BADGE_2_0	5	Enable "Mobile Badge 2.0" authentication using SearchTag(..) function. -> Parameter Flag = 0: Read BLE State machine (IDLE mode). If the operation was successful, the return value is true, otherwise it is false. Flag = 1: Set the ready state to exchange data (discover mode is established). If the operation was successful, the return value is true, otherwise it is false. Flag = 2: Read the Phone ID status. If the Phone ID is received, the return value is true, otherwise it is false.
BLE_CMD_REQUEST_RESULTCODE	6	This command requests the result code of the last command sent to the BLE modul. The complete errorlist can be requested by the support department. -> Parameter Flag: Not used.
BLE_CMD_SYSTEM_SET_TX_POWER	6	This command can be used to set the global maximum TX power for Bluetooth module on the reader. The possible range is from 0 to the maximum value of 80 (8.0 dBm). Please note: It is not allowed to set the value more than 20 (2.0 dBm) for the TWN4 MultiTech 2 BLE desktop housing version. The value above the limit falls under complete integrator/user responsibility. -> Parameter Flag: The TX Power in 0.1 dBm steps. Example: The value 10 is representing 1 dBm and the value 55 is representing 5.5 dBm.

Table 32.15: Definition of CommandCode

32.26 BLESecurity

This function handles the security of a BLE connection.

Note: The default passkey is "123456" (set with every predefined BLEInit(Mode). Call f.e. function BLESecurity(BLE_SM_SET_PASSKEY,123456,0) for own values.

```
bool BLESecurity
(
    int SMMode,
    unsigned long Flag1,
    unsigned long Flag2
)
```

Parameters:

<code>int SMMode</code>	Mode parameter for different security modes. See table SMMode below.
<code>unsigned long Flag1</code>	For more information, see the SMMode table.
<code>unsigned long Flag2</code>	For more information, see the SMMode table.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

BLE_SM_BONDING_CONFIRM	0	This mode can be used for accepting or rejecting bonding request. -> Parameter Flag1: 0 = Reject, 1 = Accept bonding request. -> Parameter Flag2: Not used.
------------------------	---	---

BLE_SM_CONFIGURE	1	This mode can be used to configure security requirements and I/O capabilities of the system. -> Parameter Flag1: Security requirement bitmask. Bit 0: 0: Allow bonding without MITM protection 1: Bonding requires MITM protection Bit 1: 0: Allow encryption without bonding 1: Encryption requires bonding. Note that this setting will also enable bonding. Bit 2: 0: Allow bonding with legacy pairing 1: Secure connections only Bit 3: 0: Bonding request does not need to be confirmed 1: Bonding requests need to be confirmed. Received bonding requests are notified with BLE_EVENT_SM_CONFIRM_BONDING. Bit 4 to 7: Reserved. Default value: 0x00 -> Parameter Flag2: I/O Capabilities 0 = Display Only 1 = Display with Yes/No-buttons 2 = Keyboard Only 3 = No Input and No Output 4 = Display with Keyboard
BLE_SM_DELETE_BONDINGS	2	This mode can be used to delete all bonding information from Persistent Store. -> Parameter Flag1: Not used. -> Parameter Flag2: Not used.

BLE_SM_STORE_BONDING_CONFIGURATION	3	This mode can be used to set maximum allowed bonding count and bonding policy. The default value is Flag1=1 for maximum number of bondings supported and Flag2=1 for overwriting the oldest existing bonding. -> Parameter Flag1: Maximum allowed bonding count. Range: 1 to 14 -> Parameter Flag2: Bonding policy. Values: 0: If database is full, new bonding attempts will fail 1: New bonding will overwrite the oldest existing bonding 2: New bonding will overwrite longest time ago used existing bonding
BLE_SM_SET_BONDABLE_MODE	4	This mode can be used to set whether the device accepts new bondings or not. -> Parameter Flag1: Bondable mode. Values: 0: New bondings not accepted 1: Bondings allowed -> Parameter Flag2: Not used.
BLE_SM_LIST_ALL_BONDINGS	5	This mode can be used to list all bondings stored in the bonding database. Bondings are reported by using the BLE_EVENT_SM_LIST_BONDING_ENTRY event for each bonding entry. BLEGetAddress() shows at each of this event the MAC-Address in parameter RemoteAddress. The report is ended with BLE_EVENT_SM_LIST_ALL_BONDINGS_COMPLETE event. Recommended to be used only for debugging purposes, because reading from the Persistent Store is relatively slow. -> Parameter Flag1: Not used. -> Parameter Flag2: Not used.
BLE_SM_ENTER_PASSKEY	10	This mode can be used to enter a passkey after receiving a passkey request event. -> Parameter Flag1: Passkey. Valid range: 0-999999. Set -1 to cancel pairing. -> Parameter Flag2: Not used.

BLE_SM_SET_PASSKEY	11	This mode can be used to enter a fixed passkey. -> Parameter Flag1: Passkey. Valid range: 0-999999. Set -1 to disable and start using random passkeys. -> Parameter Flag2: Not used.
BLE_SM_PASSKEY_CONFIRM	12	This mode can be used for accepting or rejecting reported confirm value. -> Parameter Flag1: Accept confirm value. Values: 0: Reject 1: Accept confirm value -> Parameter Flag2: Not used.
BLE_SM_INCREASE_SECURITY	20	This mode can be used to enhance the security of a connection to current security requirements. On an unencrypted connection, this will encrypt the connection and will also perform bonding if requested by both devices. On an encrypted connection, this will cause the connection re-encrypted. -> Parameter Flag1: Not used. -> Parameter Flag2: Not used.

Table 32.16: Definition of SMMode

32.27 BLESecuritySetOob

This function set the OOB data (out-of-band encryption data).

```
bool BLESecuritySetOob
(
    int SMOOBMode,
    const byte *OobData,
    int DataLength
)
```

Parameters:

<code>int SMOOBMode</code>	Mode parameter for different OOB modes. See table SMOOBMode below.
<code>const byte *OobData</code>	OOB data to be written. For more information, see the SMOOBMode table.
<code>int DataLength</code>	This parameter holds the length of OobData which shall be written. For more information, see the SMOOBMode table.

Return:

If the operation was successful, the return value is true, otherwise it is false.

BLE_SM_SET_OOB_DATA	0	This parameter can be used to set the OOB data (out-of-band encryption data) for legacy pairing for a device. The OOB data may be, for example, a PIN code exchanged over an alternate path like NFC. The device will not allow any other kind of bonding if OOB data is set. The OOB data cannot be set simultaneously with secure connections OOB data. To set OOB data, send a 16-byte array. To clear OOB data, send a zero-length array. The data length is 16 byte.
BLE_SM_SET_SC_REMOTE_OOB_DATA	1	This parameter can be used to set OOB data and confirm values (out-of-band encryption) received from the remote device for secure connections pairing. OOB data must be enabled with BLESecurityUseScOob(...) before setting the remote device OOB data. To set OOB data, send a 32-byte array. The first 16-bytes is the OOB data and the last 16-bytes the confirm value. To clear OOB data, send a zero-length array. The data length is 32 byte.
BLE_SET_MOBILE_BADGE_MASTERKEY	10	Special case: This parameter can be used to set the 16 bytes MasterKey for "Mobile Badge 2.0" authentication.

Table 32.17: Definition of SM00BMode

32.28 BLESecurityUseScOob

This command can be used to enable the use of OOB data (out-of-band encryption data) for a device for secure connections pairing. The enabling will generate new OOB data and confirm values which can be sent to the remote device. After enabling the secure connections OOB data, the remote devices OOB data can be set with BLESecuritySetOob(BLE_SM_SET_SC_REMOTE_OOB_DATA, ...). Calling this function will erase any set remote device OOB data and confirm values. The device will not allow any other kind of bonding if OOB data is set. The secure connections OOB data cannot be enabled simultaneously with legacy pairing OOB data.

```
bool BLESecurityUseScOob
(
    int Enable,
    byte *OobData,
    int *Length,
    int MaxLength
)
```


Parameters:

<code>int Enable</code>	Enable OOB with secure connections pairing. Values: 0: disable 1: enable
<code>byte *OobData</code>	Returned OOB data. 32-byte array. The first 16-bytes contain randomly generated OOB data and the last 16-bytes confirm value.
<code>int *Length</code>	This parameter contains the length of OOB data.
<code>int MaxLength</code>	The max length of OOB data.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

32.29 BLESetStreamingMode

This function initialize the streaming mode for byte or block data transfer. The data are read from and written to CHANNEL_BLE.

```
bool BLESetStreamingMode
(
    int ConnMode,
    int GattMode,
    int TransferMode
)
```

Parameters:

<code>int ConnMode</code>	Mode parameter for different connection modes. See table ConnMode below.
<code>int GattMode</code>	Mode parameter for different GATT modes. See table GattMode below.
<code>int TransferMode</code>	Mode parameter for different transfer modes. See table TransferMode below.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

BLE_STREAM_CONN_NONE	0	Streaming Mode is switched off. The "Serial Event Handler" in the firmware is deactivated.
BLE_STREAM_CONN_ADVERTISE	1	The reader is in Advertise mode and Data Streaming is activated.
BLE_STREAM_CONN_DISCOVER	2	The reader is in Discover mode and Data Streaming is activated.

Table 32.18: Definition of ConnMode

BLE_STREAM_GATT_SERVER	0	Streaming Mode is activated with GATT Server on the reader.
------------------------	---	---

BLE_STREAM_GATT_CLIENT	1	Streaming Mode is activated with GATT Client on the reader.
------------------------	---	---

Table 32.19: Definition of GattMode

BLE_STREAM_TRANSFER_BLOCKWISE	0	Streaming Mode is activated with transferring blocks (over CHANNEL_BLE). The max. block size per Write-Bytes(CHANNEL_BLE,...) is 250 bytes.
BLE_STREAM_TRANSFER_STREAMWISE	1	Streaming Mode with activated transferring as byte stream.

Table 32.20: Definition of TransferMode

32.30 BLESetStreamingUUID

This function set the UUIDs for the data streaming (see function BLESetStreamingMode(..)).

```
bool BLESetStreamingUUID(
    const byte *ServiceUUID,
    int ServiceUUIDLength,
    const byte *CharacUUID,
    int CharacUUIDLength
)
```

Parameters:

const byte *ServiceUUID 16-bit or 128-bit Service UUID of the GATT (Client or Server) for Streaming.

int ServiceUUIDLength Length of the Service UUID in bytes (e.g. 2 or 16 bytes).

const byte *CharacUUID 16-bit or 128-bit Characteristic UUID of the GATT (Client or Server) for Streaming.

int CharacUUIDLength Length of the Characteristic UUID in bytes (e.g. 2 or 16 bytes).

Return: If the operation was successful, the return value is true, otherwise it is false.

```
bool BLEGetDiscoveredData(byte *Data, int *Len, int MaxLen)
```

32.31 BLEGetDiscoveredData

This function return the complete advertiser data at event "BLE_EVENT_LE_GAP_SCAN_RESPONSE" or "BLE_EVENT_LE_GAP_EXTENDED_SCAN_RESPONSE".

```
bool BLEGetDiscoveredData(  
    byte *Data,  
    int *Len,  
    int MaxLen  
)
```

Parameters:

byte *Data

The read advertiser data is returned by this parameter.

int *Len

This parameter contains the length of the read advertiser data.

int MaxLen

The max length of the data.

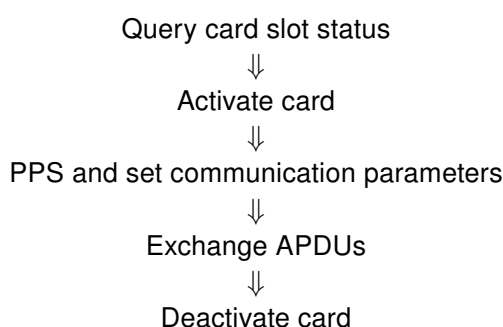
Return:

If the operation was successful, the return value is true, otherwise it is false.

33 Contact Card Operations

33.1 Microprocessor Cards

This chapter handles the usage of ISO7816 compliant Integrated Circuit Cards such as ID-1 or SAM (Secure Access Module) cards. The TWN4 ISO7816 API offers different system functions for covering different imaginable scenarios. A typical communication flow with contact cards looks like this:



33.1.1 Query Card Slot Status

This function shall be used to query information of the physical card slot status, e.g. to find out if a card is inserted or not. The function returns the slot status in CCID compliant style, this means it return information about slot status, error information and clock status. The internal state of the card is not changed. Please note, depending on the used hardware (TWN4 Desktop or TWN4 SmartCard) the amount of retrievable information differs.

```
bool IS07816_GetSlotStatus(int Channel, TIS07816SlotStatus* SlotStatus);
```

Parameters:

`int Channel`

Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.

`TIS07816SlotStatus* SlotStatus`

The card slot status is returned by this parameter. See the definition of TIS07816SlotStatus for meaning of each member.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

Members	Length (Bits)	Description
TIS07816StatusReg bStatus	8	Slot status register compliant to CCID. See the definition of TIS07816StatusReg for meaning of the different bit fields.
byte bError	8	Error code compliant to CCID.
byte bClockStatus	8	Clock status information compliant to CCID. Possible values are: IS07816_CLOCKSTATUS_RUNNING, IS07816_CLOCKSTATUS_CLKSTPL, IS07816_CLOCKSTATUS_CLKSTPH, IS07816_CLOCKSTATUS_CLKSTPU.

Table 33.1: Definition of TIS07816SlotStatus

Members	Length (Bits)	Description
byte bmICCSStatus	2	Physical status of the card slot. Possible values are: IS07816_ICCPRESENTANDACTIVE, IS07816_ICCPRESENTANDINACTIVE and IS07816_NOICCPRESENT.
byte bmRFU	4	These bits are reserved for future use.
byte bmCommandStatus	2	Command status information compliant to CCID.

Table 33.2: Definition of TIS07816StatusReg

33.1.2 Card Activation

This function shall be used to activate and initialize communication with the card inserted in one of the slots connected to the TWN4 reader. All communication parameters are reset to default. Depending on the hardware platform, the reader shows different behaviour regarding reset-handling of the card: On TWN4 Desktop, calling this function always leads to a warm reset, on TWN4 SmartCard, the first call performs a cold reset and any subsequent function call leads to a warm reset until the card is deactivated. The result of the entire operation is the receipt of the Answer To Reset (ATR) from the card. Based on the content of the ATR, the user may decide how to further proceed with the card. Note that selection of voltage level is only available for TWN4 SmartCard.

```
bool IS07816_IccPowerOn
(
    int Channel,
    byte* ATR,
    int* ATRByteCnt,
    int MaxATRByteCnt,
    byte bPowerSelect,
    TIS07816StatusReg* bStatus,
    byte* bError
);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these predefined constants.
<code>byte*</code> ATR	After successful completion of this function, the buffer referred by this parameter holds the ATR which was read from the card. Take care for adequate dimensioning.
<code>int*</code> ATRByteCnt	After successful completion of this function, this parameter holds the number of bytes, the ATR contains.
<code>int</code> MaxATRByteCnt	This parameter holds the array-size of ATR in bytes.
<code>byte</code> bPowerSelect	Specify the operating voltage level which shall be used for the card. Valid values are ISO7816_POWERSELECT_AUTO, ISO7816_POWERSELECT_5V, ISO7816_POWERSELECT_3V, or ISO7816_POWERSELECT_1V8, use one of these predefined constants.
TISO7816StatusReg* bStatus	The CCID compliant slot status register is returned by this parameter. See the definition of TISO7816StatusReg for meaning of the different bit fields.
<code>byte*</code> bError	The CCID compliant error code is returned by this parameter.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.1.3 Card Deactivation

This function shall be used to deactivate and power off the card. When this function was called on TWN4 SmartCard reader, a subsequent call of `IccPowerOn()` leads to a cold reset of the card.

```
bool ISO7816_IccPowerOff(int Channel, TISO7816SlotStatus* SlotStatus);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these predefined constants.
TISO7816SlotStatus* SlotStatus	The card slot status is returned by this parameter. See the definition of TISO7816SlotStatus for meaning of each member.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.1.4 Set Communication Settings

This function shall be used to assign new communication settings to the respective card slot. After calling this function, the communication parameters which have been negotiated with the card during Protocol And Parameter Selection (PPS) become valid. For issuing a PPS, please refer to the function `ISO7816_Transceive`. Specific communication parameters must be obtained from the ATR, for detailed information refer to standard ISO7816-3.

```
bool ISO7816_SetCommSettings  
(  
    int Channel,
```

```
const TIS07816CommSettings* CommSettings
);
```

Parameters:

int Channel

Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.

const
TIS07816CommSettings*
CommSettings

The new communication settings are passed by this parameter. See the definition of TIS07816CommSettings for meaning of each member.

Return:

If the operation was successful, the return value is true, otherwise it is false.

Members	Length (Bits)	Description
byte Protocol	8	Specify the protocol to be used. Possible values are: IS07816_PROTOCOL_T0 and IS07816_PROTOCOL_T1.
byte Freq	8	Specify the clock frequency which shall be applied to the card. Chose one of the pre-defined constants IS07816_FREQUENCY_1000000, IS07816_FREQUENCY_1250000, IS07816_FREQUENCY_1875000, IS07816_FREQUENCY_2500000, IS07816_FREQUENCY_3750000, IS07816_FREQUENCY_5000000, IS07816_FREQUENCY_7500000 or IS07816_FREQUENCY_15000000.
uint16_t F	16	Specify a non-ISO value for F.
uint16_t D	16	Specify a non-ISO value for D.
union TProtocolData ProtocolData	56	See definition of TProtocolData for details.

Table 33.3: Definition of TIS07816CommSettings

Members	Length (Bits)	Description
TProtocolDataT0 T0	40	See definition of TProtocolDataT0 for details.
TProtocolDataT1 T1	56	See definition of TProtocolDataT1 for details.

Table 33.4: Definition of TProtocolData

Members	Length (Bits)	Description
<code>byte</code> <code>bmFindexDindex</code>	8	Bit 7-4: FI, Index into table 7 of ISO/IEC 7816-3:2006 selecting a clock rate conversion factor. Bit 3-0: DI, Index into table 8 of ISO/IEC 7816-3:2006 selecting a baud rate conversion factor. This value shall be obtained from TA1 of the ATR.
<code>byte</code> <code>bmTCKCKST0</code>	8	This value shall be set to 00h.
<code>byte</code> <code>bGuardTimeT0</code>	8	Extra Guardtime between two characters. Add 0 to 254 etu to the normal guardtime of 12 etu. FFh is the same as 00h. This value shall be obtained from TC1 of the ATR.
<code>byte</code> <code>bWaitingIntegerT0</code>	8	Waiting time between transmission of a command and reception of the response. This value is specified in TC2 of the ATR. If TC2 is not present, the default value is 10.
<code>byte</code> <code>bClockStop</code>	8	This value shall be set to 00h.

Table 33.5: Definition of TProtocolDataT0

Members	Length (Bits)	Description
<code>byte</code> <code>bmFindexDindex</code>	8	Bit 7-4: FI, Index into table 7 of ISO/IEC 7816-3:2006 selecting a clock rate conversion factor. Bit 3-0: DI, Index into table 8 of ISO/IEC 7816-3:2006 selecting a baud rate conversion factor. This value shall be obtained from TA1 of the ATR.
<code>byte</code> <code>bmTCKCKST1</code>	8	This value shall be set to 00h.
<code>byte</code> <code>bGuardTimeT1</code>	8	Extra Guardtime (0 to 254 etu between two characters). If value is FFh, then guardtime is reduced by 1 etu. This value shall be obtained from TC1 of the ATR.
<code>byte</code> <code>bWaitingIntegerT1</code>	8	Bit 7-4: BWI, values 0-9 valid. Bit 3-0: CWI, values 0-Fh valid. This value is specified in the first TB for T=1 in the ATR.
<code>byte</code> <code>bClockStop</code>	8	This value shall be set to 00h.
<code>byte</code> <code>bIFSC</code>	8	Size of negotiated IFSC in bytes. This value is specified in the first TA for T=1 in the ATR.
<code>byte</code> <code>bNadValue</code>	8	This value shall be set to 00h.

Table 33.6: Definition of TProtocolDataT1

33.1.5 Transparent Data Transmission

This function shall be used for byte-wise communication with the card.

```
bool ISO7816_Transceive
(
    int Channel,
    const byte* TX,
    int LenTX,
    byte* RX,
    int* LenRX,
    int MaxRXByteCnt,
    TISO7816StatusReg* bStatus,
    byte* bError
);
```

Parameters:

<code>int Channel</code>	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>const byte* TX</code>	This buffer holds the data which shall be transmitted to the card.
<code>int LenTX</code>	This parameter specifies the data-length in bytes which shall be transmitted to the card.
<code>byte* RX</code>	This buffer holds the data which was read from the card. Take care for adequate dimensioning.
<code>int* LenRX</code>	After successful completion of this function, this parameter holds the number of bytes read from the card.
<code>int MaxRXByteCnt</code>	This parameter holds the array-size of RX in bytes.
<code>TISO7816StatusReg* bStatus</code>	The CCID compliant slot status register is returned by this parameter. See the definition of TISO7816StatusReg for meaning of the different bit fields.
<code>byte* bError</code>	The CCID compliant error code is returned by this parameter.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

33.1.6 Exchange Of APDUs

This function shall be used for APDU exchange based on T=0/T=1 protocol according to ISO7816-3.

```
bool ISO7816_ExchangeAPDU
(
    int Channel,
    const TISO7816_ProtocolHeader* Header,
    const byte* TXData,
    int TXByteCnt,
    byte* RXData,
    int* RXByteCnt,
    int MaxRXByteCnt,
    uint16_t* StatusWord
);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>const</code> TIS07816_ProtocolHeader *Header	This parameter holds basic APDU information.
<code>const byte*</code> TXData	This buffer holds the data field of the APDU.
<code>int</code> TXByteCnt	This parameter specifies the data-length in bytes of the data-field.
<code>byte*</code> RXData	This buffer holds the data-field of the received APDU.
<code>int*</code> RXByteCnt	After successful completion of this function, this parameters holds the data-field size of the received APDU.
<code>int</code> MaxRXByteCnt	This parameter holds the array-size of RXData in bytes.
<code>uint16_t*</code> StatusWord	This parameter holds the status word received from the card.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

Members	Length (Bits)	Description
<code>byte</code> CLA	8	This member holds the CLA-value.
<code>byte</code> INS	8	This member holds the INS-code.
<code>byte</code> P1	8	This member holds the parameter P1.
<code>byte</code> P2	8	This member holds the parameter P2.
<code>uint16_t</code> Lc	16	This member holds Lc which defines the size of the following data-field.
<code>uint16_t</code> Le	16	This member holds Le which defines the maximum expected size of the response data-field.
<code>struct</code> TIS07816_ProtocolHeaderFlags Flags	8	This member holds additional APDU information.

Table 33.7: Definition of TIS07816_ProtocolHeader

Members	Length (Bits)	Description
<code>byte</code> LePresent	1	If set to <code>true</code> , Le is transmitted.
<code>byte</code> ExtendedAPDU	1	If set to <code>true</code> , this APDU is sent as Extended APDU.
<code>byte</code> RFU	6	Reserved for future use.

Table 33.8: Definition of TIS07816_ProtocolHeaderFlags

33.1.7 Examples

33.1.7.1 PPS Example

The following example shows how to make a PPS with an ISO7816 card.

```
bool ISO7816_PPS(int Channel, byte Protocol, byte* bmFindexDindex)
{
    byte Cmd[4];
    byte Res[4];
    int TxByteCnt;
    int RxByteCnt;
    TISO7816StatusReg bStatus;
    byte bError;

    // PPS always starts with 0xFF
    Cmd[0] = 0xFF;
    // The second byte stores the desired protocol
    Cmd[1] = Protocol & 0x0F;
    // Is bmFindexDindex present?
    if (bmFindexDindex != NULL)
    {
        // Yes, prepare the command accordingly
        Cmd[1] |= 0x10;
        Cmd[2] = *bmFindexDindex;
        // Calculate the BCC over all command bytes
        Cmd[3] = Cmd[0] ^ Cmd[1] ^ Cmd[2];
        TxByteCnt = 4;
    }
    else
    {
        // FindexDindex is not present, calculate only BCC
        Cmd[2] = Cmd[0] ^ Cmd[1];
        TxByteCnt = 3;
    }
    // Send PPS request to the card, get response
    if (!ISO7816_Transceive(Channel, Cmd, TxByteCnt, Res,
                           &RxByteCnt, sizeof(Res), &bStatus, &bError))
        return false;
    // We expect the card to echo the request in its response
    if (RxByteCnt != TxByteCnt)
        return false;
    return memcmp(Cmd, Res, RxByteCnt) == 0;
}
```

33.1.7.2 Communication Example

The following example shows how to prepare a ISO7816 card for communication at T=1 protocol and exchange APDUs.

```
byte ATR[32];
int ATRByteCnt;

TISO7816SlotStatus SlotStatus;
TProtocolDataT1 ProtocolDataT1;
```

```
TISO7816CommSettings CommSettings;

TISO7816_ProtocolHeader Header;
byte TXData[128];
byte RXData[128];
int RXByteCnt;
uint16_t SW12;

// We want to use T=1 protocol with the following non-default values
ProtocolDataT1.bmFindexDindex = 0x98;
ProtocolDataT1.bmTCKKST1 = 0;
ProtocolDataT1.bGuardTimeT1 = 0xFF;
ProtocolDataT1.bmWaitingIntegersT1 = 0x55;
ProtocolDataT1.bClockStop = 0;
ProtocolDataT1.bIFSC = 0xFE;
ProtocolDataT1.bNadValue = 0x00;

MainLoop:
while (true)
{
    // Is a card inserted in CHANNEL_SC1?
    if (!ISO7816_GetSlotStatus(CHANNEL_SC1, &SlotStatus))
        goto MainLoop;
    // Card slot empty?
    if (SlotStatus.bStatus.bmICCStatus == ISO7816_NOICCPRESENT)
        goto MainLoop;
    // Perform activation of the card and receive ATR
    if (!ISO7816_IccPowerOn
        (
            CHANNEL_SC1,
            ATR,
            &ATRByteCnt,
            sizeof(ATR),
            ISO7816_POWERSELECT_5V,
            &SlotStatus.bStatus,
            &SlotStatus.bError
        ))
        goto MainLoop;
    // We expect the card to be present and active
    if (SlotStatus.bStatus.bmICCStatus != ISO7816_ICCPRESENTANDACTIVE)
        goto MainLoop;
    // Perform PPS for T=1 protocol
    if (!ISO7816_PPS(CHANNEL_SC1, ISO7816_PROTOCOL_T1,
                    &ProtocolDataT1.bmFindexDindex))
        goto MainLoop;
    // Setup ISO7816 UART accordingly
    CommSettings.Protocol = ISO7816_PROTOCOL_T1;
    CommSettings.Freq = ISO7816_FREQUENCY_5000000;
    CommSettings.F = 512;
    CommSettings.D = 12;
    CommSettings.ProtocolData.T1 = ProtocolDataT1;
    if (!ISO7816_SetCommSettings(CHANNEL_SC1, &CommSettings))
        goto MainLoop;
    // Let's prepare our APDU. We want to select the Masterfile (MF)
    // of a PKI card by its SFI (0x3F00).
    Header.CLA = 0x00;
    Header.INS = 0xA4;
    Header.P1 = 0x00;
```

```

Header.P2 = 0x00;
Header.Lc = 0x0002;
Header.Le = 0x0000;
Header.Flags.LePresent = true;
Header.Flags.ExtendedAPDU = false;
TXData[0] = 0x3F;
TXData[1] = 0x00;

// Exchange the APDU
if (!IS07816_ExchangeAPDU(CHANNEL_SC1, &Header, TXData, Header.Lc,
                          RXData, &RXByteCnt, sizeof(RXData), &SW12))
    goto MainLoop;
// Check status word of the received APDU
if (SW12 == 0x9000)
{
    // Further APDUs may follow...
    DoSomething();
}
}

```

33.2 SLE Memory Cards

This chapter shows how to use TWN4 with contact based memory cards such as SLE44xx or compatible cards. In order to query the card slot insertion state, the function `IS07816_GetSlotStatus` can be used.

33.2.1 Get ATR

Use this function to retrieve the ATR (Answer To Reset) from an inserted card.

```
bool SLE_GetATR(int Channel, byte* ATR);
```

Parameters:

int Channel Specify a communication channel by this parameter. Valid values are `CHANNEL_SAM1` through `CHANNEL_SAM4` or `CHANNEL_SC1`, use one of these pre-defined constants.

byte* ATR The card's ATR is returned by this buffer. The function always returns 4 bytes.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

33.2.2 Read Main Memory

Use this function to read data from the main memory.

```
bool SLE_ReadMainMemory(int Channel, int Address, byte* Data, int ByteCnt);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Address	Specify the start address in memory for reading.
<code>byte*</code> Data	This buffer holds the data read from the card. Take care for proper dimensioning.
<code>int</code> ByteCnt	Specify the number of bytes to be read.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.3 Write Main Memory

Use this function to write one byte of data to the main memory.

```
bool SLE_UpdateMainMemory(int Channel, int Address, byte Value);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Address	Specify the address in memory to be written.
<code>byte</code> Value	Specify the data byte to be written.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.4 Read Security Memory

Use this function to read out the four bytes of Security Memory.

```
bool SLE_ReadSecurityMemory(int Channel, byte* SecMemData);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>byte*</code> SecMemData	This buffer holds the Security Memory data read from the card. The function always returns 4 bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.5 Write Security Memory

Use this function to write one byte of data to the Security Memory.

```
bool SLE_UpdateSecurityMemory(int Channel, int Address, byte SecMemData);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Address	Specify the address in Security Memory to be written.
<code>byte</code> SecMemData	Specify the data byte to be written.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.6 Read Protection Memory

Use this function to read out the four bytes of Protection Memory.

```
bool SLE_ReadProtectionMemory(int Channel, byte* ProtMemData);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>byte*</code> ProtMemData	This buffer holds the Protection Memory data read from the card. The function always returns 4 bytes.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.7 Write Protection Memory

Use this function to write one byte of data to the Protection Memory.

```
bool SLE_WriteProtectionMemory(int Channel, int Address, byte ProtMemData);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Address	Specify the address in Protection Memory to be written.
<code>byte</code> ProtMemData	Specify the data byte to be written.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.2.8 Compare Verification Data

Use this function to transmit one byte of verification input to the card.

```
bool SLE_CompareVerificationData(int Channel, int Address, byte VerificationData);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Address	Specify the address of verification data byte.
<code>byte</code> VerificationData	Specify the verification data byte to be transferred to the card.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.3 I2C Memory Cards

This chapter shows how to use TWN4 with contact based I2C memory cards. In order to query the card slot insertion state, the function `IS07816_GetSlotStatus` can be used.

33.3.1 Read Memory

Use this function to read data from the memory.

```
bool I2CCard_Read(int Channel, int Addr, byte* Data, int ByteCnt);
```

Parameters:

<code>int</code> Channel	Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.
<code>int</code> Addr	Specify the start address in memory for reading.
<code>byte*</code> Data	This buffer holds the data read from the card. Take care for proper dimensioning.
<code>int</code> ByteCnt	Specify the number of bytes to be read.
<u>Return:</u>	If the operation was successful, the return value is <code>true</code> , otherwise it is <code>false</code> .

33.3.2 Write Memory

Use this function to write data to the memory.

```
bool I2CCard_Write(int Channel, int Addr, const byte* Data, int ByteCnt);
```


Parameters:`int` Channel

Specify a communication channel by this parameter. Valid values are CHANNEL_SAM1 through CHANNEL_SAM4 or CHANNEL_SC1, use one of these pre-defined constants.

`int` Addr

Specify the start address in memory for the write operation.

`const byte*` Data

Specify data to be written.

`int` ByteCnt

Specify the number of bytes to be written.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`.

34 Cryptographic Operations

The cryptographic API incorporates methods for encryption/decryption of data, these are Triple-DES (Data Encryption Standard) or AES (Advanced Encryption Standard). TDES is available in two versions that support different key-lengths: 128 bit (TDES2K) and 192 bit (TDES3K).

The implementation of TDES is based on FIPS PUB 46-3. The method always operates on entire data blocks of 8 bytes. The DES algorithm is passed three times for one TDES operation. In case of TDES2K, the 128 bit key is hereby split into two parts: K1 and K2. In case of TDES3K, the 192 bit key is split into three parts: K1, K2 and K3.

The implementation of AES is based on FIPS PUB 197. The method always operates on entire data blocks of 16 bytes, the key-length is 128 bit.

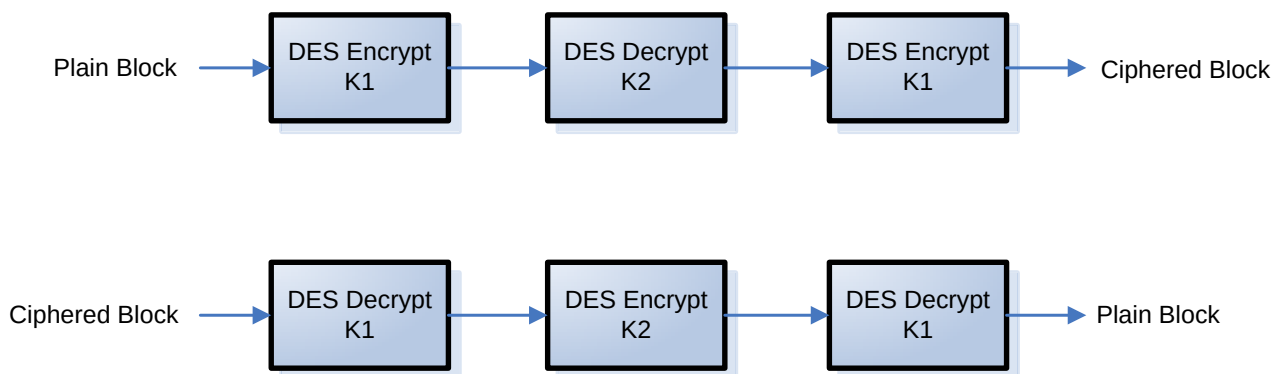


Figure 34.1: TDES2K Operation

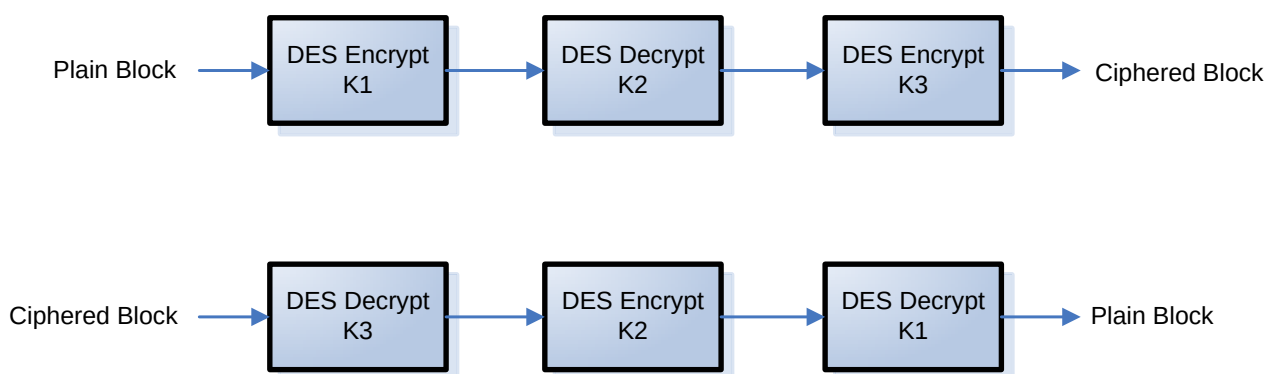


Figure 34.2: TDES3K Operation

The cryptographic API may be used to simply encrypt/decrypt a single block or to encrypt/decrypt a chain of blocks using the CBC-method (Ciphered Block Chaining).

In CBC mode, every ciphering operation depends on the foregoing step, this is achieved by involving the

so-called Init Vector IV. The first CBC-operation usually works with an Init Vector that is set to zero. For encryption, a plain data block P is logically XOR-ed with this Init Vector before it comes to encryption. The result is a ciphered block C which serves as Init Vector for the next operation. See the schematic below for details:

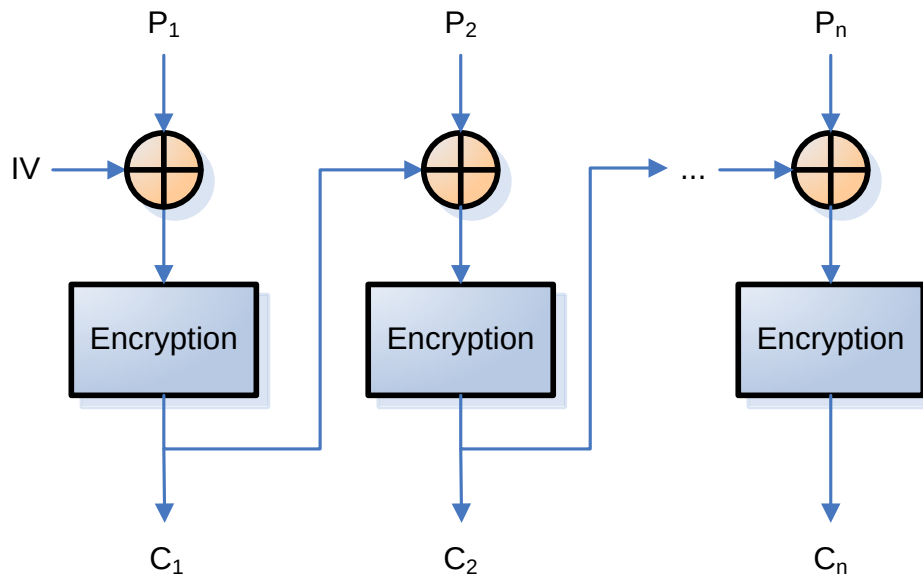


Figure 34.3: CBC Enciphering scheme

If a ciphered block C is decrypted, the result is logically XOR-ed with the Init Vector. See the schematic below for details:

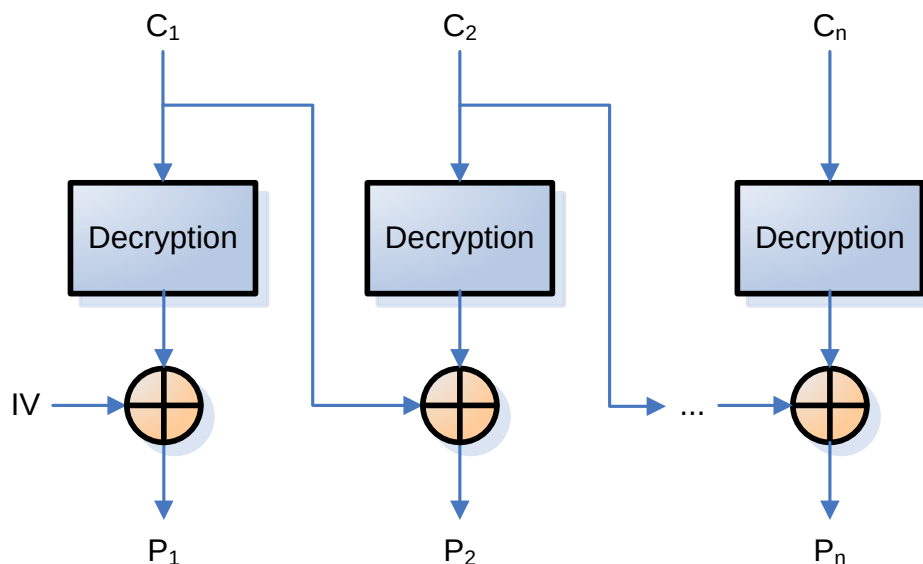


Figure 34.4: CBC Deciphering scheme

34.1 Initialization

The cryptographic API has to be initialized before it can be used. During initialization the key is passed to the cryptographic method and assigned to a cryptographic environment. After initialization the functions for encryption and decryption are set up for the desired cryptographic mode. If a cryptographic environment is configured for CBC-operation, the internally managed Init Vector is automatically reset to zero.

```
void Crypto_Init
(
    int CryptoEnv,
    int CryptoMode,
    const byte* Key,
    int KeyByteCnt
);
```

Parameters:

<code>int CryptoEnv</code>	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>int CryptoMode</code>	Specify the mode of cryptographic operation. Choose either one of the predefined non-CBC constants CRYPTOMODE_3DES, CRYPTOMODE_3K3DES, CRYPTOMODE_AES128 or one of the pre-defined CBC constants CRYPTOMODE_CBC_DES, CRYPTOMODE_CBC_DFN_DES, CRYPTOMODE_CBC_3DES, CRYPTOMODE_CBC_DFN_3DES, CRYPTOMODE_CBC_3K3DES, CRYPTOMODE_CBC_AES128.
<code>const byte* Key</code>	The key is passed by this parameter. Depending on the specified crypto mode, the key-length is either 16 or 24 bytes.
<code>int KeyByteCnt</code>	Specify the length of the key in bytes.
<u>Return:</u>	This function has no return value.

34.2 Encrypt

Use this function to encrypt a plain block of data.

```
void Encrypt
(
    int CryptoEnv,
    const byte* PlainBlock,
    byte* CiphertextBlock,
    int BlockByteCnt
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>const byte*</code> PlainBlock	Pointer to the array, that contains the plain data block to be encrypted.
<code>byte*</code> CiphersedBlock	Pointer to the array, that receives the encrypted data block. Take care for proper dimensioning.
<code>int</code> BlockByteCnt	Specify the number of bytes of a block.
<u>Return:</u>	This function has no return value.

34.3 Decrypt

Use this function to decrypt an encrypted block of data.

```
void Decrypt
(
    int CryptoEnv,
    const byte* CiphersedBlock,
    byte* PlainBlock,
    int BlockByteCnt
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<code>const byte*</code> CiphersedBlock	Pointer to the array, that holds the encrypted data block.
<code>const byte*</code> PlainBlock	Pointer to the array, that receives the decrypted data block. Take care for proper dimensioning.
<code>int</code> BlockByteCnt	Specify the number of bytes of a block.
<u>Return:</u>	This function has no return value.

34.4 Reset Init Vector

Use this function to manually reset the internally managed Init Vector of a cryptographic environment to zero.

```
void CBC_ResetInitVector
(
    int CryptoEnv
);
```

Parameters:

<code>int</code> CryptoEnv	Specify a cryptographic environment by this parameter. The valid range is CRYPTO_ENV0 to CRYPTO_ENV3, use one of these predefined constants.
<u>Return:</u>	This function has no return value.

35 Storage Functions

This chapter describes functions for accessing the storage of TWN4.

The storage memory is part of the internal flash of the main control unit (MCU) of TWN4. The gross amount of this storage is 48kByte. Due to segmentation of the memory and further control mechanisms, after deduction the memory size is 18kByte.

Before first use of the storage, the storage must be formatted. The appropriate system function for doing so is `FSFormat`.

In order to gain access to the storage memory, the file system must be initialized and connected to the internal flash. This can be achieved with the system function `FSMount`.

Why is a separate mount needed to gain access to the storage memory?

The reason for a separate mount is, that there could be a reasonable amount of time required in order to start the file system. Background is, that depending on the state of the file system, additional activities must be started, before access of the storage memory is possible. There is especially the situation, which can occur, if last file operation were interrupted by a unplanned power fail. This can lead to the situation, that the file system must be reset to the state, before the interrupted file operation was started. This clean-up is done by function `FSMount`

The structure of the storage memory is similar to a none-hierarchical file system. Following points must be known:

- Data is structured in files.
- Files are indicated by a file ID. The file ID is any 32 bit number.
- It is possible to iterate through the existing files and thus list the files stored in the memory.
- There is a maximum number of files, which can be stored in the memory. This maximum number is 16.
- In order to read from or write to files, appropriate system functions are available. In order to start a file operation, the file must be opened for appropriate file operation. The maximum number of files, which can be kept opened at a time is 4.
- File operations are kept atomic. This means: If a change to a file (some kind write operation) is interrupted by a power fail, the file system returns to the state, where the change began.

35.1 Management Functions

35.1.1 FSMount

Before any access to files can be performed, the appropriate file system must be mounted. Following steps are performed by function `FSMount`:

- Check, if the specified volume contains a valid file system.

- Check, if there is a not completed file operation.
- If applicable, unwind file system to the point where not completed file operation was started.
- Finally, create a logical link between volume and file system.

```
bool FSMount(int StorageID, int Mode)
```

Parameters:

`int StorageID` Specifies the volume, which should be mounted. Currently, there is one storage available, the internal flash. The appropriate definition for this storage is `SID_INTERNALFLASH`

`int Mode` Specifies the mode in which the volume is mounted. This can be `FS_MOUNT_NONE` (equivalent to a unmount), `FS_MOUNT_READONLY` (no write access to storage possible) or `FS_MOUNT_READWRITE` (full read/write access).

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.1.2 FSFormat

This function prepares the storage memory of TWN4 for further file operations.

— WARNING — WARNING — WARNING —

All data, which is stored on the file system will be irrecoverable deleted by calling this function!

```
bool FSFormat(int StorageID, int MagicValue)
```

Parameters:

`int StorageID` Specifies the volume, which should be formatted.

`int MagicValue` In order to avoid accidentally format of a volume, an appropriate parameter for `MagicValue` must be specified. There is a definition for this magic value, which is `FS_FORMATMAGICVALUE`.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.2 File Functions

35.2.1 FSOpen

This function must be called in order to begin any read or write operation from/to a file.

Following definitions for the parameter mode are valid:

FS_READ	Open a file for read access. If the file not exists, an error is generated. The position of the read pointer is set to zero, thus to the start of the file.
FS_WRITE	Open a file for write access. An empty file is created independently of if the file already exists or not, thus content of an earlier version of that file will be deleted.
FS_APPEND	Open a file for write access. If the file does not exist, a new file is created. If the file already exists, the file pointer is moved to the end of the file, which means, that newly written data is appended to data of existing file.

Following further considerations:

- A file can be opened one time in mode FS_WRITE or FS_APPEND, but never, if it is already opened by any other file operation.
- A file can be opened many times in mode FS_READ, but never, if it is already opened in mode FS_WRITE or FS_APPEND by another file operation.

```
bool FSOpen(int FileEnv, int StorageID, uint32_t FileID, int Mode)
```

Parameters:

int FileEnv	Specifies the environment to be used for the file operation. Up to four file operations can be opened at a time. The appropriate definitions for these environments are FILE_ENV0 - FILE_ENV3.
int StorageID	Specifies the storage on which the file resides. Currently, this parameter can be SID_INTERNALFLASH only.
uint32_t FileID	Specifies the ID of a file. The file ID is a reduced version of file name and be understood as such. File ID is an integer number from 1 to $2^{32} - 1$, thus 0x00000000 to 0xFFFFFFFF.
int Mode	Specifies, how the file is accessed (see above).

Return: If the operation was successful, the return value is true, otherwise it is false. A concrete error code can be retrieved with system function GetLastError.

X

35.2.2 FSClose

This function is used to terminate a file operation. Several actions are taken, when this function is called:

- Pending data is written to the storage system.
- If this is the last file being closed, the file system is finalized in terms, that the even loss of the power will restore this now achieved state.

```
bool FSClose(int FileEnv)
```

Parameters:

int FileEnv	Specifies the environment to be used for the file operation.
-------------	--

Return: If the operation was successful, the return value is true, otherwise it is false. A concrete error code can be retrieved with system function GetLastError.

35.2.3 FSCloseAll

This function is closing all opened file operations throughout all mounted storages. This function avoids keeping track of opened file operations.

```
void FSCloseAll(void)
```

Parameters: None.

Return: None.

35.2.4 FSeek

Read and write operations from/to a file are implemented via a file pointer, which references the point, from which next data is read or where next data is written. With this function, the file pointer can be moved throughout a file and furthermore in relation to a specific point of the file.

FS_POSABS	Move file position in relation to the start of the file. This results in a move of the file pointer to an absolute position.
FS_POSREL	Move the file pointer in relation to the current position. This allows an easy skip of a number of bytes of the file.
FS_POSEND	Move the file pointer in relation to the end of the file. This allows to move to the end of the file without knowledge and independent of the length of a file.

```
bool FSeek(int FileEnv, int Origin, int Pos)
```

Parameters:

int FileEnv	Specifies the environment to be used for the file operation.
int Origin	Specifies the reference point, from which the new file position is calculated (see above).
int Pos	Specifies the number of bytes in relation to the reference point. A negative value is treated as position before reference point, a positive value is treated as position behind the reference point.

Return: If the operation was successful, the return value is true, otherwise it is false. A concrete error code can be retrieved with system function GetLastError.

35.2.5 FTell

This function returns the position of the file pointer in relation to a reference point. Please note that in consequence, specifying FS_POSREL as origin must always return the value zero.

```
bool FTell(int FileEnv, int Origin, int *Pos)
```

Parameters:

<code>int FileEnv</code>	Specifies the environment to be used for the file operation.
<code>int Origin</code>	Specifies the reference point, under which the current position is calculated (see function <code>FSSeek</code>).
<code>int *Pos</code>	A pointer to an integer, which will receive the value of the position.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.2.6 FSReadBytes

Read bytes from a file, which has been opened in mode `FS_READ` before. Use function `FSOpen` to open the file accordingly.

The function generates the error `ERR_ENDOFFILE`, if less than the requested number of bytes were read from the file or if there are no more bytes left to be read from the file.

```
bool FSReadBytes(int FileEnv, void *Data, int ByteCount, int *BytesRead)
```

Parameters:

<code>int FileEnv</code>	Specifies the environment to be used for the file operation.
<code>void *Data</code>	Pointer to an array of bytes, which receives read data.
<code>int ByteCount</code>	Number of bytes, which should be read from the file.
<code>int *BytesRead</code>	Pointer to an integer, which receives the number of actually read bytes. The received value is valid even if the function returns with an error.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.2.7 FSWriteBytes

Write bytes to a file, which has been opened in mode `FS_WRITE` or `FS_APPEND` before. Use function `FSOpen` to open the file accordingly.

```
bool FSWriteBytes(int FileEnv, const void *Data, int ByteCount, int *BytesWritten)
```

Parameters:

<code>int FileEnv</code>	Specifies the environment to be used for the file operation.
<code>const void *Data</code>	Pointer to an array of bytes, which contains data to be written.
<code>int ByteCount</code>	Number of bytes, which should be written to the file.
<code>int *BytesWritten</code>	Pointer to an integer, which receives the number of actually written bytes. The received value is valid even if the function returns with an error.

Return:

If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.3 Directory Functions

35.3.1 FSFindFirst

The functions FSFindFirst/FSFindNext implement the possibility to enumerate the files contained in a files system. In order to begin enumeration of files the function FSFindFirst must be called.

The members of a directory entry are stored in a structure of type TFileInfo. The members of the structure are:

ID	The file ID.
Length	The length of the file.

```
bool FSFindFirst(int StorageID, TFileInfo *pFileInfo)
```

Parameters:

int StorageID	Storage ID of the file system, where files should be enumerated.
TFileInfo *pFileInfo	Pointer to a structure of type TFileInfo which receives a directory entry.

Return: If the operation was successful, the return value is true, otherwise it is false. If no directory entry was found the error code ERR_FILENOTFOUND is generated. The concrete error code can be retrieved with system function GetLastError.

35.3.2 FSFindNext

The functions FSFindFirst/FSFindNext implement the possibility to enumerate the files contained in a files system. In order to continue enumeration, once first entry has been retrieved with function FSFindFirst, the function FSFindNext must be called.

```
bool FSFindNext(TFileInfo *pFileInfo)
```

Parameters:

TFileInfo *pFileInfo	Pointer to a structure of type TFileInfo which receives a directory entry.
----------------------	--

Return: If the operation was successful, the return value is true, otherwise it is false. If no directory entry was found the error code ERR_FILENOTFOUND is generated. The concrete error code can be retrieved with system function GetLastError.

35.3.3 FSDelete

Use function FSDelete to delete files from the file system. A file, which is currently opened can not be deleted.

```
bool FSDelete(int StorageID, uint32_t FileID)
```

Parameters:

`int StorageID` Storage ID of the file in question.
`uint32_t FileID` File ID of the file to be deleted.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.3.4 FSRename

Use function `FSRename` to rename files on the file system.

```
bool FSRename(int StorageID, uint32_t OldFileID, uint32_t NewFileID)
```

Parameters:

`int StorageID` Storage ID of the file in question.
`uint32_t OldFileID` Current file ID of the file to be renamed.
`uint32_t NewFileID` Future file ID of the file to be renamed.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

35.4 Miscellaneous Functions**35.4.1 FSGetStorageInfo**

Function `FSGetStorageInfo` allows to retrieve information regarding a storage.

```
bool FSGetStorageInfo(int StorageID, TStorageInfo *pStorageInfo)
```

Parameters:

`int StorageID` ID of the storage in question.
`TStorageInfo *pStorageInfo` Pointer to a structure of type `TStorageInfo`, which receives the requested information.

Return: If the operation was successful, the return value is `true`, otherwise it is `false`. A concrete error code can be retrieved with system function `GetLastError`.

The structure `TStorageInfo` is defined as follows:

```
typedef struct
{
    byte ID;
    uint32_t Size;
    uint32_t Free;
} TStorageInfo;
```

where:

byte ID	ID of the storage in question.
uint32_t Size	Size in bytes of the storage.
uint32_t Free	Number of free bytes in the storage.

35.5 Examples

This is an example for a function, which reads a complete file from the file system. The file system must have been mounted before with function `FSMount`.

```
bool ReadFile1(uint32_t FileID, byte *Data, int *FileLength, int MaxFileLength)
{
    if (!FSOpen(FILE_ENVO, SID_INTERNALFLASH, FileID, FS_READ))
        return false;
    FSReadBytes(FILE_ENVO, Data, MaxFileLength, FileLength);
    int LastError = GetLastError();
    FSClose(FILE_ENVO);
    if (LastError != ERR_NONE && LastError != ERR_ENDOFFILE)
        return false;
    // Function was successfully completed
    return true;
}
```

Here is an example for a function, which reads a complete file from the file system but in portions of 256 bytes. This might be useful, if the implementation is actually done on a host, which is doing system calls indirectly via TWN4 Simple Protocol. The file system must have been mounted before with function `FSMount`.

```
bool ReadFile2(uint32_t FileID, byte *Data, int *Length, int ExpectedLength)
{
    *Length = 0;
    if (!FSOpen(FILE_ENVO, SID_INTERNALFLASH, FileID, FS_READ))
        return false;
    bool ReadSuccess;
    int RemainingBytes = ExpectedLength;
    do
    {
        if (RemainingBytes == 0)
        {
            FSClose(FILE_ENVO);
            return true;
        }
        const int BlockSize = 256;
        int BytesToRead = RemainingBytes;
        if (BytesToRead > BlockSize)
            BytesToRead = BlockSize;
        int BytesRead;
        ReadSuccess = FSReadBytes(FILE_ENVO, Data, BytesToRead, &BytesRead);
        Data += BytesRead;
        *Length += BytesRead;
        RemainingBytes -= BytesRead;
    }
    while (ReadSuccess);
    int LastError = GetLastError();
    FSClose(FILE_ENVO);
    if (LastError != ERR_NONE && LastError != ERR_ENDOFFILE)
```

```

        return false;
    // Function was successfully completed
    return true;
}

```

Here is an example for a function, which writes a complete file to the file system in portions of 256 bytes. This might be useful, if the implementation is actually done on a host, which is doing system calls indirectly via TWN4 Simple Protocol. The file system must have been mounted before with function FSMount.

```

bool WriteFile(uint32_t FileID, byte *Data, int Length)
{
    if (!FSOpen(FILE_ENVO, SID_INTERNALFLASH, FileID, FS_WRITE))
        return false;
    bool WriteSuccess;
    int RemainingBytes = Length;
    do
    {
        if (RemainingBytes == 0)
        {
            FSClose(FILE_ENVO);
            return true;
        }
        const int BlockSize = 256;
        int BytesToWrite = RemainingBytes;
        if (BytesToWrite > BlockSize)
            BytesToWrite = BlockSize;
        int BytesWritten;
        WriteSuccess = FSWriteBytes(FILE_ENVO, Data, BytesToWrite, &BytesWritten);
        Data += BytesWritten;
        RemainingBytes -= BytesWritten;
    }
    while (WriteSuccess);
    int LastError = GetLastError();
    FSClose(FILE_ENVO);
    if (LastError != ERR_NONE)
        return false;
    // Function was successfully completed
    return true;
}

```

36 PKOC Specific Transponder Operations

TWN4 offers support of Public Key Open Credential transponders, both BLE and NFC. The following API functions allow interactions with both types, per the specifications provided on the PSIA website.

36.1 Read NFC Credential

Use this function to handle the interaction between the reader and a selected PKOC NFC physical or mobile credential.

```
bool PKOC_ReadNFC
(
byte* PublicKeyData,
int OutputPublicKeyByteCnt
);
```

Parameters:

byte* PublicKeyData Pointer to an array that will hold the public key value returned by the credential.

int OutputPublicKeyByteCnt The number of bytes of Public Key data to read from the credential (Between 8 and 32, inclusive).

Return: If the operation was successful, the return value is `true`, otherwise it is `false`.

36.2 Set NFC Parameters

Use this function to set the Reader Identifier value to be used during the NFC PKOC transaction.

```
void PKOC_SetParamsNFC
(
byte* ReaderID_NFC
);
```

Parameters:

byte* ReaderID_NFC 32 byte array composed of 16 byte Site Key Identifier and 16 byte Reader Location Identifier.

Return: None.

36.3 Set BLE Parameters

Use this function to set the various parameters that will be used during the PKOC BLE transaction. Must be called prior to calling BLEInit with the parameter BLE_MODE_PKOC.

```
void PKOC_SetParamsBLE
(
int BLEPKOCVersion,
byte* ReaderLocationID_BLE,
byte* SiteID_BLE,
byte* SourceGUUID,
byte* BLEAdvertisedName,
int AdvertisedNameCharacters,
int OutputPublicKeyBytes
);
```

Parameters:

<code>int BLEPKOCVersion</code>	Should normally be PKOC_BLE_VERSION_3, though to work with some older PKOC demonstrator apps, it may need to be PKOC_BLE_VERSION_2.
<code>byte* ReaderLocationID_BLE</code>	16 byte array as defined in the PKOC BLE specification.
<code>byte* SiteID_BLE</code>	16 byte array as defined in the PKOC BLE specification.
<code>byte* SourceGUUID</code>	16 byte array as defined in the deprecated PKOC BLE 2.0 specification.
<code>byte* BLEAdvertisedName</code>	Array containing ASCII bytes defining the name to use in the BLE advertisement from the reader. Maximum of 10 characters.
<code>int AdvertisedNameCharacters</code>	The number of characters provided in BLEAdvertisedName.
<code>int OutputPublicKeyBytes</code>	The number of bytes of Public Key data to read from the credential (Between 8 and 32, inclusive).

Return: If the operation was successful, the return value is true, otherwise it is false.

36.4 Set Site Key

Use this function to set the site private key to be used for the ECDHE BLE transaction.

```
void PKOC_SetSiteKey
(
byte* SitePrivateKey
);
```

Parameters:

<code>byte* SitePrivateKey</code>	32 Byte private key to be used in the ECDHE BLE Transaction.
-----------------------------------	--

Return: None.

37 System Parameters

The TWN4 App-system provides methods of setting up parameters before or during runtime of Apps.

- In order to set up parameters before the App is started, a so-called Manifest can be specified as part of an App.
- In order to set up parameters during normal execution of an App there is the system function `SetParameters`.

This section describes the specification of a Manifest and all available parameters. See chapter "System Functions" for a description of function `SetParameters`.

37.1 TLV Format

Parameters for a Manifest or the system function `SetParameters` are specified in the TLV format. The TLV format specifies a chain of parameters with variable type and length. This format must follow following rules:

- Every entry (except the last entry) is a sequence of 3 items. The 3 items are 'Type', 'Length' and 'Value'.
- The name of the parameter is associated to 'Type', the length of
- the value is associated to 'Length' and the value itself is associated to 'Value'
- The TLV list must be terminated with an item consisting of just the type. This type must contain the value `TLV_END`.

37.2 Manifest

The intention for specifying a Manifest as part of an App could be to avoid opening of communication channels in order to further reduce current consumption. Another could be to modify behaviour of the USB section of TWN4.

The specification of a Manifest is pretty simple:

Define an array of bytes with the key-name Manifest. This will point the firmware of TWN4 to the position where the parameters of interest are stored. Here is an example:

Example:

```
// This sample demonstrates the specification of a Manifest:
const byte *Manifest =
{
    OPEN_PORTS, 1, OPEN_PORT_USB_MSK, // Open USB channel only
    TLV_END // End of TLV
};
```

No further action is required.

37.3 Available Parameters

Here is a list of all parameters, which are supported:

Type (Parameter)	Length	Value
TLV_END	N/A	N/A
OPEN_PORTS	1	Bitwise OR of one or more of the following definitions: OPEN_PORT_USB_MSK OPEN_PORT_COM1_MSK OPEN_PORT_COM2_MSK
EXECUTE_APP	1	EXECUTE_APP_AUTO EXECUTE_APP_ALWAYS
INDITAG_READMODE	1	INDITAG_READMODE_1 INDITAG_READMODE_2
COTAG_READMODE	1	COTAG_READMODE_HASH COTAG_READMODE_1 COTAG_READMODE_2
COTAG_VERIFY	1	COTAG_VERIFY_OFF COTAG_VERIFY_ON
HONEYTAG_READMODE	1	HONEYTAG_READMODE_HASH HONEYTAG_READMODE_1
ICLASS_READMODE	1	ICLASS_READMODE_UID ICLASS_READMODE_PAC
AT55_BITRATE	1	8 to 128 as multiple of 2
AT55_OPTIONS	1	One of the following definitions: AT55_OPT_SEQUENCE_NONE AT55_OPT_SEQUENCE_TERMINATOR AT55_OPT_SEQUENCE_STARTMARKER
CCID_MAX_SLOT_INDEX	1	Specify index of last logical CCID slot
HITAG1S_TO	1	Values from 14 to 40
HITAG1S_T1	1	Values from 14 to 40
HITAG1S_TGAP	1	Values from 2 to 14
HITAG2_TO	1	Values from 14 to 40
HITAG2_T1	1	Values from 14 to 40
HITAG2_TGAP	1	Values from 2 to 14
ISO14443_BITRATE_TX	1	One of the following possible bitrates: ISO14443_BITRATE_106 ISO14443_BITRATE_212 ISO14443_BITRATE_424 ISO14443_BITRATE_848

Continued from last page:

ISO14443_BITRATE_RX	1	One of the following possible bitrates: ISO14443_BITRATE_106 ISO14443_BITRATE_212 ISO14443_BITRATE_424 ISO14443_BITRATE_848
USB_SUPPORTREMOTEWAKEUP	1	USB_SUPPORTREMOTEWAKEUP_OFF USB_SUPPORTREMOTEWAKEUP_ON
EM4102_OPTIONS	1	Bitwise OR of one or more of the following definitions: EM4102_OPTIONS_F64 EM4102_OPTIONS_F32
EM4150_OPTIONS	1	Bitwise OR of one or more of the following definitions: EM4150_OPTIONS_F64 EM4150_OPTIONS_F40
USB_SERIALNUMBER	1	USB_SERIALNUMBER_OFF USB_SERIALNUMBER_ON
USB_KEYBOARDREPEATRATE	1	Number of milliseconds per keyboard event
SEOS_TREATMENT	1	SEOS_TREATMENT_ICLASS SEOS_TREATMENT_ISO14443A
SUPPORT_CONFIGCARD	1	Either SUPPORT_CONFIGCARD_OFF or bitwise OR of one or more of the following definitions: SUPPORT_CONFIGCARD_ON SUPPORT_UPGRADECARD_ON
ISO14443_3_TDX_CRCCONTROL	1	0x00 or bitwise OR of one or more of the following definitions: ISO14443_3_TDX_CRCCTRL_TX ISO14443_3_TDX_CRCCTRL_RX
ISO7816_CONTROL	2	0x0000 or bitwise OR of one or more of the following definitions: ISO7816_HANDLE_ERROR_SIGNAL_ATR ISO7816_TRANSMIT_ERROR_CNT ISO7816_RECEIVE_ERROR_CNT ISO7816_VOLTAGE_SYNC_CARDS ISO7816_SUPPORT_EMVCO
PN5180_LPCD_THRESHOLD	1	Values from 0 to 255
PN5180_LPCD_SENSING_PERIOD	2	Values from 1 to 2690

Continued from last page:

USB_KEYBOARDLAYOUT	1	Following keyboard layouts are supported: USB_KEYBOARDLAYOUT_ENGLISH (default) USB_KEYBOARDLAYOUT_GERMAN USB_KEYBOARDLAYOUT_FRENCH
USB_KEYBOARDSENDALTCODES	1	Instead of sending the immediate scan code of a character, TWN4 sends the input by entering the corresponding ALT-code. Following options are available: USB_KEYBOARDSENDALTCODES_OFF (default) USB_KEYBOARDSENDALTCODES_ON
ENABLE_WATCHDOG	1	Turn on/off the watchdog timer. WATCHDOG_OFF WATCHDOG_ON (default)
SUPPORT_CONFIG_CARD_HID	1	Turn on/off support for HID config cards (ICLASS/SEOS) SUPPORT_CONFIG_CARD_HID_OFF SUPPORT_CONFIG_CARD_HID_ON (default)

38 System Errors

Here is a list of all error codes, which are generated by the firmware of TWN4. The error codes can be retrieved with function `GetLastError`.

In the current version of the firmware, storage functions (FS . . .) are generating such errors.

Error Code	Description
ERR_NONE	No error occurred.
ERR_OUTOFMEMORY	The execution of a function required more memory than was available.
ERR_ISALREADYINIT	There was a try to initialize a system module, which already was initialized.
ERR_NOTINIT	There was a try to use a function from a module, which is not initialized.
ERR_ISALREADYOPEN	There was a try to open a system resource, which is already is open.
ERR_NOTOPEN	There was a try to use a system resource, which must be opened before usage.
ERR_RANGE	A specified parameters exceeded the valid range of values.
ERR_PARAMETER	A specified parameters is not in set of valid parameters.
ERR_UNKNOWNSTORAGEID	A storage ID was specified, which is not known by the firmware.
ERR_WRONGINDEX	A index was specified, which was out of the valid range.
ERR_FLASHERASE	The erase of a section of the flash failed.
ERR_FLASHWRITE	The write to the flash memory failed.
ERR_SECTORNOTFOUND	A sector of the file system was not found.
ERR_STORAGEFULL	All sectors of the file system are occupied.
ERR_STORAGEINVALID	There is an error in the file system.

ERR_TRANSACTIONLIMIT	The limit of changes in the file system is reached, which is possible within one transactions.
ERR_UNKNOWNFS	The file system on the specified storage is not supported by the current firmware.
ERR_FILENOTFOUND	The specified file was not found.
ERR_FILEALREADYEXISTS	The specified file already exists.
ERR_ENDOFFILE	The end of the file was reached. There is no more data to be read. Note: This error code is generated even the system function returned successful execution.
ERR_STORAGENOTFOUND	The specified storage was not found, e.g. because it is not mounted.
ERR_STORAGEALREADYMOUNTED	The specified storage is already mounted.
ERR_ACCESSDENIED	The access to a file was denied, e.g. write access to a file in a storage, which is mounted as read only.
ERR_FILECORRUPT	The specified file is corrupt in terms of a corrupted file system.
ERR_INVALIDFILEENV	The specified environment is invalid.
ERR_INVALIDFILEID	The specified file ID is invalid.
ERR_RESOURCELIMIT	The maximum number of available resources have been occupied.

Please see file `twm4.sys.h` (which can be found in local directory `Tools\sys\` of the developer pack) for concrete numbers, which are behind the definitions.

39 Runtime Library

There is a couple of functions, which are not part of the firmware of TWN4. Instead, they are statically linked to the App.

There are several intentions for such functions:

- Provide functions instead of having similar code in each App.
- Provide an API at a higher level to simplify writing Apps.
- Maintain a degree of compatibility to TWN3.

39.1 Timer Functions

Include file: `apptools.h`

There are three functions, which implement a simple API, which allows triggering events after a specified time. The behaviour of the functions are similar to TWN3. Compared to TWN3, there is only one timer available. Therefore no timer ID must be specified. These timer functions are implemented using system function `GetSysTicks`.

39.1.1 StartTimer

Start the timer with a specified time.

```
void StartTimer(unsigned long Duration)
```

Parameters:

`unsigned long` Duration Time in milliseconds, till function `TestTimer` returns true.

Return: None.

39.1.2 StopTimer

Stop the timer, thus function `TestTimer` will never return true.

```
void StopTimer(void);
```

Parameters: None.

Return: None.

39.1.3 TestTimer

Test, if the timer reached the timeout which was programmed by function StartTimer.

```
bool TestTimer(void);
```

Parameters: None.

Return: If the timeout has been reached, the function returns `true`, otherwise, it return `false`.

39.2 Host Communication

Include file: `apptools.h`

There are several function which implement a simplified interface for direct write to the host. The host is defined to be a communication channel, where all communication takes place. This removes the requirement to specify the communication channel every time when communication should take place.

For a more sophisticated kind of communication (binary, bidirectional), it is suggested to directly use the I/O functions from the firmware.

39.2.1 HostTestByte

Use this function to check if there is a byte available in the input buffer of the host-channel.

```
bool HostTestByte(void)
```

Parameters: None.

Return: If there is a byte available, the return value is `true`, otherwise it is `false`.

39.2.2 HostReadByte

Use this function to read a byte from the input buffer of the host-channel. If there is no byte available, the function blocks until there is one.

```
byte HostReadByte(void)
```

Parameters: None.

Return: The byte which was read from the input buffer.

39.2.3 HostTestChar

Test if a character is available from the host. The character can be read with function HostReadChar.

```
bool HostTestChar(void)
```

Parameters: None.

Return: `true` if at least one character arrived, otherwise `false`.

39.2.4 HostReadChar

Receive a character from the host. This is a blocking function. This means, it is waiting, till a character is available.

```
char HostReadChar(void)
```

Parameters: None.

Return: The character, which was read from the host.

39.2.5 HostWriteByte

Use this function to send one byte to the host through the actually configured host-channel. If the output buffer is completely occupied, the function blocks until there is enough space.

```
void HostWriteByte(byte Byte)
```

Parameters:

`byte` Byte The byte to be sent.

Return: None.

39.2.6 HostWriteChar

Send a character to the host. This is a blocking function. This means, it is waiting, till there is storage in the output buffer, to transmit the character.

```
void HostWriteChar(char Char)
```

Parameters:

`char` Char The character to be sent to the host.

Return: None.

39.2.7 HostWriteString

Send a string to the host. The string must be terminated with a null character. The string is sent without the null character.

```
void HostWriteString(const char *String)
```

Parameters:

`const char` *String Pointer to the string to be sent.

Return: None.

39.2.8 HostWriteRadix

Send a number to the host in ASCII format. The number is specified by an array of bytes containing the binary data.

```
void HostWriteRadix(const byte *ID,int BitCnt,int DigitCnt,int Radix)
```

Parameters:

<code>const byte *ID</code>	Pointer to the array of bytes.
<code>int BitCnt</code>	Number of bits stored in the array.
<code>int DigitCnt</code>	Specifies the minimum number of digits, the output should contain. If <code>DigitCnt</code> is 0, then at least 1 digit is sent. If <code>DigitCnt</code> is greater than the actual width of the number to be converted, then the number is preceded by zeros.
<code>int Radix</code>	Base for conversion from binary to ASCII. Use: • 2 for binary conversion • 8 for octal conversion • 10 for decimal conversion • 16 for hexadecimal conversion

Return: None.

39.2.9 HostWriteBin

Send a binary number to the host in ASCII format. The number is specified by an array of bytes containing the binary data.

```
void HostWriteBin(const byte *ID,int BitCnt,int DigitCnt)
```

Parameters:

<code>const byte *ID</code>	Pointer to the array of bytes.
<code>int BitCnt</code>	Number of bits stored in the array.
<code>DigitCnt</code>	Specifies the minimum number of digits, the output should contain. If <code>DigitCnt</code> is 0, then at least 1 digit is sent. If <code>DigitCnt</code> is greater than the actual width of the number to be converted, then the number is preceded by zeros.

Return: None.

39.2.10 HostWriteDec

Send a decimal number to the host in ASCII format. The number is specified by an array of bytes containing the binary data.

```
void HostWriteDec(const byte *ID,int BitCnt,int DigitCnt)
```

Parameters:

<code>const byte *ID</code>	Pointer to the array of bytes.
<code>int BitCnt</code>	Number of bits stored in the array.
<code>DigitCnt</code>	Specifies the minimum number of digits, the output should contain. If <code>DigitCnt</code> is 0, then at least 1 digit is sent. If <code>DigitCnt</code> is greater than the actual width of the number to be converted, then the number is preceded by zeros.
<u>Return:</u>	None.

39.2.11 HostWriteHex

Send a hexadecimal number to the host in ASCII format. The number is specified by an array of bytes containing the binary data.

```
void HostWriteHex(const byte *ID,int BitCnt,int DigitCnt)
```

Parameters:

<code>const byte *ID</code>	Pointer to the array of bytes.
<code>int BitCnt</code>	Number of bits stored in the array.
<code>DigitCnt</code>	Specifies the minimum number of digits, the output should contain. If <code>DigitCnt</code> is 0, then at least 1 digit is sent. If <code>DigitCnt</code> is greater than the actual width of the number to be converted, then the number is preceded by zeros.
<u>Return:</u>	None.

39.2.12 HostWriteVersion

Send the firmware version to the host. This function is sending the result of function `GetVersionString` to the host.

```
void HostWriteVersion(void)
```

<u>Parameters:</u>	None.
<u>Return:</u>	None.

39.3 Beep Functions

Include file: `apptools.h`

The beep functions implement a simplified API around the system function `Beep`.

39.3.1 SetVolume

Set the beeper volume. The default volume is 0.

```
void SetVolume(int NewVolume)
```

Parameters:

`int` NewVolume Specify the volume in percent from 0 to 100.

Return: None.

39.3.2 GetVolume

Read current volume.

`int` GetVolume(`void`);

Parameters: None.

Return: Current volume in arange from 0 to 100.

39.3.3 BeepLow

Perform a beep at a frequency of BEEP_FREQUENCY_LOW (2057 Hz) with a duration of 50 milliseconds.

`void` BeepLow(`void`)

Parameters: None.

Return: None.

39.3.4 BeepHigh

Perform a beep at a frequency of BEEP_FREQUENCY_HIGH (2400 Hz) with a duration of 50 milliseconds. This is meant to be the standard signal for a successful operation, e.g. read of a transponder.

`void` BeepHigh(`void`)

Parameters: None.

Return: None.

39.4 Compatibility to TWN3

Include file: `apptools.h`

Currently, there is one function for maintaining 100% backward compatibility to TWN3 applications.

39.4.1 ConvertTagTypeToTWN3

This functions converts a tag type from the TWN4 system to TWN3 system. Due to the fact that TWN4 covers a broader range of transponders, the situation might occure, that a conversion is not possible. Under that circumstance the TWN3 value `TAGTYPE_NONE` (0) is returned.

`int` ConvertTagTypeToTWN3(`int` TagTypeTWN4)

Parameters:

`int` TagTypeTWN4 Tag type as returned e.g. by TWN4 system function SearchTag.

Return: Corresponding tag type as it would be returned by TWN3 system function TagSearch.

39.5 Simple Protocol

Include file: `prs.h`

The Simple Protocol is the standard protocol for building solutions, which need operation of TWN4, which is controlled by the host.

There is a set of functions and definitions, which allow to implement an App, which runs the Simple Protocol. There are some options, which have influence on some details of the Simple Protocol (ASCII/binary mode, CRC). Furthermore, these functions allow to specify custom communication channel and configure the host interface before starting the communication.

The simplest App for using these functions could be written as follows:

```
#include <twn4.sys.h>
#include <prs.h>

int main(void)
{
    InitSimpleProtocol(GetHostChannel(), PRS_COMM_MODE_ASCII | PRS_COMM_CRC_OFF);
    while (true)
    {
        if (SimpleProtoTestCommand())
        {
            SimpleProtoExecuteCommand();
            SimpleProtoSendResponse();
        }
    }
}
```

39.5.1 SimpleProtoInit

Use this function to prepare the Simple Protocol for operation. Before starting this function, it is possible to e.g. prepare a serial port with appropriate communication parameters.

`bool` SimpleProtoInit(`int` Channel, `int` Mode)

Parameters:

`int` Channel This parameter specifies the communication channel for the Simple Protocol. This can be one the channels as defined by the system I/O functions.

`int` Mode This parameter specified the mode of communication. It is a or-operation, which combines mode (PRS_COMM_MODE_ASCII or PRS_COMM_MODE_BINARY and CRC (COMM_CRC_OFF or PRS_COMM_CRC_ON).

Return: This functions return true, if initialization was successful. Otherwise it returns false.

39.5.2 SimpleProtoTestCommand

This is a none-blocking function, which polls for the availability of a command from the host. If the function returns true, a command is available. The command is stored in the global variables SimpleProtoMessage and SimpleProtoMessageLength.

```
bool SimpleProtoTestCommand(void)
```

Parameters: None.

Return: This functions return true, if a command became available. Otherwise it returns false.

39.5.3 SimpleProtoExecuteCommand

This function executes a command stored in the global variables SimpleProtoMessage and SimpleProtoMessageLength. After execution of the command, these variables contain the response to be sent to the host.

```
void SimpleProtoExecuteCommand(void)
```

Parameters: None.

Return: None.

39.5.4 SimpleProtoSendResponse

This function sends a response stored in the global variables SimpleProtoMessage and SimpleProtoMessageLength to the host.

```
void SimpleProtoSendResponse(void)
```

Parameters: None.

Return: None.

40 Compatibility of TWN4 MultiTech Mini Reader

Due to reduced functionality of TWN4 MultiTech Mini Reader, several API functions are not available. If an API function is called, which is not supported by TWN4 MultiTech Mini Reader the device stops execution of the App and enters exception state (diagnostic LED is flashing three times).

API	Supported	Remark
System Functions	Yes	
I/O Functions	Yes	COM2 is not supported
Memory Functions	Yes	
Peripheral Functions	Yes	Support of GPIO0 to GPIO3 only, Beep is doing delay only
Conversion Functions	Yes	
I2C Functions	No	
RF Functions	Yes	
HITAG 1 and HITAG S Functions	No	
HITAG 2 Functions	No	
EM4x50 Functions	No	
AT55xx Functions	No	
TILF (TIRIS) Functions	No	
LEGIC Functions	No	
MIFARE Classic Functions	Yes	
MIFARE Ultralight (-C) Functions	Yes	
ISO15693 Functions	Yes	
Cryptographic Functions	Yes	
DESFire Functions	Yes	
Contact Card Functions	Yes	SAM1 only
iCLASS Functions	Yes	
ISO14443 Functions	Yes	
System Parameters	Yes	
Runtime Library	Yes	

41 Disclaimer

ELATEC GmbH reserves the right to change any information or data in this document without prior notice. The distribution and the update of this document is not controlled. ELATEC GmbH declines all responsibility for the use of product with any other specifications but the ones mentioned above. Any additional requirement for a specific custom application has to be validated by the customer himself at his own responsibility. Where application information is given, it is only advisory and does not form part of the specification.

All referenced brands, product names, service names and trademarks mentioned in this document are the property of their respective owners.